

Syntax Analysis (Parsing) - Complete Interview Guide

Compiler Design - Phase 2. This guide takes you from the basics of grammars all the way to LALR parser generators, with an obsessive interview focus. Every topic uses the same 14-part structure so you can revise fast.

How Syntax Analysis Fits in a Compiler

```

Source code
  |
  v
[ Lexical Analysis ] --> stream of tokens      (Phase 1: "words")
  |
  v
[ Syntax Analysis ] --> parse tree / AST      (Phase 2: "grammar/sentences")
<-- THIS GUIDE
  |
  v
[ Semantic Analysis] --> annotated tree      (Phase 3: "meaning")
  |
  v
[ Intermediate Code ] -> [ Optimization ] -> [ Code Generation ]

```

- **Lexer** groups characters into tokens (`int`, `x`, `=`, `5`, `;`).
- **Parser** (syntax analyzer) checks that the token stream forms a valid program according to a **grammar**, and builds a **parse tree / AST**.
- The parser is where **Context-Free Grammars**, **derivations**, and **parsing algorithms (LL / LR)** live.

Table of Contents

1. [Context-Free Grammar \(CFG\)](#)
2. [Parse Trees](#)
3. [Leftmost and Rightmost Derivation](#)
4. [Ambiguous Grammar](#)
5. [Eliminating Left Recursion](#)
6. [Left Factoring](#)
7. [Top-Down Parsing](#)
8. [Recursive-Descent Parsing](#)
9. [Predictive Parsing](#)
10. [FIRST and FOLLOW Sets](#)
11. [LL\(1\) Parsing](#)
12. [Bottom-Up Parsing](#)
13. [Shift-Reduce Parsing](#)
14. [Handle and Viable Prefix](#)
15. [LR Parsing Basics](#)

16. [SLR Parser](#)
 17. [CLR Parser \(Canonical LR\)](#)
 18. [LALR Parser](#)
 19. [Parser Conflicts](#)
 20. [Yacc / Bison Basics](#)
 21. [Generalized LR \(GLR\) Parsing](#)
 22. [Master Cheat Sheet \(All Topics\)](#)
-

1. Context-Free Grammar (CFG)

1. Overview

Definition: A Context-Free Grammar (CFG) is a formal way to describe the *syntax* (structure) of a language using a set of recursive **production rules**. Formally, a CFG is a 4-tuple **G = (V, T, P, S)**:

- **V** = set of **non-terminals** (variables / syntactic categories, e.g. *Expr*, *Stmt*).
- **T** = set of **terminals** (actual tokens/symbols, e.g. *id*, *+*, *(*, *)*).
- **P** = set of **productions** of the form **A -> alpha**, where **A** is a single non-terminal and **alpha** is a string of terminals and non-terminals.
- **S** = the **start symbol** (a special non-terminal in V).

"Context-free" means the **left-hand side is always a single non-terminal** - a rule **A -> alpha** can be applied regardless of the symbols surrounding **A** (no context needed).

Why it matters: Programming language syntax (nesting of expressions, blocks, function calls, balanced parentheses) is inherently recursive. Regular expressions / finite automata **cannot** count nesting (they cannot recognize **aⁿ bⁿ**), but CFGs can. So CFGs are the natural formalism for the parser.

Where it is used in real systems:

- Every compiler/interpreter front-end (C, Java, Python, Rust grammars are CFGs, mostly).
- Parser generators: **Yacc/Bison**, **ANTLR**, **JavaCC** take a CFG and generate a parser.
- Data formats: JSON, XML, YAML, SQL, protobuf **.proto** files are defined by CFG-like grammars.
- Markup and query languages, configuration DSLs, expression evaluators.

Why interviewers ask about it: It tests whether you understand the theoretical foundation of parsing, the Chomsky hierarchy (why not regex?), and can read/write grammar rules - a prerequisite for every other topic here.

2. Core Idea

Intuition: A grammar is a *recipe* for generating (or validating) all the strings in a language. You start from the start symbol and repeatedly **replace a non-terminal by one of its right-hand sides** until only terminals remain. Any string you can produce this way is "in the language."

Real-world analogy: Think of English sentence structure:

```

Sentence -> Subject Verb Object
Subject  -> "The" Noun
Object   -> "the" Noun
Noun     -> "cat" | "dog" | "mat"
Verb     -> "chased" | "saw"

```

From **Sentence** you can *derive* "The cat chased the dog". The grammar defines which sequences of words are grammatically valid, independent of meaning.

Small example (arithmetic expressions):

```

E -> E + E
E -> E * E
E -> ( E )
E -> id

```

Here $V = \{E\}$, $T = \{+, *, (,), id\}$, $S = E$. This generates strings like `id`, `id + id`, `id * ( id + id )`.

Step-by-step derivation of `id + id * id`:

```

E
=> E + E           (E -> E + E)
=> id + E           (E -> id)
=> id + E * E       (E -> E * E)
=> id + id * E      (E -> id)
=> id + id * id     (E -> id)

```

3. Important Subtopics

3.1 Terminals vs Non-terminals

- **What:** Terminals are the leaf symbols (tokens from the lexer); non-terminals are abstract categories that get expanded.
- **Why it matters:** Confusing them is the #1 beginner mistake. Only non-terminals appear on the LHS of a production.
- **Example:** In `Stmt -> if ( Expr ) Stmt`, terminals are `if ( )`, non-terminals are `Stmt`, `Expr`.
- **Interview angle:** "Given this rule, list the terminals and non-terminals."

3.2 Productions and the empty string (epsilon)

- **What:** A production may be `A -> epsilon` (ϵ), meaning `A` can derive the empty string.
- **Why it matters:** ϵ -productions model *optional* constructs (optional `else`, optional argument list). They heavily affect FIRST/FOLLOW and nullability.
- **Example:** `ArgListOpt -> ArgList | epsilon`.
- **Interview angle:** Tracing nullable non-terminals for FIRST/FOLLOW computation.

3.3 Derivation and the language of a grammar $L(G)$

- **What:** $L(G)$ = set of all terminal strings derivable from S . A string w is in $L(G)$ iff $S \Rightarrow^* w$.
- **Why it matters:** Parsing = "is w in $L(G)$, and if so, how was it derived?"
- **Interview angle:** "What language does this grammar generate?"

3.4 Chomsky Hierarchy placement

- **What:** Type-0 (unrestricted) $\supset$ Type-1 (context-sensitive) $\supset$ **Type-2 (context-free)** $\supset$ Type-3 (regular).
- **Why it matters:** Explains why lexers (regular) and parsers (context-free) are separate phases. CFGs are strictly more powerful than regex.
- **Example:** $\{a^n b^n \mid n \geq 1\}$ is context-free but not regular; $\{a^n b^n c^n\}$ is not even context-free.
- **Interview angle:** "Why can't a regular expression parse balanced parentheses?"

3.5 Sentential form vs sentence

- **What:** A **sentential form** is any string of terminals AND non-terminals derivable from S . A **sentence** is a sentential form with only terminals.
- **Why it matters:** Vocabulary used everywhere in derivations and LR parsing (viable prefixes are prefixes of right-sentential forms).
- **Interview angle:** "Is $E + id$ a sentence or a sentential form?" (Sentential form - it has a non-terminal E .)

4. Real-World Example

JSON parsing in a backend server. When your API receives a request body, the JSON parser uses a CFG like:

```
value    -> object | array | string | number | "true" | "false" | "null"
object   -> '{' members '}' | '{' '}'
members  -> pair | pair ',' members
pair     -> string ':' value
array    -> '[' elements ']' | '[' ']'
elements -> value | value ',' elements
```

The recursion ($value$ contains $object$ contains $value$) is exactly why a CFG - not a regex - is needed: JSON can nest arbitrarily deep. Libraries like Jackson, `serde_json`, and Python's `json` module implement a parser for this grammar.

5. Diagrams / Mental Models

A CFG as a 4-tuple:

$G = ($	$V,$	$T,$	$P,$	$S)$
	$ $	$ $	$ $	$ $
non-terminals		terminals	productions	start symbol
$\{E\}$		$\{+, *, (,), id\}$	$\{E \rightarrow E+E, \dots\}$	E

Generation vs Recognition:

Generation (grammar's job): $S \Rightarrow \dots \Rightarrow w$ (produce strings)
 Recognition (parser's job): given w , find $S \Rightarrow \dots \Rightarrow w$ (or reject)

6. Common Interview Questions

Q1. What is a context-free grammar? Define its 4 components.

- Answer: $G = (V, T, P, S)$ - non-terminals, terminals, productions (each LHS is one non-terminal), start symbol.
- Key points: emphasize "single non-terminal on LHS" = the "context-free" property.
- Common mistake: forgetting that LHS must be exactly one non-terminal.

Q2. Why "context-free"? What would context-sensitive mean?

- Answer: In CFG, $A \rightarrow \alpha$ applies regardless of surroundings. In context-sensitive, rules look like $\beta A \gamma \rightarrow \beta \alpha \gamma$ - the replacement depends on neighbors.
- Key points: link to Chomsky hierarchy.
- Common mistake: thinking "context-free" means "no ambiguity" (unrelated).

Q3. Give a language that is context-free but not regular.

- Answer: $\{a^n b^n \mid n \geq 0\}$. A finite automaton can't remember n ; a CFG can via recursion $S \rightarrow aSb \mid \epsilon$.
- Common mistake: giving a^*b^* (that IS regular).

Q4. Can a CFG describe all of a programming language?

- Answer: The *syntax* mostly yes, but not context-sensitive rules like "variable must be declared before use" or type checking - those need semantic analysis. Also $a^n b^n c^n$ is beyond CFG.
- Key points: syntax = CFG, semantics = beyond CFG.

Q5. What is the difference between a sentence and a sentential form?

- Answer: Sentence = only terminals; sentential form = terminals and/or non-terminals, both derivable from S .

Q6. What does $L(G)$ mean?

- Answer: The set of all terminal strings derivable from the start symbol.

Q7. Write a grammar for balanced parentheses.

- Answer: $S \rightarrow (S) S \mid \epsilon$ (or $S \rightarrow SS \mid (S) \mid \epsilon$).
- Common mistake: writing one that doesn't allow concatenation like $(())$.

Q8. What is an epsilon-production and why is it useful?

- Answer: $A \rightarrow \epsilon$; models optional constructs. Impacts nullable/FIRST/FOLLOW.

Q9. Is $E \rightarrow E + E$ a valid CFG production? Any downside?

- Answer: Valid, but this grammar is ambiguous and left-recursive - fine as a CFG, problematic for certain parsers.

Q10. How is a CFG different from a regular expression / DFA?

- Answer: CFGs allow recursion / a stack-like memory (matching nested structures); regex/DFA have finite memory only.
- Common mistake: saying "CFGs are just fancier regex" - they are strictly more powerful.

7. Deep-Dive Questions

D1. Prove informally why $a^n b^n$ needs a stack (CFG/PDA) not a DFA.

- A DFA has finitely many states, so for large enough n two different prefixes a^i and a^j ($i \neq j$) land in the same state; the DFA then accepts $a^i b^j$, which is wrong. A pushdown automaton pushes each a and pops on each b , matching counts.

D2. Every regular language is context-free. Is the converse true?

- No. $\{a^n b^n\}$ is context-free but not regular. $\text{Regular} \subset \text{context-free}$ (strict subset).

D3. What class of automaton recognizes exactly the context-free languages?

- Non-deterministic **Pushdown Automata (PDA)**. Deterministic PDAs recognize only the deterministic CFLs (a strict subset) - which relates to why LR grammars matter.

D4. Are context-free languages closed under intersection?

- No. $\{a^n b^n c^m\}$ and $\{a^m b^n c^n\}$ are both CFL, but their intersection $\{a^n b^n c^n\}$ is not context-free. (They ARE closed under union, concatenation, Kleene star.)

D5. Can you always convert an ambiguous grammar to an unambiguous one for the same language?

- Not always. Some CFLs are **inherently ambiguous** - no unambiguous grammar exists. For most programming languages, though, we can find an unambiguous grammar.

8. Comparison Tables

CFG vs Regular Expression

Aspect	Regular Expression / DFA	Context-Free Grammar
Memory model	Finite states only	Stack (recursion)
Power	Regular languages	Context-free languages (superset)
Nesting/balancing	Cannot count nesting	Can ((S) , $a^n b^n$)
Used in compiler	Lexer (tokens)	Parser (syntax)
Automaton	Finite automaton	Pushdown automaton

Chomsky Hierarchy

Type	Name	Grammar form (LHS -> RHS)	Automaton
0	Unrestricted	any -> any	Turing machine
1	Context-sensitive	$\beta A \gamma \rightarrow \beta \alpha \gamma$	Linear bounded automaton
2	Context-free	$A \rightarrow \alpha$ (single NT on LHS)	Pushdown automaton
3	Regular	$A \rightarrow aB$ or $A \rightarrow a$	Finite automaton

9. Common Mistakes

- Putting more than one symbol (or a terminal) on the LHS of a production - that's no longer a CFG.
- Confusing terminals with non-terminals in a rule.
- Thinking "context-free" implies "unambiguous" - unrelated concepts.
- Believing CFGs can enforce "declare before use" or type rules - they cannot (that's semantic).
- Writing $S \rightarrow a*b*$ style regex operators inside a CFG rule (use recursion instead).

10. Edge Cases / Special Cases

- **Empty language vs language containing epsilon:** $L(G)$ may be empty or may contain the empty string (if $S \Rightarrow^* \epsilon$).
- **Useless symbols:** non-terminals that are non-generating (can't derive any terminal string) or unreachable (never derived from S). Grammar cleanup removes them.
- **Left recursion / cycles:** $A \rightarrow A$ (a cycle) is legal CFG but useless and breaks many parsers.
- **Inherently ambiguous languages:** cannot be given an unambiguous grammar at all.

11. How to Explain in Interview

"A context-free grammar is a formal 4-tuple (V, T, P, S) that describes a language's syntax with recursive production rules, where each rule rewrites a *single* non-terminal into a string of terminals and non-terminals. It's called context-free because a rule applies no matter what surrounds the non-terminal. CFGs sit at Type-2 of the Chomsky hierarchy - strictly more powerful than regular expressions because recursion gives them a stack, letting them describe nested structures like balanced parentheses or arithmetic expressions, which is exactly what programming language parsers need."

12. Quick Revision Notes

- **CFG = (V, T, P, S) .** LHS of every production = exactly ONE non-terminal.
- **$L(G)$** = all terminal strings derivable from S.
- **Sentence** = terminals only; **sentential form** = terminals + non-terminals.
- **Type-2** in Chomsky hierarchy; recognized by **pushdown automata**.
- CFG > regex (can handle nesting/recursion). Regex used for lexing, CFG for parsing.
- **epsilon-production** $A \rightarrow \epsilon$ models optional parts.
- CFGs can't do "declare before use", type checking, or $a^n b^n c^n$.
- **Trap:** context-free $\neq$ unambiguous; regular languages ARE a subset of CFL.

13. Practice Tasks

1. Write a CFG for all strings of balanced parentheses over $\{ (,) \}$.
2. Write a CFG for $\{ a^n b^n \mid n \geq 1 \}$ and for $\{ a^n b^m \mid n \geq m \geq 0 \}$.
3. Write a CFG for signed integers, then for simple `if/else` statements.
4. Given $S \rightarrow aSb \mid ab$, list 3 strings in $L(G)$ and prove `aabb` is derivable.
5. Identify terminals and non-terminals in a small expression grammar.
6. Write a CFG for a comma-separated list of identifiers (allow empty list).
7. Argue why $\{ ww \mid w \in \{a,b\}^* \}$ is not context-free.

14. Final Cheat Sheet

- **Core definition:** CFG = (V, T, P, S); each production rewrites one non-terminal.
- **Why it matters:** Formal basis for all parsing; handles recursion/nesting regex can't.
- **Most asked:** Define CFG; why "context-free"; CFG vs regex; give a non-regular CFL.
- **Common comparison:** CFG vs regular expression; Chomsky hierarchy levels.
- **One-line answer:** "A CFG is a 4-tuple (V,T,P,S) whose recursive single-non-terminal rules define a language's syntax - Type-2, more powerful than regex because recursion gives it a stack."

2. Parse Trees

1. Overview

Definition: A **parse tree** (concrete syntax tree) is a tree representation of how a string is derived from a grammar. The **root** is the start symbol, **internal nodes** are non-terminals, **leaves** are terminals (read left to right = the input string / **yield**), and each internal node's children are exactly the RHS of the production applied to it.

Why it matters: It makes the *hierarchical structure* of the input explicit - precedence, grouping, and nesting all become tree shape. The parser's real output (an **AST**, a condensed parse tree) drives every later phase (semantic checks, code generation).

Where it is used in real systems:

- ASTs in compilers/interpreters (Babel, TypeScript, Clang, CPython all build them).
- Linters, formatters (Prettier), refactoring tools, and IDEs operate on parse/syntax trees.
- Query planners parse SQL into trees; template engines and expression evaluators too.

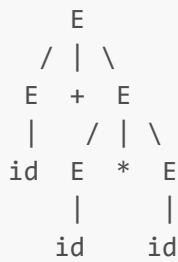
Why interviewers ask about it: Parse trees are the visual bridge between "grammar rules" and "structure." Drawing one, and knowing parse tree vs AST vs derivation, is a very common exam/interview task.

2. Core Idea

Intuition: A derivation is a *sequence* of rewrite steps; a parse tree is the *2D picture* of that derivation that throws away the order in which non-terminals were expanded. Many derivations (leftmost, rightmost, etc.) can share the **same** parse tree.

Real-world analogy: A sentence diagram in grammar class - "The cat sat" broken into subject/verb phrases as a tree. The tree shows structure; the order you drew the branches doesn't matter.

Small example: For $E \rightarrow E + E \mid E * E \mid id$ and input $id + id * id$, one parse tree:



Yield (leaves L-to-R) = $id + id * id$. This particular tree groups $id * id$ together, i.e. $id + (id * id)$.

Step-by-step (building a parse tree):

1. Start with root = start symbol S .
2. Pick a production $A \rightarrow X_1 X_2 \dots X_k$; add children $X_1..X_k$ under A .
3. Recurse on each non-terminal child.
4. Stop when all leaves are terminals; read leaves left-to-right = the input.

3. Important Subtopics

3.1 Parse Tree vs Abstract Syntax Tree (AST)

- **What:** Parse tree keeps *every* grammar symbol (including punctuation, single-child chains). AST keeps only *semantically meaningful* nodes.
- **Why it matters:** Compilers use ASTs (smaller, cleaner). (id) parse tree has parens as nodes; the AST is just id .
- **Example:** $3 + 4$: parse tree has $E \rightarrow E + E \rightarrow \dots$; AST is just $(+ 3 4)$.
- **Interview angle:** "Difference between parse tree and AST?"

3.2 Yield of a parse tree

- **What:** The string formed by concatenating leaves left to right.
- **Why it matters:** A parse tree is valid for input w iff its yield equals w and its root is S .
- **Interview angle:** "What is the yield of this tree?"

3.3 Relationship to derivations

- **What:** Each parse tree corresponds to possibly many derivations; each leftmost (or rightmost) derivation corresponds to exactly one parse tree.
- **Why it matters:** Ambiguity is defined via parse trees, not derivation order.
- **Interview angle:** "How many parse trees vs how many derivations for this string?"

3.4 Precedence and associativity via tree shape

- **What:** Which operator sits *lower* in the tree binds tighter (evaluated first).
- **Why it matters:** Grammar design encodes precedence by tree structure.
- **Example:** In $id + id * id$, $*$ lower than $+$ $\Rightarrow$ multiplication first.
- **Interview angle:** "Show how this parse tree enforces $*$ over $+$."

4. Real-World Example

A browser's JavaScript engine (e.g., V8). When JS source is loaded, the parser builds an AST (a trimmed parse tree). That AST is what the interpreter walks / the JIT compiles. Tools like ESLint traverse the same AST to find issues, and Prettier reprints it as formatted code. The tree shape directly encodes operator precedence so `a + b * c` evaluates `b * c` first.

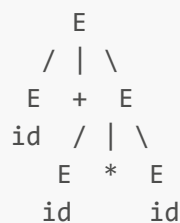
5. Diagrams / Mental Models

Derivation vs Parse Tree (same structure, different views):

Leftmost derivation: $E \Rightarrow E+E \Rightarrow id+E \Rightarrow id+E * E \Rightarrow id+id * E \Rightarrow id+id * id$

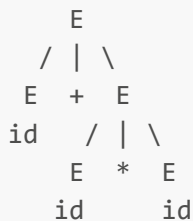
_____ collapses into _____/

Parse tree:



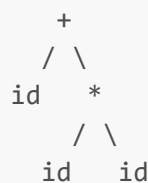
Parse tree vs AST:

Parse tree (concrete)



=>

AST (abstract)



6. Common Interview Questions

Q1. What is a parse tree?

- Answer: Tree showing the derivation of a string; root = start symbol, leaves = terminals whose left-to-right yield is the input, children = RHS of applied production.
- Common mistake: saying leaves can be non-terminals.

Q2. Parse tree vs derivation - what's the difference?

- Answer: Derivation is an ordered sequence of rewrites; parse tree is the order-independent structure. One tree = many derivations.

Q3. Parse tree vs AST?

- Answer: Parse tree includes all grammar symbols and single-productions; AST removes syntactic noise, keeping only meaningful structure. Compilers use ASTs.
- Common mistake: using the terms interchangeably.

Q4. Can one string have two parse trees? What does that mean?

- Answer: Yes - it means the grammar is **ambiguous** for that string.

Q5. What is the yield of a parse tree?

- Answer: The concatenation of leaves left to right = the input string.

Q6. How does a parse tree encode operator precedence?

- Answer: Operators deeper in the tree are applied first; grammar structure controls the depth.

Q7. Do leftmost and rightmost derivations give the same parse tree?

- Answer: For an unambiguous grammar, yes - the same tree. They differ only in the order of expansion.

Q8. Every terminal string in $L(G)$ has at least one parse tree - true?

- Answer: True by definition of derivable; if it's in $L(G)$ there is a derivation, hence a tree.

Q9. Given a grammar and a string, draw the parse tree.

- Key points: expand start symbol top-down, match leaves to input.

Q10. Why do compilers convert parse trees to ASTs?

- Answer: Efficiency and clarity - fewer nodes, no redundant punctuation/chains, easier to traverse for semantic analysis and codegen.

7. Deep-Dive Questions

D1. Two different derivations, same parse tree - reconcile.

- The tree abstracts away *which non-terminal you chose to expand next*. Leftmost and rightmost pick different orders but produce identical parent-child structure, hence the same tree.

D2. Two different parse trees for one string - what's guaranteed about derivations?

- There exist two distinct leftmost derivations (and two distinct rightmost derivations). This is the formal definition of ambiguity.

D3. Can an AST lose information that the parse tree had? Does it matter?

- Yes - it drops parentheses, some keywords, chain productions. It doesn't matter because that information is either redundant (parens already reflected in structure) or recoverable; semantics are preserved.

D4. Is the number of parse trees for a string always finite?

- If the grammar has no cycles ($A \Rightarrow^+ A$) and no ϵ -loops, yes. Cycles can create infinitely many derivations but they collapse structurally; well-formed grammars avoid this.

D5. How would you build a parse tree during recursive-descent parsing?

- Each parse function creates a node for its non-terminal and attaches child nodes returned by the sub-calls it makes; the call tree mirrors the parse tree.

8. Comparison Tables

Parse Tree vs AST vs Derivation

Aspect	Derivation	Parse Tree	AST
Form	Sequence of strings	Tree	Tree (condensed)
Order captured	Yes (LM/RM)	No	No
Punctuation/keywords	n/a	Kept	Often dropped
Single-child chains	n/a	Kept	Collapsed
Used by	Theory/proofs	Teaching, some tools	Compilers, linters

Parse Tree vs Syntax Tree (AST) quick view

Feature	Parse (concrete) tree	Abstract syntax tree
Size	Larger	Smaller
Interior nodes	Non-terminals	Operators/constructs
Leaves	Terminals (incl. <code>(,)</code>)	Operands only
Reflects grammar exactly	Yes	No (abstracted)

9. Common Mistakes

- Treating a parse tree and a derivation as the same thing.
- Using "parse tree" and "AST" interchangeably.
- Drawing leaves as non-terminals or reordering leaves (yield must equal input, in order).
- Thinking different derivation *orders* imply ambiguity - only different *trees* do.
- Forgetting to include terminals like parentheses as leaf nodes in a concrete parse tree.

10. Edge Cases / Special Cases

- epsilon leaves:** an `A -> epsilon` node has a single ϵ leaf (contributes nothing to the yield).
- Single-production chains** (`A -> B -> C -> id`): long thin branches, usually collapsed in the AST.
- Ambiguous grammar:** same yield, multiple trees - must pick one via precedence/associativity rules.
- Left/right recursion:** produces left-leaning vs right-leaning trees, affecting associativity.

11. How to Explain in Interview

"A parse tree is the tree-shaped record of how the grammar derives an input: the root is the start symbol, internal nodes are non-terminals expanded by their productions, and the leaves read left to right give back the exact input. It captures structure - precedence and nesting - independent of the order in which we expanded non-terminals, so many derivations map to one tree. In real compilers we usually build the leaner

abstract syntax tree, which drops punctuation and trivial chains but keeps the meaningful structure for later phases."

12. Quick Revision Notes

- Parse tree: root = start symbol, leaves = terminals, yield = input.
- One parse tree = many derivations; LM/RM derivation = exactly one tree.
- **Two parse trees for one string => ambiguous grammar.**
- AST = condensed parse tree (compilers use it).
- Tree depth encodes precedence (deeper = tighter binding).
- **Trap:** different derivation order != ambiguity; different *trees* = ambiguity.

13. Practice Tasks

1. For $E \rightarrow E + E \mid E * E \mid id$, draw both parse trees of $id + id * id$.
2. Convert one of those parse trees into its AST.
3. Draw the parse tree for $(id + id)$ and mark the parenthesis leaves.
4. For grammar $S \rightarrow aSb \mid ab$, draw the parse tree of $aabb$.
5. Given a parse tree, write out its leftmost derivation.
6. Show a grammar where the parse tree is left-leaning (left recursion) vs right-leaning.

14. Final Cheat Sheet

- **Core definition:** Tree of a derivation; root = start symbol, leaves' yield = input.
- **Why it matters:** Makes structure/precedence explicit; AST drives compilation.
- **Most asked:** Parse tree vs AST vs derivation; two trees => ambiguity.
- **Common comparison:** Parse tree vs AST; derivation vs tree.
- **One-line answer:** "A parse tree is the structural, order-independent picture of a derivation whose leaf yield is the input; two distinct trees for one string means the grammar is ambiguous."

3. Leftmost and Rightmost Derivation

1. Overview

Definition: A **derivation** is the step-by-step process of rewriting the start symbol into a terminal string using productions.

- **Leftmost derivation (LMD):** at each step, replace the **leftmost** non-terminal.
- **Rightmost derivation (RMD)** (a.k.a. **canonical derivation**): at each step, replace the **rightmost** non-terminal.

We write $\alpha \Rightarrow \beta$ for one step and $\alpha \Rightarrow^* \beta$ for zero-or-more steps.

Why it matters: LMD is the model behind **top-down (LL) parsing**; RMD (in reverse) is the model behind **bottom-up (LR) parsing**. Understanding both is essential to understanding every parser.

Where it is used in real systems:

- LL parsers / recursive descent mimic leftmost derivation.
- LR/LALR parsers (Yacc/Bison) build a rightmost derivation in reverse.
- Proving strings belong to a language, and detecting ambiguity.

Why interviewers ask about it: Classic, high-frequency written-test item ("give the leftmost and rightmost derivation of X"), and it ties directly to LL vs LR parsing.

2. Core Idea

Intuition: Both derivations produce the *same string* and (for unambiguous grammars) the *same parse tree*; they differ only in the **order** they expand non-terminals - always-leftmost vs always-rightmost.

Real-world analogy: Filling in a form with nested sections. LMD = always complete the topmost/leftmost blank first; RMD = always complete the last/rightmost blank first. Same completed form, different filling order.

Small example: Grammar $E \rightarrow E + E \mid E * E \mid id$, string $id + id * id$.

Leftmost derivation:

```
E => E + E      (expand leftmost E)
=> id + E       (leftmost E -> id)
=> id + E * E    (E -> E * E)
=> id + id * E   (E -> id)
=> id + id * id  (E -> id)
```

Rightmost derivation:

```
E => E + E      (expand rightmost E... here only one split; take E+E)
=> E + E * E     (rightmost E -> E * E)
=> E + E * id    (rightmost E -> id)
=> E + id * id   (rightmost E -> id)
=> id + id * id  (E -> id)
```

Both yield $id + id * id$.

3. Important Subtopics

3.1 Sentential forms: left- vs right-

- **What:** A **left-sentential form** appears in some leftmost derivation; a **right-sentential form** appears in some rightmost derivation.
- **Why it matters:** LR parsing works with right-sentential forms; viable prefixes are prefixes of them.
- **Interview angle:** "Is $id + E * E$ a left- or right-sentential form?" (Left - it appears in the LMD above.)

3.2 Reverse of rightmost = bottom-up parse order

- **What:** Bottom-up parsers reduce the input to S , which is exactly a rightmost derivation read backwards.
- **Why it matters:** Explains why LR parsers are said to construct "rightmost derivation in reverse."

- **Interview angle:** "What derivation does an LR parser build?" (Rightmost, in reverse.)

3.3 One tree, two orders

- **What:** For an unambiguous grammar, LMD and RMD of the same string give the same parse tree.
- **Why it matters:** Confirms derivation order != structure.
- **Interview angle:** "Do LMD and RMD always give the same tree?" (Same tree if grammar unambiguous; ambiguity is about multiple trees, not order.)

3.4 Detecting ambiguity via derivations

- **What:** Two distinct leftmost (or two distinct rightmost) derivations of one string => ambiguous.
- **Why it matters:** The standard formal test for ambiguity.
- **Interview angle:** "Show the grammar is ambiguous." => give two different LMDs.

4. Real-World Example

Yacc/Bison-generated parser in a SQL engine. The parser reduces tokens bottom-up; the sequence of reductions is precisely a **rightmost derivation in reverse** of the SQL statement. When you enable `%debug/-v` and read the trace, each "reduce by rule X" line is one step of that reverse-rightmost derivation - which is how the engine turns `SELECT a FROM t WHERE a > 1` into a structured query tree.

5. Diagrams / Mental Models

Which non-terminal is chosen (marked with ^):

```
LMD:  E => E + E => id + E => id + E * E => ...
      ^         ^         ^      (always the LEFT one)

RMD:  E => E + E => E + E * E => E + E * id => ...
      ^         ^         ^      (always the RIGHT one)
```

Direction summary:

```
Leftmost derivation  <->  Top-down (LL) parsing      (forward)
Rightmost derivation  <->  Bottom-up (LR) parsing    (reverse of RMD)
```

6. Common Interview Questions

Q1. Define leftmost and rightmost derivation.

- Answer: LMD replaces the leftmost non-terminal each step; RMD replaces the rightmost.
- Common mistake: mixing which side; or thinking they change the final string.

Q2. Do LMD and RMD produce the same string?

- Answer: Yes, always the same terminal string.

Q3. Do they produce the same parse tree?

- Answer: For an unambiguous grammar, yes. Different order, same tree.

Q4. Which derivation relates to top-down parsing? Which to bottom-up?

- Answer: Leftmost $\Rightarrow$ top-down/LL; rightmost (reversed) $\Rightarrow$ bottom-up/LR.

Q5. What does "LR builds the rightmost derivation in reverse" mean?

- Answer: Reductions performed by an LR parser, read from last to first, reconstruct a rightmost derivation.

Q6. How do you prove a grammar is ambiguous using derivations?

- Answer: Exhibit two distinct leftmost derivations (or two distinct parse trees) for the same string.

Q7. Give the leftmost derivation of $\text{id} + \text{id} * \text{id}$ (with the expression grammar).

- Answer: as shown in section 2.
- Common mistake: not consistently expanding the leftmost non-terminal.

Q8. What is a right-sentential form?

- Answer: A sentential form that occurs in a rightmost derivation.

Q9. If a grammar is unambiguous, how many leftmost derivations does a string have?

- Answer: Exactly one.

****Q10. Can a string have a leftmost derivation but no rightmost? ****

- Answer: No - if it is derivable, it has both (and they yield the same tree in an unambiguous grammar).

7. Deep-Dive Questions

D1. Why is rightmost derivation called "canonical"?

- Because LR parsing (the canonical shift-reduce technique) naturally produces it in reverse; it's the standard reference derivation for bottom-up analysis.

D2. Number of leftmost derivations vs number of parse trees?

- They are equal: there is a one-to-one correspondence between leftmost derivations and parse trees. Same for rightmost.

D3. If LMD is unique but you found two RMDs, is the grammar ambiguous?

- Contradiction - LMD unique implies exactly one tree implies exactly one RMD. If you truly found two RMDs, the LMD wasn't unique either; the grammar is ambiguous.

D4. Given an LR parser's reduction sequence, reconstruct the RMD.

- List the reductions in the order performed, then reverse them; each reversed reduction becomes a derivation step from S down to the input.

D5. For $S \rightarrow SS \mid (S) \mid \epsilon$ (ambiguous), show two leftmost derivations of $()()$.

- Tree 1: $S \rightarrow S S \rightarrow (S) S \rightarrow () S \rightarrow () (S) \rightarrow ()()$. Tree 2 groups differently via a different first split - two distinct LMDs $\Rightarrow$ ambiguous.

8. Comparison Tables

Leftmost vs Rightmost Derivation

Aspect	Leftmost (LMD)	Rightmost (RMD / canonical)
Non-terminal expanded	Leftmost first	Rightmost first
Final string	Same	Same
Parse tree (unambiguous G)	Same	Same
Parsing style	Top-down / LL	Bottom-up / LR (in reverse)
Sentential forms produced	Left-sentential forms	Right-sentential forms

Derivation vs Parsing direction

Parser family	Scans input	Derivation modeled	Order
LL (top-down)	Left to right	Leftmost	Forward
LR (bottom-up)	Left to right	Rightmost	Reverse

9. Common Mistakes

- Swapping which side "leftmost" refers to.
- Believing LMD and RMD produce different strings or different trees (unambiguous grammar $\Rightarrow$ same tree).
- Saying "two different derivation orders $\Rightarrow$ ambiguous" - only two different *trees* (or two LMDs) prove ambiguity.
- Forgetting LR builds rightmost *in reverse*, not forward.
- Inconsistently expanding non-terminals mid-derivation.

10. Edge Cases / Special Cases

- Ambiguous grammars:** more than one LMD (and more than one RMD) for the same string.
- epsilon-productions:** an ϵ step still counts; the leftmost/rightmost non-terminal may derive ϵ .
- Single non-terminal grammars:** if only one non-terminal exists per form, LMD and RMD steps can look identical even though the rule is "leftmost/rightmost".
- Cyclic grammars:** infinitely many derivations - avoid.

11. How to Explain in Interview

"A derivation rewrites the start symbol into the input one production at a time. In a leftmost derivation you always expand the leftmost non-terminal; in a rightmost derivation, the rightmost. They generate the same string and, for an unambiguous grammar, the same parse tree - they differ only in order. This matters because

top-down LL parsers trace a leftmost derivation, while bottom-up LR parsers effectively produce a rightmost derivation in reverse. And the clean test for ambiguity is: does some string have two distinct leftmost derivations?"

12. Quick Revision Notes

- LMD: expand leftmost NT. RMD (canonical): expand rightmost NT.
- Same string; same tree if grammar unambiguous.
- Leftmost $\leftrightarrow$ LL/top-down. Rightmost (reversed) $\leftrightarrow$ LR/bottom-up.
- Two distinct LMDs (or two trees) $\Rightarrow$ **ambiguous**.
- Right-sentential form: appears in an RMD (key for LR viable prefixes).
- **Trap:** derivation order differs $\neq$ ambiguity.

13. Practice Tasks

1. Give leftmost AND rightmost derivations of $id * id + id$.
2. For $S \rightarrow aSb \mid ab$, give the LMD and RMD of $aabb$ (they'll match).
3. For $S \rightarrow SS \mid (S) \mid \epsilon$, give two leftmost derivations of $()()$ to prove ambiguity.
4. Take an LR parser reduction trace and reconstruct the rightmost derivation by reversing it.
5. Identify whether $E + id * E$ is a left- or right-sentential form for the expression grammar.
6. Draw the shared parse tree for your LMD and RMD in task 1.

14. Final Cheat Sheet

- **Core definition:** LMD expands leftmost NT each step; RMD expands rightmost.
- **Why it matters:** LMD = top-down/LL; RMD reversed = bottom-up/LR.
- **Most asked:** Give LMD/RMD of a string; prove ambiguity via two LMDs.
- **Common comparison:** Leftmost vs rightmost; LL vs LR mapping.
- **One-line answer:** "Leftmost and rightmost derivations expand the leftmost or rightmost non-terminal respectively - same string, same tree if unambiguous - and they model top-down and (reversed) bottom-up parsing."

4. Ambiguous Grammar

1. Overview

Definition: A grammar is **ambiguous** if there exists **at least one string** in its language that has **more than one parse tree** (equivalently, more than one leftmost derivation, or more than one rightmost derivation).

Why it matters: Ambiguity means the *structure* of the program is not uniquely determined - $a + b * c$ could mean $(a+b)*c$ or $a+(b*c)$. A compiler must pick exactly one interpretation, so parsers require either an unambiguous grammar or extra disambiguation rules (precedence/associativity).

Where it is used in real systems:

- The classic **dangling-else** problem in C/Java grammars.
- Expression grammars in every language need precedence disambiguation.

- Yacc/Bison report **shift/reduce** and **reduce/reduce** conflicts, which are the practical face of ambiguity.

Why interviewers ask about it: It's conceptually rich (trees vs order), practically important (conflicts, precedence), and a favorite "prove this grammar is ambiguous" written question.

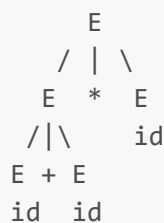
2. Core Idea

Intuition: Ambiguity is a property of the **grammar**, not the language. The same language can often be described by both an ambiguous and an unambiguous grammar. To fix ambiguity you rewrite the grammar so each valid string has exactly one tree.

Real-world analogy: The English sentence "I saw the man with the telescope." Who has the telescope - me or the man? Two valid parse structures = ambiguity. Natural language tolerates it; compilers cannot.

Small example: $E \rightarrow E + E \mid E * E \mid id$. The string $id + id * id$ has **two** parse trees:

Tree A: $(id + id) * id$



Tree B: $id + (id * id)$



Two trees => ambiguous.

Step-by-step (how to prove ambiguity):

1. Find a candidate string (often one mixing two operators or nested **if**).
2. Construct two distinct parse trees (or two distinct leftmost derivations).
3. That single witness proves ambiguity.

3. Important Subtopics

3.1 Dangling-else ambiguity

- **What:** $S \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } S \text{ else } S \mid \text{other}$. For $\text{if } e1 \text{ then if } e2 \text{ then } s1 \text{ else } s2$, the **else** can attach to either **if**.
- **Why it matters:** Real languages hit this; the standard fix is "match **else** to the nearest unmatched **if**."
- **Example:** grammar rewrite into "matched" vs "unmatched" statements, or a Yacc **%prec/%nonassoc** rule.
- **Interview angle:** "Explain and resolve the dangling-else problem."

3.2 Precedence and associativity

- **What:** Ambiguity in $E \rightarrow E + E \mid E * E$ is removed by layering the grammar into E, T, F so $*$ binds tighter and $+$ is left-associative.
- **Why it matters:** This is THE canonical fix and a must-know grammar.
- **Example:**

```

E -> E + T | T
T -> T * F | F
F -> ( E ) | id

```

- **Interview angle:** "Rewrite the ambiguous expression grammar to enforce precedence and left-associativity."

3.3 Inherent ambiguity

- **What:** Some context-free *languages* have NO unambiguous grammar at all (e.g., $\{a^i b^j c^k \mid i=j \text{ or } j=k\}$).
- **Why it matters:** Ambiguity isn't always fixable by rewriting.
- **Interview angle:** "Can every ambiguous grammar be made unambiguous?" (No - some languages are inherently ambiguous.)

3.4 Ambiguity, conflicts, and parser generators

- **What:** Ambiguous grammars cause shift/reduce or reduce/reduce conflicts in LR tools.
- **Why it matters:** Connects theory to the errors you see in Bison output.
- **Interview angle:** "How does ambiguity show up in Yacc?" (As conflicts.)

4. Real-World Example

C/Java compilers and the dangling else. The C grammar is technically ambiguous around *if/else*. Compilers resolve it with the rule "an *else* binds to the nearest preceding *if* without an *else*." In a Yacc/Bison grammar this appears as a **shift/reduce conflict** that the generator resolves by *shifting* (favoring the inner *if*), or the grammar is rewritten into matched/unmatched statement forms to remove the conflict entirely.

5. Diagrams / Mental Models

Ambiguity test:

```

Does SOME string have >= 2 parse trees ?
    |                               |
    YES  -> ambiguous             NO  -> unambiguous

```

Dangling-else attachment:

```
if e1 then if e2 then s1 else s2
```

Reading A (else with inner if): `if e1 then ( if e2 then s1 else s2 )` <- standard

Reading B (else with outer if): `if e1 then ( if e2 then s1 ) else s2`

Undecidability note:

"Is an arbitrary CFG ambiguous?" -> UNDECIDABLE (no general algorithm)

6. Common Interview Questions

Q1. What is an ambiguous grammar?

- Answer: A grammar where some string has more than one parse tree (or >1 leftmost derivation).
- Common mistake: saying "more than one derivation" without specifying *leftmost*; every string has many derivation orders.

Q2. Is ambiguity a property of the grammar or the language?

- Answer: The grammar. Most languages have both ambiguous and unambiguous grammars.

Q3. Prove $E \rightarrow E + E \mid E * E \mid id$ is ambiguous.

- Answer: $id + id * id$ has two parse trees (or two leftmost derivations).

Q4. How do you remove ambiguity from an expression grammar?

- Answer: Introduce precedence levels and associativity: $E \rightarrow E + T \mid T, T \rightarrow T * F \mid F, F \rightarrow (E) \mid id$.

Q5. What is the dangling-else problem and how is it resolved?

- Answer: An `else` ambiguously attaches to two `ifs`; resolved by "nearest unmatched if" (grammar rewrite or shift preference).

Q6. Can every ambiguous grammar be converted to unambiguous?

- Answer: No - inherently ambiguous languages exist.

Q7. Is checking whether an arbitrary grammar is ambiguous decidable?

- Answer: No, it's undecidable in general.

Q8. How does ambiguity relate to LR parser conflicts?

- Answer: Ambiguity manifests as shift/reduce or reduce/reduce conflicts.

Q9. Does ambiguity affect the language accepted?

- Answer: No - it affects structure/interpretation, not membership. Same set of strings.

Q10. Why is ambiguity bad for compilers?

- Answer: The meaning (evaluation order, associativity) becomes undefined; compiler must produce one deterministic tree.

7. Deep-Dive Questions

D1. Give an inherently ambiguous language and explain why.

- $L = \{a^i b^j c^k \mid i=j \text{ or } j=k\}$. Strings where $i=j=k$ can be parsed either as matching a's-with-b's or b's-with-c's; no single grammar avoids two structures for those strings.

D2. Left recursion vs ambiguity - related?

- Distinct. $E \rightarrow E + E$ is both left-recursive and ambiguous, but $E \rightarrow E + T \mid T$ is left-recursive yet unambiguous. Left recursion is a parsing obstacle for top-down; ambiguity is a structural defect.

D3. Why is ambiguity detection undecidable?

- It reduces to the Post Correspondence Problem; no algorithm decides ambiguity for all CFGs. Tools only detect *conflicts* in specific parser constructions.

D4. Two grammars, same language, one ambiguous one not - is the unambiguous one always "better"?

- Generally yes for compilers (deterministic structure), but the ambiguous one may be shorter/clearer for humans; tools then add precedence declarations to disambiguate rather than rewrite.

D5. How does Bison resolve a shift/reduce conflict by default, and is that "solving" ambiguity?

- It defaults to **shift**. That makes parsing deterministic but silently picks one interpretation - it hides ambiguity rather than proving the grammar unambiguous; you should still understand/verify the choice.

8. Comparison Tables

Ambiguous vs Unambiguous Grammar

Aspect	Ambiguous	Unambiguous
Parse trees per string	Some string has ≥ 2	Every string has exactly 1
Leftmost derivations	Some string has ≥ 2	Exactly 1 per string
Suitable for parser generator	Causes conflicts	Clean tables
Meaning/precedence	Undetermined	Determined
Example	$E \rightarrow E+E \mid E^*E \mid id$ $E \rightarrow E+T \mid T; T \rightarrow T^*F \mid F$	

Ways to disambiguate

Technique	Idea	Example
Precedence layering	Separate NTs per precedence level	E/T/F grammar
Associativity	Left/right recursion choice	$E \rightarrow E + T$ (left-assoc)
Grammar rewrite	Matched/unmatched forms	Dangling-else fix
Generator directives	<code>%left, %right, %prec</code>	Yacc/Bison

9. Common Mistakes

- Saying "multiple derivations" instead of "multiple *leftmost* derivations / parse trees."
- Claiming ambiguity is a language property (it's a grammar property, except inherent ambiguity).
- Thinking removing left recursion removes ambiguity (independent issues).
- Believing a tool that "compiles without error" proves the grammar is unambiguous (default conflict resolution can hide it).
- Assuming every ambiguous grammar can be fixed by rewriting.

10. Edge Cases / Special Cases

- **Inherently ambiguous languages:** unfixable by rewriting.
- **Dangling else:** the canonical ambiguity in imperative languages.
- **Undecidability:** no general ambiguity checker exists.
- **Hidden ambiguity:** a grammar may be ambiguous only on rare/long strings.
- **epsilon and unit productions** can introduce subtle extra trees.

11. How to Explain in Interview

"A grammar is ambiguous if some string in its language has two different parse trees - equivalently two distinct leftmost derivations. The classic example is $E \rightarrow E + E \mid E * E \mid id$, where $id + id * id$ parses as either $(id+id)*id$ or $id+(id*id)$. Ambiguity is a property of the grammar, not the language: we fix it by layering the grammar into precedence levels (E/T/F) and choosing associativity via left or right recursion. The famous real-world case is the dangling-else, resolved by attaching **else** to the nearest unmatched **if**. In parser generators, ambiguity shows up as shift/reduce or reduce/reduce conflicts."

12. Quick Revision Notes

- Ambiguous = some string has ≥ 2 parse trees / ≥ 2 leftmost derivations.
- Property of the **grammar**, not the language (except inherently ambiguous languages).
- Fix: precedence layering (E/T/F), associativity (left/right recursion), grammar rewrite, generator directives.
- Dangling-else = classic; resolve by "nearest unmatched if" / shift.
- Ambiguity detection is **undecidable**; shows up as **conflicts** in LR tools.
- **Trap:** "more derivations" != ambiguity; must be different *trees*.

13. Practice Tasks

1. Prove $E \rightarrow E + E \mid E * E \mid id$ is ambiguous with two leftmost derivations of $id + id * id$.
2. Rewrite it into the unambiguous E/T/F grammar and re-derive $id + id * id$.
3. Write the dangling-else grammar and show the two trees for $if\ e1\ then\ if\ e2\ then\ s1\ else\ s2$.
4. Rewrite the dangling-else grammar into matched/unmatched form to remove ambiguity.
5. Show $S \rightarrow SS \mid (S) \mid \epsilon$ is ambiguous using $()()$.
6. Explain why $\{a^i b^j c^k \mid i=j\ or\ j=k\}$ is inherently ambiguous.

14. Final Cheat Sheet

- **Core definition:** Some string has more than one parse tree.
- **Why it matters:** Program structure/meaning becomes non-deterministic; parsers need one tree.
- **Most asked:** Prove ambiguity; remove it with E/T/F; dangling-else.

- **Common comparison:** Ambiguous vs unambiguous; ambiguity vs left recursion.
- **One-line answer:** "An ambiguous grammar lets some string have two parse trees; it's a grammar defect fixed by precedence/associativity layering, and detecting it in general is undecidable."

5. Eliminating Left Recursion

1. Overview

Definition: A grammar is **left-recursive** if a non-terminal can derive itself as its leftmost symbol: $A \Rightarrow^+ A \alpha$. **Immediate** left recursion is $A \rightarrow A \alpha \mid \beta$; **indirect** (mutual) is $A \rightarrow B \dots$, $B \rightarrow A \dots$. **Eliminating left recursion** rewrites the grammar to an equivalent one (same language) with no left recursion.

Why it matters: Top-down / recursive-descent / LL parsers cannot handle left recursion - a rule like $A \rightarrow A \alpha$ makes the parser call $A()$ first thing inside $A()$, recursing forever with no input consumed (infinite loop). Removing it is a mandatory preprocessing step for LL parsing.

Where it is used in real systems:

- Building any hand-written recursive-descent parser (must convert left-recursive expression grammars).
- LL(1) table construction (Yacc/LR tolerates left recursion; LL does not).
- Grammar tools like ANTLR do this transformation automatically.

Why interviewers ask about it: It's a concrete, mechanical transformation with a formula - easy to test on paper, and it reveals whether you understand why LL parsers loop forever otherwise.

2. Core Idea

Intuition: Left recursion builds structure "growing to the left" $((a+b)+c)+d$. Top-down parsing must decide what to do *before* consuming input, so an unbounded left spine traps it. We convert the left recursion into **right recursion** using a new helper non-terminal, so the parser consumes a terminal before recursing.

Real-world analogy: Trying to peel an onion from the inside-out. Left recursion asks you to reach the innermost layer first without removing any outer layer - impossible top-down. Right recursion peels outer layers (consumes tokens) first.

The formula (immediate left recursion): For $A \rightarrow A a_1 \mid A a_2 \mid \dots \mid A a_m \mid b_1 \mid b_2 \mid \dots \mid b_n$ (each b_i does NOT start with A), rewrite as:

```
A  -> b1 A' | b2 A' | ... | bn A'
A' -> a1 A' | a2 A' | ... | am A' | epsilon
```

Small example:

```
Before (left-recursive):  E -> E + T | T
After:                   E  -> T E'
```

$$E' \rightarrow + T E' \mid \epsilon$$

Similarly $T \rightarrow T * F \mid F$ becomes $T \rightarrow F T', T' \rightarrow * F T' \mid \epsilon$.

Step-by-step:

1. Group productions of A into left-recursive ($A \alpha_i$) and non-left-recursive (β_j).
2. Create A' (fresh non-terminal).
3. A produces each $\beta_j A'$.
4. A' produces each $\alpha_i A'$ plus ϵ .

3. Important Subtopics

3.1 Immediate left recursion

- **What:** $A \rightarrow A \alpha \mid \beta$ - A directly starts with itself.
- **Why it matters:** Most common case; the formula above handles it directly.
- **Example:** $E \rightarrow E + T \mid T$.
- **Interview angle:** "Remove immediate left recursion from this rule."

3.2 Indirect (mutual) left recursion

- **What:** $A \rightarrow B a \mid \dots, B \rightarrow A b \mid \dots$ - recursion appears after substitution.
- **Why it matters:** The simple formula isn't enough; you need the general algorithm.
- **Interview angle:** "This grammar has no $A \rightarrow A \dots$ rule - is it still left-recursive?" (Could be, indirectly.)

3.3 General algorithm (for indirect recursion)

- **What:** Order non-terminals $A_1 \dots A_n$. For $i=1..n$: for $j=1..i-1$, replace $A_i \rightarrow A_j \gamma$ by substituting A_j 's productions; then eliminate immediate left recursion in A_i .
- **Why it matters:** Systematic way to kill all left recursion, direct and indirect.
- **Interview angle:** "Give the algorithm for arbitrary left recursion."

3.4 Left recursion vs right recursion trade-offs

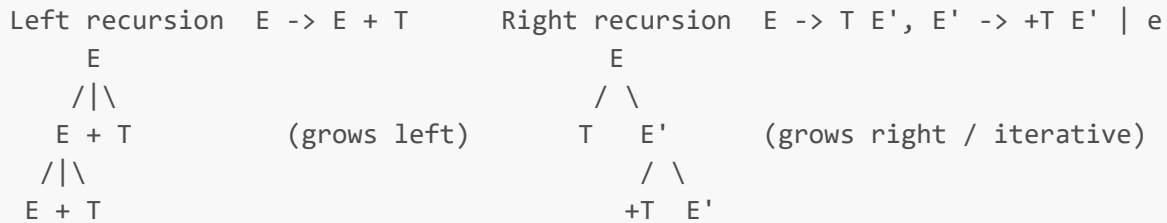
- **What:** Left recursion $\Rightarrow$ left-associative, needs bottom-up or transformation; right recursion $\Rightarrow$ right-associative, LL-friendly but uses more parser stack.
- **Why it matters:** Affects associativity of operators and stack usage.
- **Interview angle:** "After removing left recursion, is - still left-associative?" (Not automatically - you must rebuild associativity in semantic actions.)

4. Real-World Example

Hand-written recursive-descent expression parser. You cannot code `parseE()` from $E \rightarrow E + T$ directly - it would call `parseE()` immediately and stack-overflow. So compiler courses and real hand-written parsers (e.g., in many interpreters and calculators) convert to $E \rightarrow T E' / E' \rightarrow + T E' \mid \epsilon$, or equivalently use a `while` loop: `parseE(){ t = parseT(); while(peek=='+'){ eat('+'); parseT(); } }`. That loop is the eliminated-left-recursion form in disguise.

5. Diagrams / Mental Models

Left vs right recursion shape:



Why LL loops on left recursion:

```

parseE():           # from E -> E + T
  parseE()           # <-- calls itself with NO token consumed -> infinite loop
  ...

```

6. Common Interview Questions

Q1. What is left recursion and why is it a problem?

- Answer: $A \Rightarrow^+ A$ **alpha**; top-down parsers recurse on A without consuming input $\Rightarrow$ infinite loop.
- Common mistake: saying it breaks all parsers (LR/bottom-up handles it fine).

Q2. Eliminate left recursion from $E \rightarrow E + T \mid T$.

- Answer: $E \rightarrow T E', E' \rightarrow + T E' \mid \text{epsilon}$.

Q3. Immediate vs indirect left recursion?

- Answer: Immediate = $A \rightarrow A \dots$; indirect = recursion appears only after substituting other non-terminals.

Q4. Does eliminating left recursion change the language?

- Answer: No - the transformed grammar generates the same language.

Q5. Which parsers require left-recursion removal?

- Answer: Top-down / LL / recursive-descent. Bottom-up/LR do not.

Q6. What non-terminal do you introduce and what does it contain?

- Answer: A fresh A' that carries the "tail" productions plus an **epsilon**.

Q7. Give the general algorithm for indirect left recursion.

- Answer: Order non-terminals; substitute earlier ones into later ones; then remove immediate recursion at each step.

Q8. After removing left recursion, is associativity preserved?

- Answer: The *language* is preserved, but the natural left-associativity must be restored via semantic actions / how you fold the loop.

Q9. Is $E \rightarrow E + T \mid T$ ambiguous?

- Answer: No - it's left-recursive but unambiguous. Different issues.

Q10. Convert $A \rightarrow A a \mid A b \mid c \mid d$.

- Answer: $A \rightarrow c A' \mid d A', A' \rightarrow a A' \mid b A' \mid \text{epsilon}$.

7. Deep-Dive Questions

D1. Why can LR parsers handle left recursion but LL cannot?

- LR is bottom-up: it shifts input symbols and reduces once it has seen a full RHS, so a left-recursive rule just means "keep reducing" - it actually prefers left recursion (bounded stack). LL must predict and expand top-down, so left recursion has no base case reachable before consuming input.

D2. Show indirect left recursion elimination on $S \rightarrow Aa \mid b; A \rightarrow Ac \mid Sd \mid e$.

- Substitute S into A's Sd: $A \rightarrow Ac \mid Aad \mid bd \mid e$. Now immediate: $A \rightarrow bd A' \mid e A', A' \rightarrow c A' \mid ad A' \mid \text{epsilon}$. S unchanged.

D3. Does removing left recursion always yield an LL(1) grammar?

- No. It removes one obstacle, but the result may still need **left factoring** and may still have FIRST/FOLLOW conflicts. Left-recursion removal is necessary, not sufficient, for LL(1).

D4. What's the cost of the new epsilon-productions introduced?

- They complicate FIRST/FOLLOW (A' is nullable) and can introduce new conflicts; but they're necessary to terminate the tail recursion.

D5. Can you avoid the transformation entirely?

- Yes - use an iterative (loop-based) recursive-descent for the operator, or use a bottom-up/LR parser, or a Pratt/precedence-climbing parser. The transformation is only needed for pure LL table-driven parsing.

8. Comparison Tables

Left Recursion vs Right Recursion

Aspect	Left recursion $A \rightarrow A a$	Right recursion $A \rightarrow a A$
Top-down (LL)	Infinite loop (bad)	Works
Bottom-up (LR)	Preferred (bounded stack)	Larger stack
Associativity	Left-associative	Right-associative
Parser stack	Shallow (LR)	Deep (LL)

Left Recursion vs Left Factoring

Aspect	Left recursion elimination	Left factoring
Problem solved	$A \rightarrow A \text{ alpha}$ loops	Common prefix $A \rightarrow a B \mid a C$
Fix	Introduce A' tail, make right-recursive	Factor prefix: $A \rightarrow a A', A' \rightarrow B \mid C$
Needed for	LL parsing termination	LL(1) determinism (single lookahead)

9. Common Mistakes

- Applying the formula when some **beta_j** actually starts with A (must not).
- Forgetting the **epsilon** in the A' rule (parser never terminates the tail).
- Missing **indirect** left recursion because there's no literal $A \rightarrow A \dots$.
- Thinking left recursion = ambiguity (independent).
- Assuming associativity is automatically preserved after the rewrite.

10. Edge Cases / Special Cases

- **Indirect recursion** hidden across several non-terminals.
- **Nullable betas**: if a **beta_j** can derive epsilon, extra care with FIRST/FOLLOW.
- **Combined with left factoring**: often you must do both before LL(1) works.
- **LR grammars**: don't remove left recursion - LR handles it and prefers it.

11. How to Explain in Interview

"Left recursion is when a non-terminal can derive itself as its leftmost symbol, like $E \rightarrow E + T$. Top-down parsers loop forever on it because they'd call the same rule without consuming input. To fix immediate left recursion $A \rightarrow A \text{ alpha} \mid \text{beta}$, I introduce a helper A' : $A \rightarrow \text{beta } A'$ and $A' \rightarrow \text{alpha } A' \mid \text{epsilon}$, which converts it to right recursion. So $E \rightarrow E + T \mid T$ becomes $E \rightarrow T E', E' \rightarrow + T E' \mid \text{epsilon}$. For indirect recursion I first substitute earlier non-terminals to expose the immediate form. Note this is only needed for LL parsers - LR/bottom-up parsers actually prefer left recursion."

12. Quick Revision Notes

- Left recursion: $A \Rightarrow^+ A \text{ alpha}$. Immediate $A \rightarrow A a$; indirect via other NTs.
- **Only LL/top-down needs removal**; LR handles/prefers left recursion.
- Formula: $A \rightarrow A a \mid b$ becomes $A \rightarrow b A', A' \rightarrow a A' \mid \text{epsilon}$.
- Removal preserves the language, NOT automatically associativity.
- Necessary but not sufficient for LL(1) (may still need left factoring).
- **Trap**: left recursion $\neq$ ambiguity; watch for indirect recursion.

13. Practice Tasks

1. Eliminate immediate left recursion from $E \rightarrow E + T \mid E - T \mid T$.
2. Do the same for $T \rightarrow T * F \mid T / F \mid F$.
3. Eliminate indirect left recursion in $S \rightarrow Aa \mid b; A \rightarrow Ac \mid Sd \mid e$.
4. Convert $A \rightarrow A a \mid A b \mid c$ and list $\text{FIRST}(A')$, which is nullable.
5. Rewrite an operator parser as a loop and match it to the eliminated form.

6. Argue why an LR parser doesn't need any of this.

14. Final Cheat Sheet

- **Core definition:** $A \Rightarrow^+ A \text{ alpha}$; remove so top-down parsers don't loop.
- **Why it matters:** LL/recursive-descent can't parse left-recursive rules.
- **Most asked:** Apply the $A \rightarrow bA', A' \rightarrow aA' \mid \text{epsilon}$ formula; indirect case.
- **Common comparison:** Left vs right recursion; recursion removal vs left factoring.
- **One-line answer:** "Left recursion ($A \rightarrow A \text{ alpha}$) loops top-down parsers; rewrite $A \rightarrow A a \mid b$ as $A \rightarrow b A', A' \rightarrow a A' \mid \text{epsilon}$ to make it right-recursive - LR parsers don't need this."

6. Left Factoring

1. Overview

Definition: Left factoring is a grammar transformation that removes a **common prefix** shared by two or more productions of the same non-terminal, so a predictive parser can decide which rule to use with limited lookahead. $A \rightarrow \text{alpha beta1} \mid \text{alpha beta2}$ becomes $A \rightarrow \text{alpha } A', A' \rightarrow \text{beta1} \mid \text{beta2}$.

Why it matters: A top-down predictive (LL(1)) parser chooses a production by looking at **one** input token. If two productions of A start with the same symbol(s), the parser cannot decide which to pick from one token - it would have to backtrack. Left factoring defers the decision until after the common prefix is consumed.

Where it is used in real systems:

- Preparing grammars for LL(1) / recursive-descent parsers.
- Classic case: $\text{stmt} \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } S \text{ else } S$ (common prefix $\text{if } E \text{ then } S$).
- ANTLR and other LL tools rely on factored (or otherwise disambiguated) grammars.

Why interviewers ask about it: It's a small, mechanical, high-yield transformation frequently paired with left-recursion removal as "make this grammar LL(1)."

2. Core Idea

Intuition: Don't commit to a branch until you have enough information. If both branches begin with **alpha**, first match **alpha**, THEN look at what follows to decide between **beta1** and **beta2**.

Real-world analogy: A form asks "Are you paying by (a) card ending in ... credit, or (b) card ending in ... debit?" Both start with "card ending in ...". You fill the common part first, then decide credit vs debit. Committing upfront would force you to restart (backtrack).

Small example:

```
Before:  A -> a B C D | a B C E
Common prefix: a B C
After:   A  -> a B C A'
        A' -> D | E
```

Classic if-else:

```
Before: S -> if E then S | if E then S else S | other
After:  S -> if E then S S' | other
        S' -> else S | epsilon
```

Step-by-step:

1. Find the longest common prefix **alpha** among productions of A.
2. Replace them with **A -> alpha A'**.
3. **A' -> (remaining tails) | epsilon** (epsilon if one alternative was exactly **alpha**).
4. Repeat until no two productions of any non-terminal share a prefix.

3. Important Subtopics

3.1 Longest common prefix

- **What:** Factor the *longest* shared prefix to maximize determinism.
- **Why it matters:** Factoring only one symbol may leave further conflicts.
- **Example:** **A -> a b c | a b d** - factor **a b**, not just **a**.
- **Interview angle:** "How much of the prefix do you factor?"

3.2 Repeated / iterative factoring

- **What:** After one factoring, the new non-terminal A' may itself need factoring.
- **Why it matters:** One pass may not be enough.
- **Interview angle:** "Is one round of left factoring always sufficient?" (No.)

3.3 Relationship to LL(1) and backtracking

- **What:** Unfactored common prefixes cause FIRST/FIRST conflicts, forcing backtracking or LL(k>1).
- **Why it matters:** Left factoring is a prerequisite for a conflict-free LL(1) table.
- **Interview angle:** "Why does the LL(1) table get two entries in one cell without factoring?"

3.4 Left factoring vs left recursion removal

- **What:** Different problems: factoring targets common prefixes; recursion removal targets **A -> A alpha**.
- **Why it matters:** Both usually needed to reach LL(1); students conflate them.
- **Interview angle:** "Which transformation fixes **A -> A a | b** vs **A -> a b | a c**?"

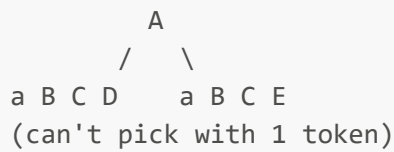
4. Real-World Example

A hand-written config/DSL parser using recursive descent. Suppose commands are **set X = Y** and **set X += Y**. Both start with **set X**. A one-token-lookahead parser can't choose immediately. Left factoring gives **command -> set ID assignTail, assignTail -> = expr | += expr**. Now the parser matches **set ID**, then peeks at **=** vs **+=** to pick the tail - no backtracking, single pass. This is exactly how many real recursive-descent parsers keep their prediction deterministic.

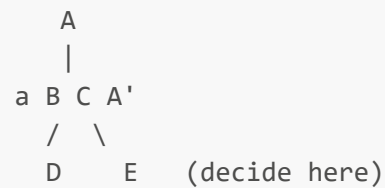
5. Diagrams / Mental Models

Factoring shape:

Before:



After:



Decision timing:

Unfactored: decide branch at token 1 (not enough info -> backtrack)

Factored: consume common prefix, decide branch AFTER it (1 token suffices)

6. Common Interview Questions

Q1. What is left factoring and why is it needed?

- Answer: Removing a common prefix so a predictive (LL(1)) parser can choose a production with one lookahead token, avoiding backtracking.
- Common mistake: confusing it with left-recursion removal.

Q2. Left factor $A \rightarrow aB \mid aC$.

- Answer: $A \rightarrow a A', A' \rightarrow B \mid C$.

Q3. Left factor the if-else grammar.

- Answer: $S \rightarrow \text{if } E \text{ then } S \ S' \mid \text{other}, S' \rightarrow \text{else } S \mid \text{epsilon}$.

Q4. Does left factoring change the language?

- Answer: No - equivalent grammar, same language.

Q5. How much of the prefix should you factor?

- Answer: The longest common prefix.

Q6. Is one pass of left factoring always enough?

- Answer: No - the introduced non-terminal may need further factoring.

Q7. What conflict does an unfactored prefix cause in LL(1)?

- Answer: A FIRST/FIRST conflict - two productions share a FIRST symbol, so a table cell gets two entries.

Q8. Left factoring vs left recursion elimination?

- Answer: Factoring removes common prefixes; recursion elimination removes $A \rightarrow A \text{ alpha}$. Both often needed for LL(1).

Q9. When is the **epsilon** alternative introduced?

- Answer: When one original production equals the common prefix exactly (its tail is empty).

Q10. Does left factoring guarantee LL(1)?

- Answer: No - it removes one obstacle; FIRST/FOLLOW conflicts may remain.

7. Deep-Dive Questions

D1. Factor $A \rightarrow a b B \mid a b C \mid a d$.

- Longest common prefix across all three is a . Factor: $A \rightarrow a A'$, $A' \rightarrow b B \mid b C \mid d$. Then A' still has common prefix b : $A' \rightarrow b A'' \mid d$, $A'' \rightarrow B \mid C$.

D2. Can left factoring introduce ambiguity or new conflicts?

- It doesn't add ambiguity (language preserved), but the new epsilon-productions can create FOLLOW-based conflicts; you may need to iterate and re-check FIRST/FOLLOW.

D3. Why is one token of lookahead the constraint that motivates factoring?

- LL(1) predicts using exactly one symbol; shared prefixes make FIRST sets overlap so one symbol can't disambiguate. Factoring shifts the decision point past the shared symbols.

D4. Relationship between left factoring and LL(k)?

- With more lookahead (LL(k)) some unfactored prefixes become decidable, but factoring is the general grammar-level fix that works even for LL(1).

D5. Does bottom-up (LR) parsing need left factoring?

- No. LR delays decisions until it has seen the whole RHS (reduce time), so common prefixes are fine - another reason LR handles more grammars than LL.

8. Comparison Tables

Left Factoring vs Left Recursion Elimination

Aspect	Left factoring	Left recursion elimination
Target pattern	$A \rightarrow \text{alpha } b1 \mid \text{alpha } b2$	$A \rightarrow A \text{ alpha} \mid \text{beta}$
Symptom fixed	FIRST/FIRST conflict, backtracking	Infinite top-down loop
Transformation	$A \rightarrow \text{alpha } A'; A' \rightarrow b1 \mid b2$	$A \rightarrow \text{beta } A'; A' \rightarrow \text{alpha } A' \mid \text{epsilon}$
Needed by	LL(1) determinism	LL termination
LR needs it?	No	No

Before vs After (if-else)

Grammar

Before $S \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } S \text{ else } S \mid \text{other}$

After $S \rightarrow \text{if } E \text{ then } S S' \mid \text{other}; S' \rightarrow \text{else } S \mid \text{epsilon}$

9. Common Mistakes

- Factoring only the first symbol instead of the longest common prefix.
- Forgetting to iterate (the new A' still shares a prefix).
- Confusing left factoring with left-recursion elimination.
- Omitting the **epsilon** alternative when one production was exactly the prefix.
- Thinking factoring alone guarantees LL(1) (FIRST/FOLLOW conflicts may remain).

10. Edge Cases / Special Cases

- **Nested common prefixes** requiring multiple rounds.
- **One production equals the prefix** $\Rightarrow$ introduce epsilon in A' .
- **Prefix is a non-terminal** (e.g., $A \rightarrow B c \mid B d$): still factor B .
- **Interaction with FOLLOW**: new epsilon-productions can create FIRST/FOLLOW conflicts to recheck.
- **Dangling-else** appears as a shift/reduce even after factoring in LR tools.

11. How to Explain in Interview

"Left factoring removes a common prefix shared by alternatives of a non-terminal so a predictive parser can choose with a single lookahead token. If $A \rightarrow \text{alpha } \beta_1 \mid \text{alpha } \beta_2$, a one-token LL parser can't tell the branches apart, so I rewrite it as $A \rightarrow \text{alpha } A'$ and $A' \rightarrow \beta_1 \mid \beta_2$, deferring the decision until after the shared prefix. The classic example is the if-else grammar, where both alternatives start with **if E then S**. It's language-preserving, often needs multiple passes, and is usually done together with left-recursion removal to make a grammar LL(1). LR parsers don't need it because they decide at reduce time."

12. Quick Revision Notes

- Left factoring: $A \rightarrow \text{alpha } \beta_1 \mid \text{alpha } \beta_2$ becomes $A \rightarrow \text{alpha } A', A' \rightarrow \beta_1 \mid \beta_2$.
- Fixes **FIRST/FIRST conflicts** / backtracking in LL(1).
- Factor the **longest** common prefix; may need **multiple passes**.
- Add **epsilon** when one alternative equals the prefix.
- Different from left-recursion removal; both usually needed for LL(1).
- LR/bottom-up parsers do NOT need left factoring.

13. Practice Tasks

1. Left factor $S \rightarrow i E t S \mid i E t S e S \mid a$.
2. Left factor $A \rightarrow a b c \mid a b d \mid a e$ (expect multiple rounds).
3. Left factor $A \rightarrow B c \mid B d$ where B is a non-terminal.
4. Show the FIRST/FIRST conflict that exists before factoring $A \rightarrow aB \mid aC$.
5. Combine: take a left-recursive AND common-prefix grammar, apply both transformations.
6. Confirm the factored if-else grammar builds a conflict-free LL(1) entry (except dangling-else).

14. Final Cheat Sheet

- **Core definition:** Remove common production prefix for single-token prediction.
- **Why it matters:** Enables deterministic LL(1)/recursive-descent (no backtracking).
- **Most asked:** Factor $A \rightarrow aB \mid aC$; factor the if-else grammar.
- **Common comparison:** Left factoring vs left-recursion removal.
- **One-line answer:** "Left factoring rewrites $A \rightarrow \alpha b1 \mid \alpha b2$ as $A \rightarrow \alpha A', A' \rightarrow b1 \mid b2$ so an LL(1) parser can pick a production with one lookahead token."

7. Top-Down Parsing

1. Overview

Definition: Top-down parsing builds the parse tree from the **root (start symbol) down to the leaves (terminals)**, trying to trace a **leftmost derivation** of the input. At each step it expands a non-terminal by choosing one of its productions, matching terminals against the input left to right.

Why it matters: It is the simplest, most intuitive parsing strategy and the basis for **recursive-descent** and **LL(1)** parsers. Many hand-written parsers (compilers, interpreters, config readers) are top-down because they're easy to write and debug.

Where it is used in real systems:

- Hand-written recursive-descent parsers (Clang's C++ parser, many language front-ends, JSON parsers).
- ANTLR generates top-down (LL(*)) parsers.
- Simple DSLs, expression evaluators, calculators.

Why interviewers ask about it: It ties together grammars, derivations, FIRST/FOLLOW, and the transformations (left-recursion removal, left factoring). It's the natural first parsing algorithm and pairs with "why can't top-down handle left recursion?"

2. Core Idea

Intuition: Start at the goal (start symbol) and repeatedly ask "which production should I use to make progress toward matching the remaining input?" Expand, match terminals, recurse. It's **predictive** if you can always pick the right production by lookahead; otherwise it must **backtrack**.

Real-world analogy: Solving a maze from the entrance (start) forward, guessing turns. A **predictive** solver reads a sign at each junction (lookahead) and never backtracks; a **backtracking** solver tries a path and rewinds on dead ends.

Small example: Grammar $E \rightarrow T E', E' \rightarrow + T E' \mid \epsilon, T \rightarrow id$. Input $id + id$:

```
Goal: E
E -> T E'      match T
  T -> id       consume 'id'   remaining: + id
  E' -> + T E'  consume '+'    remaining: id
```

```
T -> id      consume 'id'    remaining: (empty)
E' -> epsilon done
```

Tree grew from E downward; input fully matched.

Step-by-step (generic top-down):

1. Push start symbol as current goal.
2. If top is a terminal, match it with the current input token (consume) or fail.
3. If top is a non-terminal, choose a production (via lookahead or trial) and replace.
4. Repeat until input consumed and stack empty (accept), else reject.

3. Important Subtopics

3.1 Backtracking (brute-force) top-down

- **What:** Try a production; if it fails later, undo and try another.
- **Why it matters:** General but slow (potentially exponential); rarely used in production.
- **Example:** Grammar $S \rightarrow c A d, A \rightarrow a b \mid a$; on input `cad`, trying $A \rightarrow ab$ fails, backtrack to $A \rightarrow a$.
- **Interview angle:** "Why avoid backtracking parsers?"

3.2 Predictive (non-backtracking) top-down

- **What:** Use lookahead + FIRST/FOLLOW to choose the correct production deterministically. No undo.
- **Why it matters:** Linear time; this is what LL(1)/recursive-descent aim for.
- **Interview angle:** "What makes top-down predictive?"

3.3 Prerequisites: no left recursion, left factored

- **What:** Top-down parsers require left recursion removed and common prefixes factored.
- **Why it matters:** Otherwise infinite loops / non-determinism.
- **Interview angle:** "What must you do to a grammar before top-down parsing?"

3.4 Recursive-descent vs table-driven

- **What:** Recursive-descent = one function per non-terminal (uses the call stack). Table-driven LL(1) = explicit stack + parsing table.
- **Why it matters:** Two concrete implementations of the same idea.
- **Interview angle:** "Two ways to implement top-down parsing?"

4. Real-World Example

A JSON parser in a backend service. Most JSON parsers are top-down recursive-descent: `parseValue()` looks at the first token - `{` calls `parseObject()`, `[` calls `parseArray()`, a quote calls `parseString()`, etc. Because JSON's grammar is LL(1) (each construct is distinguishable by its first token), no backtracking is needed and parsing is linear - ideal for high-throughput servers deserializing millions of payloads.

5. Diagrams / Mental Models

Direction of tree construction:

Top-down:	Bottom-up:
S (start, known)	S (built last)
/ \ expand downward	/ \
... match leaves last	leaves matched first (input)

Predictive vs backtracking:

Backtracking: try -> fail -> undo -> try again (maybe exponential)
 Predictive: peek lookahead -> pick correct rule (linear, no undo)

6. Common Interview Questions

Q1. What is top-down parsing?

- Answer: Building the parse tree from the start symbol down, tracing a leftmost derivation, matching input left to right.
- Common mistake: confusing it with bottom-up (which builds from leaves up).

Q2. Which derivation does top-down parsing simulate?

- Answer: Leftmost derivation.

Q3. What grammar preprocessing does top-down need?

- Answer: Remove left recursion; left factor common prefixes.

Q4. Backtracking vs predictive top-down?

- Answer: Backtracking tries and undoes (slow, general); predictive uses lookahead + FIRST/FOLLOW (linear, no undo).

Q5. Why can't top-down parse left-recursive grammars?

- Answer: It would expand the same non-terminal without consuming input, looping forever.

Q6. Two implementations of top-down parsing?

- Answer: Recursive-descent (call stack) and table-driven LL(1) (explicit stack + table).

Q7. What is the time complexity of predictive top-down parsing?

- Answer: Linear $O(n)$ in input length.

Q8. What is lookahead and how does top-down use it?

- Answer: The next k input tokens; used to pick which production to expand ($k=1$ for LL(1)).

Q9. Is top-down parsing as powerful as bottom-up?

- Answer: No - LL grammars are a strict subset of LR grammars; some grammars are LR but not LL.

Q10. What happens on a parse error in predictive parsing?

- Answer: The lookahead doesn't match any valid production/terminal; report error (and possibly recover via panic mode).

7. Deep-Dive Questions

D1. Why is backtracking top-down potentially exponential?

- Overlapping alternatives cause the parser to re-parse the same substrings many times across different failed guesses; without memoization the work multiplies. (Packrat/PEG parsers add memoization to make it linear.)

D2. How do recursive-descent parsers implement the parsing stack?

- Implicitly via the program's function-call stack: each active `parseX()` frame is a symbol on the conceptual stack.

D3. LL(1) vs LL(k) vs LL(*) - what changes?

- More lookahead resolves more conflicts. LL(1) uses 1 token; LL(k) uses k; ANTLR's LL(*) uses arbitrary lookahead via a sub-automaton. All are top-down.

D4. Can top-down parsers handle all deterministic CFLs?

- No. Deterministic CFLs correspond to LR(1); LL(1) is strictly weaker. Some deterministic languages have no LL grammar.

D5. How does a top-down parser build the actual tree/AST?

- Each expansion creates a node; matched terminals become leaves. In recursive descent, each function returns its subtree, and the caller links it in.

8. Comparison Tables

Top-Down vs Bottom-Up Parsing

Aspect	Top-down	Bottom-up
Tree built	Root to leaves	Leaves to root
Derivation	Leftmost	Rightmost (reverse)
Left recursion	Must remove	Handled/preferred
Left factoring	Needed	Not needed
Power	LL grammars (smaller class)	LR grammars (larger)
Typical impl	Recursive descent, LL(1)	Shift-reduce, LR/LALR
Ease to hand-write	Easy	Hard (usually generated)

Backtracking vs Predictive Top-Down

Aspect	Backtracking	Predictive
Undo on failure	Yes	No
Time complexity	Up to exponential	Linear
Uses FIRST/FOLLOW	Not necessarily	Yes
Used in practice	Rarely (PEG/packrat memoizes)	Commonly (LL(1))

9. Common Mistakes

- Confusing top-down (root-first, leftmost) with bottom-up (leaf-first, rightmost).
- Forgetting the grammar must be left-recursion-free and left-factored.
- Assuming top-down is as powerful as bottom-up (it isn't).
- Thinking recursive descent needs an explicit stack (it uses the call stack).
- Believing backtracking is efficient (it can be exponential without memoization).

10. Edge Cases / Special Cases

- **Left recursion:** infinite loop - must transform first.
- **Common prefixes:** non-determinism - must left factor.
- **epsilon-productions:** need FOLLOW to decide when to use them.
- **LL(1) conflicts:** grammar not LL(1) even after transforms => need more lookahead or bottom-up.
- **Error recovery:** panic-mode skipping to a synchronizing token.

11. How to Explain in Interview

"Top-down parsing builds the parse tree starting from the start symbol and expanding productions downward until the leaves match the input - it's essentially tracing a leftmost derivation. The practical, efficient form is predictive parsing, which uses one lookahead token plus FIRST/FOLLOW sets to always pick the right production, giving linear time with no backtracking. It's implemented either as recursive descent (a function per non-terminal, using the call stack) or as a table-driven LL(1) parser. The catch is that the grammar must be free of left recursion and left-factored first, and top-down handles a strictly smaller class of grammars than bottom-up LR parsing."

12. Quick Revision Notes

- Top-down = root to leaves, simulates **leftmost derivation**, scans input L-to-R.
- Needs: **no left recursion, left factored**.
- Two flavors: **backtracking** (slow) vs **predictive** (linear, uses FIRST/FOLLOW).
- Implementations: **recursive descent** (call stack) and **table-driven LL(1)**.
- LL grammars $\subset$ LR grammars (top-down weaker than bottom-up).
- **Trap:** left recursion => infinite loop; common prefix => non-determinism.

13. Practice Tasks

1. Hand-trace a top-down parse of `id + id * id` using the factored E/T/F grammar.

2. Convert $E \rightarrow E + T \mid T$ so it's top-down parseable, then parse `id + id`.
3. Write pseudocode for a backtracking parser of $S \rightarrow cAd; A \rightarrow ab \mid a$ on `cad`.
4. Identify which grammars from earlier topics need transformation before top-down.
5. Explain, with a trace, why $E \rightarrow E + T$ loops in top-down.
6. Compare the parse of the same string top-down vs bottom-up.

14. Final Cheat Sheet

- **Core definition:** Build parse tree root-to-leaves, tracing a leftmost derivation.
- **Why it matters:** Basis of recursive-descent and LL(1); easy to hand-write.
- **Most asked:** Top-down vs bottom-up; backtracking vs predictive; why no left recursion.
- **Common comparison:** Top-down vs bottom-up; predictive vs backtracking.
- **One-line answer:** "Top-down parsing expands from the start symbol down, tracing a leftmost derivation; the predictive form uses one lookahead and FIRST/FOLLOW for linear-time parsing, but needs a left-recursion-free, left-factored grammar."

8. Recursive-Descent Parsing

1. Overview

Definition: Recursive-descent parsing is a top-down parsing technique implemented as a **set of mutually recursive functions - one per non-terminal**. Each function is responsible for recognizing its non-terminal, matching terminals and calling other functions for nested non-terminals. It can be **predictive** (no backtracking, one lookahead) or **with backtracking**.

Why it matters: It's the most common **hand-written** parsing method because it's simple, readable, and maps directly to the grammar. Real production compilers (GCC, Clang, TypeScript, many interpreters) use hand-written recursive-descent parsers for better error messages and flexibility.

Where it is used in real systems:

- Clang (C/C++), the TypeScript compiler, the Go compiler, CPython's PEG parser (a variant), Roslyn (C#).
- JSON/YAML/config parsers, expression evaluators, query parsers.
- Compiler courses' first parser project.

Why interviewers ask about it: It's the practical face of top-down parsing; interviewers may ask you to *write* one for a small grammar, and it tests grammar transformation knowledge (left recursion, factoring).

2. Core Idea

Intuition: Turn each grammar rule into a function. $A \rightarrow X Y Z$ becomes "in `A()`, handle X, then Y, then Z." Terminals are matched by a **match/expect** helper; non-terminals are handled by calling their functions. The program's **call stack** IS the parser stack.

Real-world analogy: A recipe with sub-recipes. "Make lasagna" calls "make sauce" and "make pasta", each a self-contained routine that may call further sub-routines. The nesting of calls mirrors the nesting of the dish (the parse tree).

Small example (predictive, using a global lookahead token):

```
// Grammar: E -> T E'      E' -> + T E' | epsilon      T -> id
void E() { T(); Eprime(); }
void Eprime() {
    if (look == '+') { match('+'); T(); Eprime(); }
    // else epsilon: do nothing
}
void T() { match(id); }
void match(t){ if (look==t) look = nextToken(); else error(); }
```

Step-by-step:

1. Prime **look** with the first token.
2. Call the start-symbol function.
3. Each function matches terminals and calls sub-functions based on **look**.
4. At end, expect end-of-input marker **\$**; success = accept.

3. Important Subtopics

3.1 Predictive recursive descent (no backtracking)

- **What:** Uses one lookahead token and FIRST/FOLLOW to choose alternatives; never undoes.
- **Why it matters:** Linear time; the standard practical form. Requires an LL(1)-friendly grammar.
- **Example:** the `Eprime()` above chooses **+** branch vs epsilon by peeking.
- **Interview angle:** "How does a recursive-descent parser choose between alternatives?"

3.2 Backtracking recursive descent

- **What:** Tries an alternative; if a later match fails, restores the input position and tries the next.
- **Why it matters:** Handles more grammars but can be exponential; needs input rewinding.
- **Interview angle:** "How do you implement backtracking and what's the cost?"

3.3 Handling epsilon and choosing productions

- **What:** For $A \rightarrow \alpha \mid \epsilon$, take **alpha** if lookahead is in FIRST(alpha), else take epsilon if lookahead in FOLLOW(A).
- **Why it matters:** Correct epsilon handling is where most bugs hide.
- **Interview angle:** "When does your function do nothing (take epsilon)?"

3.4 Iterative form for left recursion / operators

- **What:** Replace $E \rightarrow E + T$ with a loop: `E(){ T(); while(look=='+'){match('+'); T();} }`.
- **Why it matters:** Avoids formal left-recursion elimination and preserves left-associativity naturally.
- **Interview angle:** "How do you parse a left-associative operator in recursive descent?"

3.5 Building the AST and semantic actions

- **What:** Each function returns a subtree/node; callers combine them.

- **Why it matters:** The parser usually outputs an AST, not just a yes/no.
- **Interview angle:** "How do you construct the tree during descent?"

4. Real-World Example

The TypeScript compiler's parser. It's a hand-written recursive-descent parser: functions like `parseStatement`, `parseExpression`, `parseBinaryExpression` call each other following the language grammar. It's hand-written (not generated) precisely because recursive descent gives fine-grained control over **error recovery** and **high-quality diagnostics** (";' expected") that users see in their editor - something table-generated parsers do less gracefully. Operator precedence is handled with precedence-climbing loops rather than deep recursion.

5. Diagrams / Mental Models

Function calls mirror the parse tree:

Grammar: $E \rightarrow T E'$ $\Rightarrow$ `E() { T(); Eprime(); }`

Call tree for `id + id`:

```
E()
|- T() -> id
|- Eprime()
    |- match('+')
    |- T() -> id
    |- Eprime() -> (epsilon)
```

Parse tree:

```
      E
     /\
    T E'
 id /\
  + T E'
   id (e)
```

One function per non-terminal:

```
Non-terminals: E, E', T -> functions: E(), Eprime(), T()
Terminals:    + id    -> match()/expect()
```

6. Common Interview Questions

Q1. What is recursive-descent parsing?

- Answer: Top-down parsing with one recursive function per non-terminal; the call stack is the parser stack.
- Common mistake: describing it as bottom-up or table-driven.

Q2. Predictive vs backtracking recursive descent?

- Answer: Predictive uses lookahead (no undo, linear); backtracking tries and rewinds (general, can be exponential).

Q3. Why can't plain recursive descent handle left recursion?

- Answer: `E()` would call `E()` first with no token consumed $\Rightarrow$ infinite recursion / stack overflow.

Q4. How do you parse a left-associative operator without formal left-recursion removal?

- Answer: Use a `while` loop: `E(){ T(); while(look==op){match(op); T();} }`.

Q5. How does the parser decide which production to use?

- Answer: By the lookahead token, using FIRST (and FOLLOW for epsilon) of each alternative.

Q6. What preprocessing does the grammar need?

- Answer: Remove left recursion, left factor common prefixes (for predictive descent).

Q7. How do you build an AST in recursive descent?

- Answer: Each function creates/returns its node; parents attach children returned by sub-calls.

Q8. Why do real compilers hand-write recursive-descent parsers?

- Answer: Better error messages/recovery, flexibility, and control vs generated table parsers.

Q9. What is the role of the `match/expect` function?

- Answer: Consume the expected terminal and advance lookahead, or raise a syntax error.

Q10. Is recursive descent LL(1)?

- Answer: Predictive recursive descent corresponds to LL(1) (or LL(k) with more lookahead); backtracking descent can go beyond.

7. Deep-Dive Questions

D1. How does precedence climbing / Pratt parsing relate to recursive descent?

- They're recursive-descent variants that handle operator precedence with a precedence parameter or binding-power table instead of many layered non-terminal functions - compact and efficient for expressions.

D2. How do you implement error recovery in recursive descent?

- Panic-mode: on error, skip tokens until a synchronizing token (in FOLLOW of the current construct, e.g., `;` or `}`), then resume. Phrase-level recovery inserts/deletes tokens locally.

D3. What's the downside of using the call stack as the parser stack?

- Deep right recursion can overflow the native stack; some parsers convert to iteration/explicit stacks for very deep inputs.

D4. How would you memoize a backtracking recursive-descent parser?

- Cache (non-terminal, position) -> result. This is **packrat parsing** (PEG), giving linear time at the cost of memory.

D5. Recursive descent vs table-driven LL(1) - when choose which?

- Recursive descent for hand-written parsers needing great errors/flexibility; table-driven when auto-generating from a grammar or when you want data-driven compactness.

8. Comparison Tables

Recursive Descent vs Table-driven LL(1)

Aspect	Recursive descent	Table-driven LL(1)
Stack	Program call stack (implicit)	Explicit stack
Code	One function per non-terminal	Generic driver + parse table
Readability	High, close to grammar	Table is opaque
Error messages	Easy to customize	Harder
Typical origin	Hand-written	Generated

Predictive vs Backtracking Recursive Descent

Aspect	Predictive	Backtracking
Lookahead	1 (or k)	Trial and error
Undo/rewind	No	Yes
Complexity	$O(n)$	Up to exponential (unless memoized)
Grammar class	LL(1)/LL(k)	PEG / broader

9. Common Mistakes

- Writing a function directly from a left-recursive rule (stack overflow).
- Not left factoring, so a function can't decide which branch to take.
- Handling epsilon incorrectly (using FIRST but forgetting FOLLOW).
- Forgetting to advance the lookahead in `match` (infinite loop).
- Not checking for end-of-input `$` at the end (accepting partial input).

10. Edge Cases / Special Cases

- **Left recursion:** must convert to loop or eliminate.
- **Right-associative operators:** recurse instead of loop (`A -> B op A`).
- **Optional / repeated constructs:** use `if/while` guided by FIRST.
- **Deeply nested input:** native stack overflow risk.
- **Ambiguity/dangling-else:** resolved in code (attach `else` to nearest `if`).

11. How to Explain in Interview

"Recursive-descent parsing implements top-down parsing as one recursive function per non-terminal; the program's call stack acts as the parser stack. Each function matches its rule - consuming terminals with an `expect` helper and calling other functions for sub-non-terminals - and it picks among alternatives using the lookahead token and FIRST/FOLLOW. The predictive form never backtracks, so it runs in linear time, but it

needs the grammar left-recursion-free and left-factored; left-associative operators are typically parsed with a **while** loop instead. It's the go-to method for hand-written parsers - Clang, TypeScript, Go all use it - mainly because it gives excellent, customizable error messages."

12. Quick Revision Notes

- One recursive function per non-terminal; **call stack = parser stack**.
- Predictive (lookahead, linear) vs backtracking (rewind, can be exponential).
- Needs left-recursion removal + left factoring.
- Left-associative operator => **while** loop; right-associative => recurse.
- **match/expect** consumes a terminal and advances lookahead.
- Used in real compilers (Clang, TS, Go) for great error messages.
- **Trap:** coding a left-recursive rule literally => stack overflow.

13. Practice Tasks

1. Write a recursive-descent parser for $E \rightarrow T E', E' \rightarrow + T E' \mid \text{epsilon}, T \rightarrow \text{id}$.
2. Rewrite it using a **while** loop directly from $E \rightarrow E + T \mid T$.
3. Extend it to handle ***** with correct precedence over **+**.
4. Add AST construction so it returns a tree, then evaluate it.
5. Implement panic-mode recovery that skips to **;** on error.
6. Add parentheses $F \rightarrow (E) \mid \text{id}$ and test $(\text{id} + \text{id}) * \text{id}$.

14. Final Cheat Sheet

- **Core definition:** Top-down parser as mutually recursive functions, one per non-terminal.
- **Why it matters:** Standard hand-written parser; great errors, close to grammar.
- **Most asked:** Write one for a small grammar; handle left recursion via loops.
- **Common comparison:** Recursive descent vs table LL(1); predictive vs backtracking.
- **One-line answer:** "Recursive-descent parsing codes one recursive function per non-terminal (the call stack being the parse stack), choosing productions by lookahead - the standard hand-written, top-down, LL(1) technique."

9. Predictive Parsing

1. Overview

Definition: Predictive parsing is a **non-backtracking top-down** method that uses **lookahead** (usually 1 token) plus **FIRST/FOLLOW** sets to decide, deterministically, which production to apply at each step. Its table-driven form is the **LL(1) parser**: a **parsing table + explicit stack + input buffer** driven by a fixed algorithm.

Why it matters: It parses in **linear time** with no backtracking, making it efficient and predictable. It formalizes exactly when top-down parsing can be done deterministically (LL(1) grammars).

Where it is used in real systems:

- LL(1) table-driven parsers in compiler courses and some tools.

- The decision logic inside predictive recursive-descent parsers.
- Simple, well-behaved languages and data formats (JSON is LL(1)).

Why interviewers ask about it: It combines FIRST/FOLLOW, table construction, and the parsing algorithm - a compact package that tests whether you can actually *run* a parser by hand.

2. Core Idea

Intuition: "Predict" the production to use by peeking one token ahead. Build a table $M[\text{Non-terminal}, \text{terminal}] = \text{production}$ once; then a generic driver repeatedly: look at (stack top, current input) and either match a terminal or expand a non-terminal using the table.

Real-world analogy: A GPS with a lookup table. At each intersection (non-terminal) it reads the road sign ahead (lookahead token) and consults its route table to pick exactly one turn - never doubling back.

The driver algorithm (table-driven predictive / LL(1)):

```
push $, push S ; read first token a
repeat:
  X = top of stack
  if X is terminal or $:
    if X == a: pop X, advance a          (match)
    else error
  else (X non-terminal):
    if M[X, a] = X -> Y1..Yk:
      pop X, push Yk..Y1 (reverse order); output the production
    else error
until X == $   (accept if input also at $)
```

Small example: For the factored E/T/F grammar, parsing `id + id` walks the table with entries like $M[E, id]=E \rightarrow TE'$, $M[E', +]=E' \rightarrow +TE'$, $M[E', \$]=E' \rightarrow \epsilon$.

3. Important Subtopics

3.1 The predictive parsing table $M[A, a]$

- **What:** Rows = non-terminals, columns = terminals (plus \$). Cell holds the production to use.
- **Why it matters:** The heart of the parser; conflicts (2 entries in a cell) mean not LL(1).
- **Interview angle:** "Construct the LL(1) table for this grammar."

3.2 Table construction rule (FIRST/FOLLOW)

- **What:** For each $A \rightarrow \alpha$: put $A \rightarrow \alpha$ in $M[A, a]$ for every a in $\text{FIRST}(\alpha)$; if ϵ in $\text{FIRST}(\alpha)$, also for every b in $\text{FOLLOW}(A)$ (and $\$$ if $\$$ in FOLLOW).
- **Why it matters:** The exact recipe examiners want.
- **Interview angle:** "State the rule for filling the LL(1) table."

3.3 Stack + input buffer model

- **What:** Explicit stack holds grammar symbols; input buffer holds remaining tokens ending in \$.

- **Why it matters:** Contrasts with recursive-descent's implicit call stack.
- **Interview angle:** "Trace the stack/input as you parse `id + id`."

3.4 Error detection and recovery

- **What:** A blank table cell or terminal mismatch = syntax error. Panic-mode recovery uses synchronizing tokens (FOLLOW sets).
- **Why it matters:** Real parsers must report and recover from errors.
- **Interview angle:** "How does a predictive parser detect an error and recover?"

4. Real-World Example

A configuration / expression mini-language in an app. Because the grammar is designed to be LL(1), the app ships a small table-driven predictive parser: a 2D array (the table) plus a 20-line driver loop. It's compact, fast, and data-driven - you can tweak the grammar by regenerating the table without rewriting control flow. JSON parsers exploit the same LL(1) property (each value's first token uniquely selects the branch) for linear-time, no-backtrack parsing at scale.

5. Diagrams / Mental Models

Components:

```
Input:  id + id $          <- tokens, ends with $
Stack:  $ E                <- grows/shrinks; top is decision point
Table M: rows=NTs, cols=terminals+$  -> M[X,a] = production
Driver: match terminals, expand non-terminals via M, until stack empties
```

Sample table (E/T/F grammar):

	id	+	*	(	)	\$
E	E->TE'			E->TE'		
E'		E'->+TE'			E'->e	E'->e
T	T->FT'			T->FT'		
T'		T'->e	T'->*FT'		T'->e	T'->e
F	F->id			F->(E)		
(e = epsilon)						

6. Common Interview Questions

Q1. What is predictive parsing?

- Answer: Non-backtracking top-down parsing that picks productions using lookahead + FIRST/FOLLOW; table-driven form = LL(1).
- Common mistake: equating it with backtracking top-down.

Q2. How is the predictive parsing table constructed?

- Answer: For $A \rightarrow \alpha$, add it to $M[A, a]$ for a in $\text{FIRST}(\alpha)$; if α is nullable, add for b in $\text{FOLLOW}(A)$.

Q3. What do the stack and input buffer contain?

- Answer: Stack = grammar symbols (init $\$S$); input = remaining tokens ending in $\$$.

Q4. What does a filled table cell tell the parser?

- Answer: Which production to expand the top non-terminal by, given the lookahead.

Q5. What happens if a cell has two productions?

- Answer: A conflict - the grammar is not LL(1).

Q6. How does the parser accept the input?

- Answer: When the stack reduces to $\$$ and the input is also at $\$$.

Q7. How are errors detected?

- Answer: Empty table cell for (non-terminal, lookahead), or terminal mismatch on match.

Q8. Predictive parsing vs recursive descent?

- Answer: Same top-down idea; predictive (table) uses an explicit stack + table, recursive descent uses the call stack + code.

Q9. Grammar prerequisites for predictive parsing?

- Answer: No left recursion, left factored, and LL(1) (no FIRST/FIRST or FIRST/FOLLOW conflicts).

Q10. Why is predictive parsing linear time?

- Answer: Each step either matches a token or expands a non-terminal a bounded number of times; no backtracking, so total work is proportional to input + tree size.

7. Deep-Dive Questions

D1. Why include FOLLOW(A) when alpha is nullable?

- Because if $A \Rightarrow^* \epsilon$, the parser must decide to erase A when the lookahead is whatever can legally follow A - that's exactly $\text{FOLLOW}(A)$.

D2. Two nullable alternatives - what conflict arises?

- If two productions of A are both nullable or their FIRST/FOLLOW overlap, a cell gets two entries (FIRST/FOLLOW conflict) - grammar not LL(1).

D3. How does panic-mode recovery use the table?

- On error, skip input tokens until one is in the FOLLOW (synchronizing) set of the stack-top non-terminal, pop that non-terminal, and continue - minimizing cascaded errors.

D4. Can predictive parsing handle every unambiguous grammar?

- No - many unambiguous grammars aren't LL(1) (e.g., need >1 lookahead or are inherently non-LL). LR is stronger.

D5. Relationship between the table and recursive descent code?

- Each table row corresponds to a function; each cell's production corresponds to a branch chosen by lookahead. They're two encodings of the same LL(1) logic.

8. Comparison Tables

Predictive (table LL(1)) vs Recursive Descent

Aspect	Table-driven predictive	Recursive descent
Stack	Explicit	Call stack
Representation	Table + driver	Functions
Generated vs hand	Usually generated	Usually hand-written
Error messages	Generic	Customizable
Same grammar class	LL(1)	LL(1)/LL(k)

Predictive vs Backtracking Top-Down

Aspect	Predictive	Backtracking
Decision	1 lookahead + table	Trial/undo
Time	Linear	Up to exponential
Needs LL(1)	Yes	No

9. Common Mistakes

- Filling the table using only FIRST and forgetting the FOLLOW rule for nullable productions.
- Pushing production RHS onto the stack in the wrong order (must push reversed so leftmost is on top).
- Forgetting the \$ end marker on stack and input.
- Treating a conflict cell as "pick either" - it means not LL(1).
- Confusing FIRST of a *string* alpha with FIRST of a single non-terminal.

10. Edge Cases / Special Cases

- **Nullable non-terminals:** require FOLLOW in table construction.
- **Multiple entries in a cell:** grammar not LL(1); needs transformation or a stronger parser.
- **epsilon at end of input:** $M[A, \$]$ may hold an epsilon-production.
- **Error cells:** blanks trigger recovery.
- **Left recursion / common prefix:** must be removed before building the table.

11. How to Explain in Interview

"Predictive parsing is deterministic top-down parsing: it looks one token ahead and uses FIRST/FOLLOW to pick exactly one production, so it never backtracks and runs in linear time. In its table-driven form - the LL(1) parser - I precompute a table $M[A, a]$: for each production $A \rightarrow \alpha$, I put it under every terminal in $\text{FIRST}(\alpha)$, and if α can be empty, under every terminal in $\text{FOLLOW}(A)$. Then a simple driver uses an explicit stack initialized with $\$S$, matching terminals and expanding non-terminals via the table until the stack empties. If any cell needs two productions, the grammar isn't LL(1)."

12. Quick Revision Notes

- Predictive = non-backtracking top-down using 1 lookahead + FIRST/FOLLOW.
- Table-driven form = **LL(1)**: table $M[A, a]$ + explicit stack ($\$S$) + input ($\dots \$$).
- Table rule: $A \rightarrow \alpha$ into $M[A, a]$ for a in $\text{FIRST}(\alpha)$; if nullable, for b in $\text{FOLLOW}(A)$.
- Push RHS **reversed** (leftmost symbol on top).
- Accept when stack = $\$$ and input = $\$$.
- Two entries in a cell => **not LL(1)**.
- **Trap**: forgetting the FOLLOW rule; wrong push order.

13. Practice Tasks

1. Compute FIRST/FOLLOW and build the LL(1) table for the E/T/F grammar.
2. Trace the stack/input/action columns while parsing $\text{id} + \text{id} * \text{id}$.
3. Trace a rejected input like $\text{id} + + \text{id}$ and show where the error is detected.
4. Add a synchronizing-token panic-mode recovery to your trace.
5. Show a grammar whose table has a conflict and explain why it's not LL(1).
6. Convert your table into equivalent recursive-descent functions.

14. Final Cheat Sheet

- **Core definition**: Deterministic top-down parsing via lookahead + FIRST/FOLLOW; table form = LL(1).
- **Why it matters**: Linear-time, no backtracking; defines the LL(1) grammar class.
- **Most asked**: Build the table; trace a parse; table-construction rule.
- **Common comparison**: Predictive vs recursive descent; predictive vs backtracking.
- **One-line answer**: "Predictive parsing chooses productions with one lookahead using FIRST/FOLLOW; the LL(1) table-driven parser runs a stack $\$S$ against the input in linear time with no backtracking."

10. FIRST and FOLLOW Sets

1. Overview

Definition:

- **FIRST(α)** = the set of terminals that can begin some string derived from α . If $\alpha \Rightarrow^* \epsilon$, then ϵ is in $\text{FIRST}(\alpha)$.
- **FOLLOW(A)** = the set of terminals that can appear **immediately after** the non-terminal A in some sentential form (with $\$$, the end marker, included if A can be last).

Why it matters: They are the fuel for **predictive/LL(1) table construction** and for detecting parser conflicts. FIRST tells you which production to start with; FOLLOW tells you when to apply an epsilon-production or reduce.

Where it is used in real systems:

- LL(1) parsing table construction.
- SLR parser construction (FOLLOW decides reductions).
- Parser generators (Yacc/Bison, ANTLR) compute them internally; grammar-conflict diagnostics use them.

Why interviewers ask about it: Computing FIRST/FOLLOW is the single most common written-exam parsing task, and it's a prerequisite to LL(1), SLR, and conflict analysis. Getting the rules exactly right (especially epsilon) is a strong signal.

2. Core Idea

Intuition: FIRST answers "what terminal can I see first if I start expanding this?" FOLLOW answers "if this non-terminal just finished, what terminal legally comes next?" Together they let a parser predict actions with one lookahead.

Real-world analogy: Reading a menu with combos. FIRST(combo) = what the first item of the combo could be. FOLLOW(combo) = what dish typically comes right after this combo in the full meal. Knowing both lets a waiter anticipate your order from the first word.

FIRST rules:

1. If X is a terminal, $\text{FIRST}(X) = \{X\}$.
2. If $X \rightarrow \text{epsilon}$, add epsilon to $\text{FIRST}(X)$.
3. If $X \rightarrow Y_1 Y_2 \dots Y_k$: add $\text{FIRST}(Y_1)$ minus $\{\text{epsilon}\}$. If epsilon in $\text{FIRST}(Y_1)$, add $\text{FIRST}(Y_2)$ minus $\{\text{epsilon}\}$, and so on. If ALL Y_i are nullable, add epsilon.

FOLLOW rules:

1. Add $\$$ to $\text{FOLLOW}(\text{start symbol})$.
2. For $A \rightarrow \alpha B \beta$: add $\text{FIRST}(\beta)$ minus $\{\text{epsilon}\}$ to $\text{FOLLOW}(B)$.
3. For $A \rightarrow \alpha B$ (B at end) or $A \rightarrow \alpha B \beta$ where β is nullable: add $\text{FOLLOW}(A)$ to $\text{FOLLOW}(B)$.

Small example (E/T/F grammar):

```
E -> T E'      E' -> + T E' | epsilon
T -> F T'      T' -> * F T' | epsilon
F -> ( E ) | id
```

```
FIRST(F)=FIRST(T)=FIRST(E) = { (, id }
FIRST(E') = { +, epsilon }    FIRST(T') = { *, epsilon }
FOLLOW(E) = FOLLOW(E') = { ), $ }
FOLLOW(T) = FOLLOW(T') = { +, ), $ }
FOLLOW(F) = { *, +, ), $ }
```

3. Important Subtopics

3.1 Nullable non-terminals

- **What:** A is nullable if $A \Rightarrow^* \epsilon$. Compute this first; it drives both FIRST and FOLLOW.
- **Why it matters:** Nullability decides whether FIRST "carries over" to the next symbol and whether FOLLOW propagates.
- **Interview angle:** "Which non-terminals are nullable here?"

3.2 FIRST of a string (not just a symbol)

- **What:** $\text{FIRST}(X_1 X_2 \dots X_k)$ uses the carry-over rule across nullable symbols.
- **Why it matters:** LL(1) tables use FIRST of an entire production RHS.
- **Interview angle:** "Compute $\text{FIRST}(A B c)$ where A, B may be nullable."

3.3 FOLLOW propagation and iteration

- **What:** FOLLOW is computed by repeatedly applying rules until no set changes (fixed point).
- **Why it matters:** A single pass is often insufficient; you must iterate.
- **Interview angle:** "Why do you iterate FOLLOW computation to a fixed point?"

3.4 epsilon is in FIRST but never in FOLLOW

- **What:** FOLLOW sets contain terminals and $\$$ only - never epsilon.
- **Why it matters:** A very common student error.
- **Interview angle:** "Can epsilon be in a FOLLOW set?" (No.)

4. Real-World Example

Yacc/Bison and ANTLR grammar analysis. When you feed a grammar to a parser generator, it computes FIRST/FOLLOW (or LR item lookaheads) internally to build tables and to report conflicts. For an LL tool like ANTLR, overlapping FIRST sets between alternatives produce the warning that the grammar isn't LL-decidable at that point; for SLR-style analysis, FOLLOW sets determine where reduce actions go and thus whether a shift/reduce conflict exists. So FIRST/FOLLOW are literally what the tool runs under the hood.

5. Diagrams / Mental Models

FIRST vs FOLLOW at a glance:

```

      A -> alpha B beta
          ^^^^^ ^^^^^
FIRST(A): terminals that can START A (look inside alpha)
FOLLOW(B): terminals right AFTER B (= FIRST(beta), plus FOLLOW(A) if beta
nullable)

```

Carry-over when nullable:

```
FIRST(Y1 Y2 Y3):  
  add FIRST(Y1)\{e}  
  if Y1 nullable -> add FIRST(Y2)\{e}  
  if Y1,Y2 nullable -> add FIRST(Y3)\{e}  
  if Y1,Y2,Y3 all nullable -> add epsilon
```

6. Common Interview Questions

Q1. Define FIRST and FOLLOW.

- Answer: FIRST(α) = terminals that can begin a string derived from α (plus epsilon if nullable). FOLLOW(A) = terminals that can immediately follow A (plus \$).
- Common mistake: putting epsilon in FOLLOW, or defining FOLLOW as "what A derives".

Q2. Compute FIRST and FOLLOW for the E/T/F grammar.

- Answer: as in section 2.

Q3. When is epsilon in FIRST(A)?

- Answer: When $A \Rightarrow^* \text{epsilon}$ (A is nullable).

Q4. Can epsilon be in FOLLOW(A)?

- Answer: No - FOLLOW contains only terminals and \$.

Q5. Why do we need FOLLOW in LL(1)?

- Answer: To decide when to apply an epsilon-production - use it when the lookahead is in FOLLOW(A).

Q6. How do you compute FIRST of a multi-symbol string?

- Answer: Union FIRST of symbols left to right, carrying over past nullable ones; add epsilon only if all are nullable.

Q7. What is FOLLOW of the start symbol guaranteed to contain?

- Answer: The end marker \$.

Q8. Why iterate to a fixed point for FOLLOW?

- Answer: FOLLOW dependencies are recursive (FOLLOW(B) may depend on FOLLOW(A)); repeat until stable.

Q9. Given $A \rightarrow B C$, when does FOLLOW(A) flow into FOLLOW(B)?

- Answer: When C is nullable (or absent), so B can be effectively last.

Q10. What's the difference between FIRST and FOLLOW usage in the LL(1) table?

- Answer: FIRST decides columns for a production normally; FOLLOW decides columns when the production is nullable (epsilon case).

7. Deep-Dive Questions

D1. Give an algorithm to compute all nullable non-terminals.

- Initialize nullable = { A : A → ε }. Repeat: if $A \rightarrow Y_1 \dots Y_k$ with all Y_i nullable, add A. Until fixed point.

D2. FIRST/FIRST vs FIRST/FOLLOW conflict - define both.

- FIRST/FIRST: two productions of A have overlapping FIRST sets (can't choose by lookahead).
- FIRST/FOLLOW: a nullable production's FOLLOW(A) overlaps another production's FIRST - can't decide between expanding and taking epsilon.

D3. Why can a naive single-pass FOLLOW computation be wrong?

- Because FOLLOW(A) contributions to FOLLOW(B) might be added after you've already processed B; recursion/back-edges require iterating to a fixed point.

D4. How do FIRST/FOLLOW relate to SLR construction?

- SLR places a reduce action $A \rightarrow \alpha$ in state columns for every terminal in FOLLOW(A). Overly broad FOLLOW causes SLR conflicts that CLR/LALR (using precise lookaheads) avoid.

D5. Does adding epsilon-productions always enlarge FOLLOW sets?

- It can, because nullable symbols cause FOLLOW to propagate through more positions, widening lookahead sets and risking conflicts.

8. Comparison Tables

FIRST vs FOLLOW

Aspect	FIRST(α)	FOLLOW(A)
Defined for	Any grammar symbol/string	A non-terminal only
Contains	Terminals that can begin α	Terminals that can follow A
epsilon allowed?	Yes (if nullable)	No
\$ allowed?	No (it's not a start terminal)	Yes
Used for	Choosing a production to start	Deciding epsilon / reductions

FIRST/FIRST vs FIRST/FOLLOW conflict

Conflict	Cause	Fix
FIRST/FIRST	Two alternatives share a FIRST terminal	Left factoring
FIRST/FOLLOW	Nullable production's FOLLOW overlaps a FIRST	Grammar redesign / stronger parser

9. Common Mistakes

- Putting **epsilon in FOLLOW** (never allowed).
- Forgetting to add $\$$ to FOLLOW(start).
- Not carrying FIRST across nullable symbols (stopping too early).
- Computing FOLLOW in one pass instead of to a fixed point.
- Adding epsilon to FOLLOW(B) via rule 3 - only FOLLOW(A)'s *terminals* propagate, not epsilon.
- Confusing FIRST of a non-terminal with FIRST of a whole production string.

10. Edge Cases / Special Cases

- **Fully nullable RHS:** FIRST includes epsilon; FOLLOW(A) flows to the last symbol.
- **Start symbol also appears mid-rule:** its FOLLOW gets both $\$$ and internal contributions.
- **Chains of nullable non-terminals:** FIRST and FOLLOW propagate through all of them.
- **Self-referential FOLLOW:** requires iteration.
- **Terminal-only productions:** FIRST is trivially the terminal; contributes to others' FOLLOW.

11. How to Explain in Interview

"FIRST(alpha) is the set of terminals that can appear at the start of any string derived from alpha, plus epsilon if alpha can derive the empty string. FOLLOW(A) is the set of terminals that can come immediately after A in some derivation, always including $\$$ for the start symbol - and FOLLOW never contains epsilon. I compute nullable non-terminals first, then FIRST bottom-up carrying over across nullable symbols, then FOLLOW by iterating three rules to a fixed point: $\$$ in FOLLOW(start); FIRST(beta) into FOLLOW(B) for $A \rightarrow \text{alpha } B \text{ beta}$; and FOLLOW(A) into FOLLOW(B) when beta is nullable or B is last. These sets drive LL(1) and SLR table construction."

12. Quick Revision Notes

- FIRST(alpha): terminals that can begin alpha (+ epsilon if nullable).
- FOLLOW(A): terminals right after A (+ $\$$ for start). **Never epsilon.**
- Compute **nullable** first, then FIRST, then FOLLOW (iterate to fixed point).
- FIRST carries across nullable symbols; add epsilon only if all nullable.
- FOLLOW rules: $\$$ in FOLLOW(start); FIRST(beta){e} into FOLLOW(B); FOLLOW(A) into FOLLOW(B) if beta nullable.
- Used by LL(1) tables and SLR reductions.
- **Trap:** epsilon in FOLLOW; single-pass FOLLOW; not carrying FIRST past nullables.

13. Practice Tasks

1. Compute nullable, FIRST, FOLLOW for $S \rightarrow A B$; $A \rightarrow a \mid \text{epsilon}$; $B \rightarrow b \mid \text{epsilon}$.
2. Do the full FIRST/FOLLOW for the E/T/F grammar (verify against section 2).
3. Compute FIRST(A B c) where A and B are nullable.
4. Find a grammar with a FIRST/FIRST conflict and one with a FIRST/FOLLOW conflict.
5. Show a FOLLOW computation that needs two iterations to stabilize.
6. Use your FIRST/FOLLOW to fill an LL(1) table and check for conflicts.

14. Final Cheat Sheet

- **Core definition:** FIRST = terminals that can start alpha (+epsilon); FOLLOW = terminals that can follow A (+\$).
- **Why it matters:** Drives LL(1) and SLR table construction and conflict detection.
- **Most asked:** Compute FIRST/FOLLOW for a grammar; epsilon rules.
- **Common comparison:** FIRST vs FOLLOW; FIRST/FIRST vs FIRST/FOLLOW conflict.
- **One-line answer:** "FIRST(alpha) is what can begin alpha (plus epsilon if nullable); FOLLOW(A) is what can immediately follow A (plus \$, never epsilon) - together they let a one-lookahead parser predict productions."

11. LL(1) Parsing

1. Overview

Definition: LL(1) parsing is table-driven predictive parsing where:

- First **L** = scan input **L**eft to right,
- Second **L** = produce a **L**eftmost derivation,
- **1** = use **1** token of lookahead.

A grammar is **LL(1)** iff its predictive parsing table has **no cell with more than one production** (no conflicts). It uses a **stack + parsing table** $M[A, a]$ + **input buffer**.

Why it matters: LL(1) is the cleanest, most teachable deterministic top-down parser: linear time, small tables, easy to hand-implement. It precisely characterizes which grammars simple predictive parsers can handle.

Where it is used in real systems:

- Hand-written and generated predictive parsers for well-designed languages/DSLs.
- JSON and many config formats (LL(1) by design).
- Teaching and rapid prototyping of parsers.

Why interviewers ask about it: It's the capstone of the top-down half - you must combine FIRST/FOLLOW, table building, the driver algorithm, conflict detection, and the LL(1) condition. Very common in written rounds.

2. Core Idea

Intuition: Precompute, for each (non-terminal, next token) pair, exactly which production to use. Then run a stack machine: expand non-terminals via the table, match terminals against input, until done. If the table is conflict-free, one lookahead token always suffices.

Real-world analogy: A vending machine keypad map: each (current menu, button pressed) maps to exactly one next action. No ambiguity, no going back.

LL(1) condition (formal): For every non-terminal A with productions $A \rightarrow \alpha \mid \beta$:

1. FIRST(alpha) and FIRST(beta) are disjoint (no FIRST/FIRST conflict).
2. At most one of alpha, beta is nullable.
3. If beta is nullable, FIRST(alpha) and FOLLOW(A) are disjoint (no FIRST/FOLLOW conflict).

Small example (parse `id + id` with E/T/F table):

Stack	Input	Action
\$E	id + id \$	E -> T E'
\$E'T	id + id \$	T -> F T'
\$E'T'F	id + id \$	F -> id
\$E'T'id	id + id \$	match id
\$E'T'	+ id \$	T' -> epsilon
\$E'	+ id \$	E' -> + T E'
\$E'T+	+ id \$	match +
\$E'T	id \$	T -> F T'
\$E'T'F	id \$	F -> id
\$E'T'id	id \$	match id
\$E'T'	\$	T' -> epsilon
\$E'	\$	E' -> epsilon
\$	\$	accept

3. Important Subtopics

3.1 Building the LL(1) table

- **What:** For $A \rightarrow \alpha$: put it in $M[A, a]$ for each a in $\text{FIRST}(\alpha)$; if epsilon in $\text{FIRST}(\alpha)$, also for each b in $\text{FOLLOW}(A)$ (and $\$$ if in FOLLOW).
- **Why it matters:** The core construction; a conflict here means not LL(1).
- **Interview angle:** "Construct the table and identify conflicts."

3.2 The LL(1) condition and conflicts

- **What:** No table cell may hold two productions. Conflicts: FIRST/FIRST or FIRST/FOLLOW.
- **Why it matters:** Determines whether a grammar is LL(1) at all.
- **Interview angle:** "Is this grammar LL(1)? Justify with the conditions."

3.3 Grammar prerequisites

- **What:** LL(1) requires no left recursion and left factoring; even then a grammar may not be LL(1).
- **Why it matters:** Transformations are necessary but not sufficient.
- **Interview angle:** "You removed left recursion and factored - is it now LL(1)?" (Maybe not.)

3.4 Error detection and recovery

- **What:** Blank cell or terminal mismatch = error; panic-mode uses FOLLOW as synchronizing set.
- **Why it matters:** Production parsers need graceful recovery.
- **Interview angle:** "How does LL(1) detect and recover from errors?"

3.5 LL(1) vs LL(k) and limits

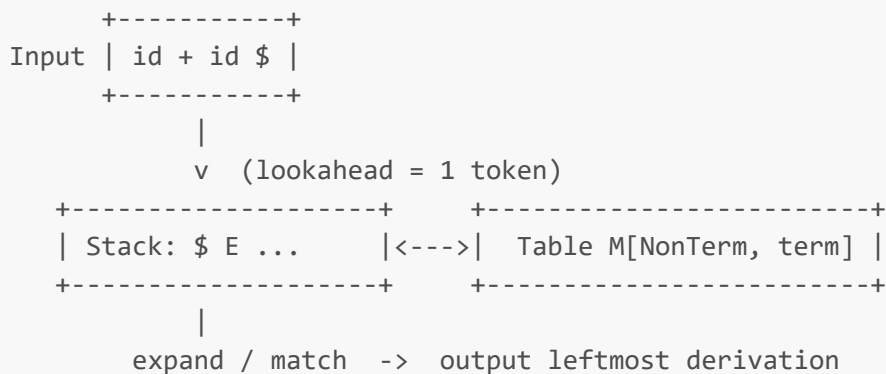
- **What:** LL(1) uses 1 lookahead; some grammars need more (LL(k)) or are not LL at all.
- **Why it matters:** Explains why LR is used for complex languages.
- **Interview angle:** "Give a grammar that's not LL(1) but is LL(2) or LR(1)."

4. Real-World Example

A JSON parser. JSON's grammar is LL(1): the first token of any **value** uniquely determines the production (`{` => object, `[` => array, `"` => string, `t/f/n` => literals, `digit/-` => number). So a table-driven or predictive recursive-descent LL(1) parser handles JSON with one lookahead, in linear time, no backtracking - which is why JSON deserialization is fast and simple across every language's standard library.

5. Diagrams / Mental Models

LL(1) machine:



LL(1) decision:

```

top of stack = A, lookahead = a
M[A,a] empty      -> error
M[A,a] = A->gamma -> pop A, push gamma reversed
top = terminal a  -> match & advance

```

6. Common Interview Questions

Q1. What does LL(1) stand for?

- Answer: Left-to-right scan, Leftmost derivation, 1 lookahead token.
- Common mistake: saying the second L means "left recursion".

Q2. When is a grammar LL(1)?

- Answer: Its predictive table has no multiply-defined entries; equivalently the FIRST/FIRST and FIRST/FOLLOW disjointness conditions hold.

Q3. Build the LL(1) parsing table for the E/T/F grammar.

- Answer: as in topic 9 section 5.

Q4. What are the two kinds of LL(1) conflicts?

- Answer: FIRST/FIRST (overlapping FIRST sets) and FIRST/FOLLOW (nullable production overlaps FOLLOW).

Q5. Is every unambiguous grammar LL(1)?

- Answer: No - many unambiguous grammars are not LL(1).

Q6. Is every LL(1) grammar unambiguous?

- Answer: Yes - LL(1) implies unambiguous.

Q7. What transformations help make a grammar LL(1)?

- Answer: Eliminate left recursion and left factor - necessary but not always sufficient.

Q8. Trace an LL(1) parse of `id + id`.

- Answer: as in section 2.

Q9. How does the LL(1) parser accept/reject?

- Answer: Accept when stack and input both reach `$`; reject on empty cell or mismatch.

Q10. LL(1) vs LR(1) power?

- Answer: LL(1) is strictly weaker; every LL(1) grammar is LR(1), not vice versa.

7. Deep-Dive Questions

D1. Prove LL(1) implies unambiguous.

- If a grammar were ambiguous, some string would have two leftmost derivations, requiring two productions for the same (A, lookahead) at some step - a table conflict, contradicting LL(1).

D2. Give a grammar that's LR(1) but not LL(1).

- `S -> A a | B b; A -> c; B -> c` is not LL(1) (both start with `c`, can't decide with 1 lookahead) but is easily LR(1)/LALR. Also any left-recursive expression grammar.

D3. Why is left recursion fatal specifically for LL(1)?

- The predictive table would need to expand A on a lookahead that's in FIRST of A itself, recursing without consuming input - no finite table entry resolves it.

D4. How much does LL(k) buy you over LL(1)?

- More lookahead resolves some FIRST/FIRST overlaps decidable within k tokens, but there are grammars not LL(k) for any k, and LL(k) tables grow fast; LR(1) remains strictly more powerful.

D5. How does panic-mode recovery choose synchronizing tokens?

- Typically FOLLOW(A) of the non-terminal on top of stack (and often statement delimiters like `;`, `}`); skip input until a sync token, pop A, resume - limiting cascaded errors.

8. Comparison Tables

LL(1) vs LR(1)

Aspect	LL(1)	LR(1)
Direction	Top-down	Bottom-up
Derivation	Leftmost	Rightmost (reverse)
Left recursion	Forbidden	Allowed/preferred
Left factoring	Required	Not required
Power	Weaker (subset)	Stronger (superset)
Table size	Small	Larger
Decision point	Before expanding	At reduce time

LL(1) conflicts

Conflict	Condition	Typical fix
FIRST/FIRST	$\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) \neq \text{empty}$	Left factoring
FIRST/FOLLOW	β nullable, $\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) \neq \text{empty}$	Rewrite grammar / use LR

9. Common Mistakes

- Thinking the second L means "left recursion" (it means Leftmost derivation).
- Assuming removing left recursion + factoring guarantees LL(1).
- Filling table cells with FIRST only, ignoring the FOLLOW rule for nullable productions.
- Believing unambiguous $\Rightarrow$ LL(1) (false).
- Pushing the RHS onto the stack in the wrong (non-reversed) order.
- Treating a two-entry cell as acceptable.

10. Edge Cases / Special Cases

- **Dangling-else:** not LL(1) as-is (FIRST/FOLLOW conflict on `else`); resolved by rule preference.
- **Nullable productions:** require the FOLLOW rule; common conflict source.
- **Grammar LL(1) only after transformation** - or never (needs LR).
- **Error cells** trigger recovery.
- **Multiple nullable alternatives** for one non-terminal $\Rightarrow$ automatically not LL(1).

11. How to Explain in Interview

"LL(1) parsing scans left to right, produces a leftmost derivation, and uses one lookahead token. It's table-driven: I compute FIRST/FOLLOW, then build a table $M[A, a]$ where $A \rightarrow \alpha$ goes under $\text{FIRST}(\alpha)$, plus under $\text{FOLLOW}(A)$ if α is nullable. A driver runs a stack initialized to $\$S$, expanding non-terminals via the table and matching terminals until both stack and input hit $\$$. The grammar is LL(1) only if no table cell has two productions - which requires disjoint FIRST sets among alternatives and no FIRST/FOLLOW overlap for nullable ones. It needs left-recursion removal and left factoring first, and it's strictly weaker than LR(1), but it's simple, linear-time, and great for clean grammars like JSON."

12. Quick Revision Notes

- $LL(1)$ = Left-to-right, Leftmost derivation, 1 lookahead.
- $LL(1)$ iff parsing table has **no multiply-defined cells**.
- Conditions: disjoint FIRST of alternatives; nullable in at most one; $FIRST(\alpha) \cap FOLLOW(A)$ empty if α nullable.
- Needs left-recursion removal + left factoring (necessary, not sufficient).
- **$LL(1) \Rightarrow \text{unambiguous}$** , but $\text{unambiguous} \not\Rightarrow LL(1)$.
- **$LL(1) \subset LR(1)$** in power.
- **Trap:** second L = Leftmost, not left recursion.

13. Practice Tasks

1. Build the full $LL(1)$ table for the E/T/F grammar and parse `id * ( id + id )`.
2. Show the dangling-else grammar is not $LL(1)$ (find the conflicting cell).
3. Take `S -> A a | B b`; `A -> c`; `B -> c` and show the FIRST/FIRST conflict.
4. Fix a non- $LL(1)$ grammar by factoring, then rebuild the table.
5. Trace an error input and apply panic-mode recovery using FOLLOW.
6. Prove your resulting grammar is $LL(1)$ by verifying the three conditions.

14. Final Cheat Sheet

- **Core definition:** Predictive, table-driven, left-to-right, leftmost, 1-lookahead parser.
- **Why it matters:** Linear-time deterministic top-down; defines the $LL(1)$ grammar class.
- **Most asked:** Build the table; check $LL(1)$ conditions; $LL(1)$ vs $LR(1)$.
- **Common comparison:** $LL(1)$ vs $LR(1)$; FIRST/FIRST vs FIRST/FOLLOW conflict.
- **One-line answer:** " $LL(1)$ is predictive top-down parsing (left-to-right, leftmost, 1 lookahead) that works exactly when the FIRST/FOLLOW-built table has no conflicting cell - simple and linear but strictly weaker than LR."

12. Bottom-Up Parsing

1. Overview

Definition: Bottom-up parsing builds the parse tree from the **leaves (input tokens) up to the root (start symbol)**. It repeatedly **reduces** a substring matching the right-hand side of a production to that production's left-hand side, effectively constructing a **rightmost derivation in reverse**. The dominant technique is **shift-reduce** parsing (LR family).

Why it matters: Bottom-up parsers handle a **larger class of grammars** than top-down (all deterministic CFLs = $LR(1)$), including left-recursive grammars, without transformation. This is why real parser generators (Yacc/Bison) are bottom-up.

Where it is used in real systems:

- Yacc/Bison-generated parsers for C, SQL engines, many DSLs.
- Language front-ends where a formal grammar is compiled to tables.
- Any tool that wants maximal grammar coverage with generated parsers.

Why interviewers ask about it: It's conceptually harder (handles, viable prefixes, item automata) and directly leads to SLR/CLR/LALR - a rich area that separates strong candidates.

2. Core Idea

Intuition: Instead of predicting from the top, watch the input accumulate on a stack; whenever the top of the stack matches a production's RHS (a **handle**), replace it with the LHS (**reduce**). Keep going until only the start symbol remains. It's like assembling a structure from parts.

Real-world analogy: Building with LEGO by following instructions in reverse - you see small assembled clusters (handles) and snap them into bigger components until the whole model (start symbol) is complete.

Small example: Grammar $E \rightarrow E + T \mid T, T \rightarrow T * F \mid F, F \rightarrow id$. Input $id + id$:

Stack	Input	Action
(empty)	$id + id \$$	shift id
id	$+ id \$$	reduce $F \rightarrow id$; then $T \rightarrow F$
T	$+ id \$$	reduce $E \rightarrow T$
E	$+ id \$$	shift $+$
$E +$	$id \$$	shift id
$E + id$	$\$$	reduce $F \rightarrow id$; $T \rightarrow F$
$E + T$	$\$$	reduce $E \rightarrow E + T$
E	$\$$	accept

Reading the reductions bottom to top reconstructs a rightmost derivation.

Step-by-step (shift-reduce skeleton):

1. **Shift** the next input token onto the stack.
2. If the stack top is a handle, **reduce** it to the corresponding non-terminal.
3. Repeat until the stack is the start symbol and input is empty (**accept**).
4. If neither shift nor reduce is valid, **error**.

3. Important Subtopics

3.1 Reductions and rightmost derivation in reverse

- **What:** Each reduction is one step of a rightmost derivation, applied backwards.
- **Why it matters:** Explains the theoretical model of LR parsing.
- **Interview angle:** "What derivation does bottom-up parsing construct?" (Rightmost, in reverse.)

3.2 Handles

- **What:** A **handle** is the substring (and the production) that should be reduced next to stay on a valid rightmost derivation.
- **Why it matters:** Correct parsing = always reducing the handle; the whole difficulty is *finding* it.
- **Interview angle:** "What is a handle and why is finding it hard?"

3.3 Shift-reduce mechanism and conflicts

- **What:** Two actions - shift or reduce; ambiguity/limitations cause shift/reduce and reduce/reduce conflicts.
- **Why it matters:** Central to LR table behavior.
- **Interview angle:** "What are the two conflict types?"

3.4 Handles vs top-down prediction

- **What:** Top-down predicts a production before seeing the RHS; bottom-up decides after seeing the whole RHS.
- **Why it matters:** That "decide late" is why bottom-up is more powerful.
- **Interview angle:** "Why is bottom-up more powerful than top-down?"

4. Real-World Example

A SQL database's query parser (Bison-generated). SQL grammars are large and naturally left-recursive (expression lists, joins). A bottom-up LALR parser generated by Bison shifts tokens and reduces by grammar rules, building the query's parse structure without any left-recursion removal. The reduction sequence is a reverse rightmost derivation of the SQL statement, which the engine turns into a logical query plan. Bottom-up is chosen precisely because it swallows the messy, left-recursive grammar directly.

5. Diagrams / Mental Models

Direction:

```
Bottom-up:  input tokens (leaves)  --reduce-->  ...  --reduce-->  S (root)
           built from the bottom up
```

Shift vs reduce:

```
SHIFT:  move next input token onto stack
REDUCE:  stack top matches RHS of A->beta  ->  replace beta with A
ACCEPT:  stack = S, input = $
```

Handle pruning:

```
Rightmost derivation:  S => ... => gamma A w => gamma beta w => ... => input
Reverse (parsing):    input -> ... -> gamma beta w -> gamma A w -> ... -> S
                        ^^^^ handle reduced each step
```

6. Common Interview Questions

Q1. What is bottom-up parsing?

- Answer: Building the tree from leaves to root by reducing handles; constructs a rightmost derivation in reverse.

- Common mistake: confusing with top-down.

Q2. What are the two basic actions?

- Answer: Shift (push input) and reduce (replace handle with LHS). Plus accept and error.

Q3. What derivation does it build?

- Answer: Rightmost derivation, in reverse.

Q4. Why can bottom-up handle left recursion?

- Answer: It decides reductions after seeing the full RHS, so left-recursive rules just reduce repeatedly with a bounded stack.

Q5. What is a handle?

- Answer: The substring + production to reduce next to retrace the rightmost derivation.

Q6. Bottom-up vs top-down power?

- Answer: Bottom-up (LR) handles a strictly larger grammar class than top-down (LL).

Q7. What are the conflict types?

- Answer: Shift/reduce and reduce/reduce.

Q8. Why do real parser generators use bottom-up?

- Answer: Broader grammar coverage, no left-recursion/factoring transforms, handles ambiguity via precedence declarations.

Q9. Does bottom-up need left factoring?

- Answer: No.

Q10. How does bottom-up decide between shift and reduce?

- Answer: Using an automaton of items (LR states) plus lookahead - the LR parsing table.

7. Deep-Dive Questions

D1. Why is finding the handle the crux of bottom-up parsing?

- Reducing a non-handle substring leads to a dead end (can't reach S). The LR construction builds a DFA over "viable prefixes" that recognizes exactly when a handle is on top of the stack.

D2. Relationship between the stack contents and viable prefixes.

- At every step the stack holds a **viable prefix** - a prefix of some right-sentential form that doesn't extend past a handle. The LR automaton's states track this.

D3. Why is bottom-up strictly more powerful than top-down?

- Deciding at reduce time (after the whole RHS is visible) gives more information than predicting at expansion time; formally $LL(k) \subsetneq LR(k)$, and $LR(1)$ captures all deterministic CFLs.

D4. Can a bottom-up parser be built for an ambiguous grammar?

- Not deterministically without extra rules - ambiguity yields conflicts; tools resolve them via precedence/associativity declarations or you use GLR.

D5. Complexity of shift-reduce parsing?

- Linear $O(n)$ for LR grammars with the precomputed table; each token is shifted once and reductions are bounded by tree size.

8. Comparison Tables

Bottom-Up vs Top-Down

Aspect	Bottom-up (LR)	Top-down (LL)
Tree built	Leaves to root	Root to leaves
Derivation	Rightmost (reverse)	Leftmost
Left recursion	Fine/preferred	Must remove
Left factoring	Not needed	Needed
Grammar power	Larger (LR)	Smaller (LL)
Decision timing	At reduce (late)	At expand (early)
Typical origin	Generated (Yacc)	Hand-written / generated

Shift-reduce actions

Action	Meaning	When
Shift	Push next token	Handle not yet complete
Reduce	Replace handle with LHS	Handle on top of stack
Accept	Done	Stack = S, input = \$
Error	Reject	No valid action

9. Common Mistakes

- Confusing bottom-up (rightmost, reverse) with top-down (leftmost).
- Thinking bottom-up needs left-recursion removal or factoring (it doesn't).
- Reducing a substring that isn't a handle (leads to failure).
- Believing shift/reduce parsing is inherently ambiguous-safe (conflicts can occur).
- Saying it builds a leftmost derivation.

10. Edge Cases / Special Cases

- **Shift/reduce conflict:** stack top could be reduced or extended - needs lookahead/precedence.
- **Reduce/reduce conflict:** two productions match - grammar problem.
- **Ambiguous grammars:** need precedence/associativity declarations.
- **epsilon-productions:** reduce on empty handle in the right state.
- **Very deep nesting:** stack grows but stays linear in input.

11. How to Explain in Interview

"Bottom-up parsing constructs the parse tree from the input tokens up to the start symbol. It works by shift-reduce: it shifts tokens onto a stack, and whenever the top of the stack forms the right-hand side of a production - a handle - it reduces that to the left-hand side. The sequence of reductions is a rightmost derivation read in reverse. Because it commits to a production only after seeing the entire right-hand side, it handles left-recursive grammars directly and covers a strictly larger class than top-down LL parsing - which is why Yacc/Bison and most real parser generators are bottom-up. The engineering challenge is finding the handle, which the LR automaton solves."

12. Quick Revision Notes

- Bottom-up = leaves to root; **rightmost derivation in reverse**.
- Actions: **shift, reduce**, accept, error.
- **Handle** = substring+production to reduce next.
- Handles left recursion; **no** left factoring needed.
- More powerful than top-down: **LL $\subset$ LR**.
- Decides at **reduce time** (late) - source of its power.
- **Trap:** don't call it leftmost; don't transform left recursion away.

13. Practice Tasks

1. Hand-trace a shift-reduce parse of `id * id + id` with the E/T/F grammar.
2. Mark the handle at each reduction step.
3. Reconstruct the rightmost derivation by reversing your reductions.
4. Show a step where both shift and reduce look possible (a conflict).
5. Parse `( id )` bottom-up and draw the resulting tree from the bottom.
6. Compare the same string parsed top-down vs bottom-up side by side.

14. Final Cheat Sheet

- **Core definition:** Build tree leaves-to-root via shift-reduce; rightmost derivation reversed.
- **Why it matters:** Larger grammar class (LR); no left-recursion/factoring transforms.
- **Most asked:** Shift vs reduce; handle; why more powerful than top-down.
- **Common comparison:** Bottom-up vs top-down; shift vs reduce.
- **One-line answer:** "Bottom-up parsing reduces handles from the input up to the start symbol - a reversed rightmost derivation - handling left recursion directly and covering strictly more grammars than top-down."

13. Shift-Reduce Parsing

1. Overview

Definition: Shift-reduce parsing is the concrete mechanism behind bottom-up parsing. It uses a **stack** and an **input buffer** and performs four kinds of actions:

- **Shift:** push the next input token onto the stack.
- **Reduce:** replace a handle (RHS of a production) on the stack top with its LHS.
- **Accept:** stack holds the start symbol and input is exhausted.
- **Error:** no valid action.

Why it matters: It's the operational core of all LR parsers (SLR, CLR, LALR) and the framework in which handles, viable prefixes, and conflicts are defined. Understanding shift-reduce is prerequisite to everything LR.

Where it is used in real systems:

- Every Yacc/Bison-generated parser executes a shift-reduce loop driven by ACTION/GOTO tables.
- Expression evaluators using operator-precedence parsing.
- Compilers and interpreters with generated parsers.

Why interviewers ask about it: It's the most tangible bottom-up procedure; interviewers love "trace the shift-reduce parse and identify the conflict."

2. Core Idea

Intuition: Grow the stack by shifting tokens until the top matches a complete production body, then collapse (reduce) it. The tricky decisions are *when to shift vs reduce* and *which production to reduce by* - resolved by the LR automaton and lookahead.

Real-world analogy: A cafeteria tray line. You keep adding items (shift) until you have a complete meal combo (handle), then you swap the combo for a single "meal" token (reduce). Repeat until you have one "tray" (start symbol).

Small example: Grammar $E \rightarrow E + E \mid E * E \mid id$ (ambiguous - shows conflicts). Input $id + id * id$:

Stack	Input	Action
	$id + id * id \$$	shift
id	$+ id * id \$$	reduce $E \rightarrow id$
E	$+ id * id \$$	shift $+$
$E +$	$id * id \$$	shift id
$E + id$	$* id \$$	reduce $E \rightarrow id$
$E + E$	$* id \$$	*** shift/reduce conflict *** (reduce $E \rightarrow E + E$ OR shift $*$)

The conflict is why we need precedence rules or an unambiguous grammar.

Step-by-step:

1. Initialize empty stack, input ends with $\$$.
2. Consult action: shift, reduce, accept, or error.
3. On shift, push token; on reduce, pop [RHS] symbols and push LHS.

4. Loop until accept or error.

3. Important Subtopics

3.1 The stack and handle-on-top invariant

- **What:** Reductions always apply to a handle at the **top** of the stack.
- **Why it matters:** Guarantees the parser tracks a valid rightmost derivation in reverse.
- **Interview angle:** "Why must the handle be on top of the stack?"

3.2 Shift/reduce conflict

- **What:** A state where the parser could either shift the next token or reduce the stack top.
- **Why it matters:** Classic ambiguity symptom (e.g., dangling-else). Bison defaults to shift.
- **Example:** `E + E . * id` - reduce `E+E` or shift `*`?
- **Interview angle:** "Give an example of a shift/reduce conflict."

3.3 Reduce/reduce conflict

- **What:** Two different productions could reduce the same stack top.
- **Why it matters:** Usually a genuine grammar defect; harder to fix than shift/reduce.
- **Interview angle:** "What causes a reduce/reduce conflict?"

3.4 Operator-precedence parsing (a special shift-reduce)

- **What:** Uses precedence relations (`<`, `=`, `>`) between terminals to decide shift vs reduce for operator grammars.
- **Why it matters:** Simple, table-light method for expression grammars; historically important.
- **Interview angle:** "How does operator-precedence parsing decide reductions?"

4. Real-World Example

A Bison-generated C parser resolving dangling-else. The `if-then-else` grammar produces a shift/reduce conflict at the point where the parser has `if ( expr ) stmt` on the stack and sees `else`: it could reduce the `if-then` statement or shift `else`. Bison's default (and the C standard's intent) is to **shift**, binding `else` to the nearest `if`. This is a real, everyday shift-reduce decision baked into every C compiler's generated parser.

5. Diagrams / Mental Models

Action space:

	+-----+		
next token	SHIFT	REDUCE	
& stack top	+-----+		
	push tok	pop RHS, push LHS	
	+-----+		
	ACCEPT (S,\$)	ERROR	
	+-----+		

Conflicts:

Shift/Reduce: stack top is a handle AND next token could extend it
e.g. ... $E + E . *$ reduce $E+E$? or shift $*$?
Reduce/Reduce: stack top matches TWO productions' RHS
e.g. ... $a .$ where $A \rightarrow a$ and $B \rightarrow a$ both apply

6. Common Interview Questions

Q1. What are the four shift-reduce actions?

- Answer: Shift, reduce, accept, error.
- Common mistake: forgetting accept/error or confusing shift with reduce.

Q2. What is a handle in shift-reduce parsing?

- Answer: The stack-top substring (and production) whose reduction continues a valid rightmost derivation in reverse.

Q3. What is a shift/reduce conflict? Give an example.

- Answer: The parser can both shift and reduce; e.g., dangling-else, or $E + E . *$ in the ambiguous expression grammar.

Q4. What is a reduce/reduce conflict?

- Answer: Two productions could reduce the same handle; typically a grammar design flaw.

Q5. How does Bison resolve a shift/reduce conflict by default?

- Answer: It shifts (and warns).

Q6. Why must reductions occur at the top of the stack?

- Answer: Because shift-reduce reconstructs a rightmost derivation in reverse, whose handle is always a suffix of the current right-sentential form = stack top.

Q7. Trace a shift-reduce parse of $id + id$.

- Answer: shift id , reduce to E ; shift $+$, shift id , reduce to E , reduce $E+E$ to E , accept.

Q8. What data structures does a shift-reduce parser use?

- Answer: A stack (of states/symbols) and an input buffer with $\$$.

Q9. Is shift-reduce parsing linear time?

- Answer: Yes, for LR grammars with the precomputed table.

Q10. How is operator-precedence parsing related?

- Answer: It's a lightweight shift-reduce method using terminal precedence relations for operator grammars.

7. Deep-Dive Questions

D1. How does an LR parser know whether to shift or reduce without ambiguity?

- The LR automaton's current state encodes all viable prefixes seen; the ACTION table maps (state, lookahead) to exactly one of shift/reduce/accept when the grammar is LR - no guessing.

D2. Why are reduce/reduce conflicts usually worse than shift/reduce?

- Shift/reduce often reflect a benign preference (shift) matching intended semantics (dangling-else). Reduce/reduce usually mean two rules genuinely overlap, indicating a real grammar ambiguity that default resolution can silently mis-handle.

D3. Show the stack of *states* vs stack of *symbols*.

- Real LR parsers push states (integers). Symbols are implied by transitions. The state on top plus lookahead indexes ACTION/GOTO. Teaching examples often show symbols for clarity.

D4. How do precedence/associativity declarations resolve conflicts internally?

- They assign precedence to tokens and rules; at a shift/reduce conflict the parser compares the precedence of the lookahead token vs the rule and picks shift or reduce accordingly (higher precedence / associativity decides).

D5. Can every shift/reduce conflict be resolved by precedence?

- No - only those expressible via token precedence/associativity (mostly operator grammars). Structural conflicts need grammar changes.

8. Comparison Tables

Shift vs Reduce

Action	Stack effect	Trigger
Shift	Push lookahead token	Handle incomplete
Reduce	Pop RHS, push LHS	Handle complete on top

Shift/Reduce vs Reduce/Reduce Conflict

Aspect	Shift/Reduce	Reduce/Reduce
Nature	Shift or reduce both valid	Two reductions both valid
Common cause	Dangling-else, precedence	Overlapping rules / bad grammar
Default resolution (Bison)	Shift	Reduce by earlier rule
Severity	Often benign	Usually a real defect

9. Common Mistakes

- Reducing a substring that is not a handle / not on top of stack.
- Forgetting the accept and error actions.
- Treating Bison's default shift resolution as "the grammar is fine".
- Confusing shift/reduce with reduce/reduce conflicts.
- Popping the wrong number of symbols on reduce (must equal $|RHS|$).

10. Edge Cases / Special Cases

- **Dangling-else:** canonical shift/reduce, resolved by shifting.
- **epsilon-productions:** reduce with zero pops (empty handle).
- **Ambiguous grammars:** produce conflicts requiring precedence declarations.
- **Reduce/reduce from unit productions or overlapping rules.**
- **Precedence ties:** `%nonassoc` can turn a conflict into a syntax error (e.g., `a < b < c`).

11. How to Explain in Interview

"Shift-reduce parsing is the engine of bottom-up parsing. It keeps a stack and does one of four actions: shift a token onto the stack, reduce when the stack top matches a production's right-hand side (a handle) by replacing it with the left-hand side, accept when the start symbol is left, or error. The reduction sequence is a reverse rightmost derivation. The hard part is deciding shift vs reduce, and the LR automaton plus lookahead makes that deterministic for LR grammars. When the grammar is ambiguous you get shift/reduce or reduce/reduce conflicts - like the dangling-else, which Bison resolves by shifting."

12. Quick Revision Notes

- Four actions: **shift, reduce, accept, error**.
- Reduce replaces a **handle** (top of stack) with its LHS; pop $|RHS|$, push LHS.
- **Shift/reduce conflict:** both valid (dangling-else); Bison shifts.
- **Reduce/reduce conflict:** two rules match; usually real defect; Bison uses earlier rule.
- Reconstructs **rightmost derivation in reverse**; linear time with LR table.
- **Trap:** conflicts silently auto-resolved $\neq$ correct; handle must be on top.

13. Practice Tasks

1. Trace a shift-reduce parse of `id + id * id` with the unambiguous E/T/F grammar (no conflicts).
2. Repeat with the ambiguous `E -> E+E | E*E | id` and mark the shift/reduce conflict.
3. Construct a small grammar with a reduce/reduce conflict.
4. Show how `%left '+' '*'` precedence resolves the ambiguous expression conflict.
5. Trace the dangling-else conflict and resolve by shifting.
6. Implement a stack-based operator-precedence evaluator for `+ - * /`.

14. Final Cheat Sheet

- **Core definition:** Stack-based bottom-up parsing with shift/reduce/accept/error.
- **Why it matters:** Operational core of all LR parsers; defines conflicts.
- **Most asked:** Trace a parse; shift/reduce vs reduce/reduce; dangling-else.
- **Common comparison:** Shift vs reduce; the two conflict types.

- **One-line answer:** "Shift-reduce parsing shifts tokens onto a stack and reduces handles to non-terminals until only the start symbol remains, with shift/reduce and reduce/reduce conflicts arising from grammar ambiguity."

14. Handle and Viable Prefix

1. Overview

Definition:

- A **handle** of a right-sentential form is a substring that matches the RHS of a production **and** whose reduction to the LHS is a step in reversing the rightmost derivation. Formally, if $S \Rightarrow^* \alpha A w$, then $\alpha \beta w$, then β (at that position) is a handle for the production $A \rightarrow \beta$.
- A **viable prefix** is any prefix of a right-sentential form that does **not extend past the right end of a handle** - i.e., a string that can appear on an LR parser's stack during a valid parse.

Why it matters: Bottom-up parsing = repeatedly finding and reducing handles. The entire LR construction exists to recognize viable prefixes (and thus detect handles) using a DFA. These two concepts are the theoretical backbone of LR parsing.

Where it is used in real systems:

- LR automaton construction (the DFA of items recognizes viable prefixes).
- Explaining why LR parsers are correct and when conflicts arise.
- Parser-generator internals and grammar debugging.

Why interviewers ask about it: They are precise, often-confused definitions that test deep understanding; "what is a handle / viable prefix?" separates memorizers from understanders.

2. Core Idea

Intuition: During bottom-up parsing, the stack always holds a **viable prefix**. When that prefix ends exactly with a **handle**, it's time to reduce. The LR automaton is a machine whose states tell you "which handles could be completing right now."

Real-world analogy: Assembling a jigsaw. A **viable prefix** is a partially assembled region that's still consistent with the final picture. A **handle** is a cluster of pieces that clearly forms one recognizable object you can now treat as a single unit (reduce).

Small example: Grammar $E \rightarrow E + T \mid T, T \rightarrow T * F \mid F, F \rightarrow id$. Right-sentential form $E + T * id$:

- Rightmost derivation step producing it: $\dots \Rightarrow E + T * F \Rightarrow E + T * id$ ($F \rightarrow id$). So id is the handle for $F \rightarrow id$.
- **Viable prefixes** of $E + T * id$ include: $E, E +, E + T, E + T *, E + T * id$ up to the handle - but NOT something extending past the handle inconsistently.

Step-by-step (finding a handle):

1. Take the current right-sentential form.

2. Identify the last production applied in the rightmost derivation.
3. Its RHS, at that position, is the handle.
4. Reducing it gives the previous right-sentential form.

3. Important Subtopics

3.1 Handle vs "any matching RHS"

- **What:** Not every substring matching a RHS is a handle - only the one that keeps you on the rightmost derivation.
- **Why it matters:** Reducing a non-handle leads to a dead end.
- **Example:** In $E + E$ (ambiguous grammar) both E s match, but only the correct-position one is the handle.
- **Interview angle:** "Is every RHS-matching substring a handle?" (No.)

3.2 Handle pruning

- **What:** The process of repeatedly locating and reducing handles from the input back to S.
- **Why it matters:** This IS bottom-up parsing.
- **Interview angle:** "Describe handle pruning."

3.3 Viable prefix as stack content

- **What:** The set of viable prefixes is exactly the set of possible LR stack contents; it's a **regular** language, recognizable by a DFA.
- **Why it matters:** That regularity is why a finite automaton (LR(0) items) can drive parsing.
- **Interview angle:** "Why can a DFA recognize viable prefixes?"

3.4 LR(0) items and viable prefixes

- **What:** An LR(0) item $A \rightarrow \text{beta} \cdot \text{gamma}$ means "a viable prefix ending in beta has been seen; expecting gamma." Sets of items = DFA states.
- **Why it matters:** Directly links viable prefixes to the LR automaton.
- **Interview angle:** "What does an item's dot position represent?"

4. Real-World Example

Inside a Bison-generated parser's state machine. Each parser **state** corresponds to a set of LR(0)/LR(1) items, which collectively recognize a set of **viable prefixes**. As tokens shift, the parser walks this DFA; when it reaches a state whose item has the dot at the end ($A \rightarrow \text{beta} \cdot$), the stack top is a **handle** for $A \rightarrow \text{beta}$, so it reduces. Every "reduce by rule N" you see in a Bison debug trace is the parser having recognized a viable prefix that ends in a handle.

5. Diagrams / Mental Models

Handle in a rightmost derivation:

```
S =>rm alpha A w =>rm alpha beta w
               ^^^^ handle (reduce beta -> A to go backwards)
```

w = string of terminals to the right

Viable prefix boundary:

```
right-sentential form:  alpha  beta  w
                        |-----|      <- viable prefixes end
                        anywhere up to end of beta (the handle)
                        extending into w is NOT a viable prefix
```

Stack = viable prefix:

```
LR stack:  [ viable prefix ]      lookahead: next terminal
when the prefix ends in a handle -> REDUCE
otherwise                          -> SHIFT
```

6. Common Interview Questions

Q1. Define a handle.

- Answer: A substring beta of a right-sentential form matching $A \rightarrow \text{beta}$, whose reduction reverses one rightmost-derivation step (at the correct position).
- Common mistake: "any substring matching a production's RHS".

Q2. Define a viable prefix.

- Answer: A prefix of a right-sentential form that does not extend past the right end of a handle; exactly the possible LR stack contents.

Q3. Is every RHS-matching substring a handle?

- Answer: No - only the one on the current rightmost derivation.

Q4. What is handle pruning?

- Answer: Repeatedly finding and reducing handles from input back to the start symbol = bottom-up parsing.

Q5. Why is the set of viable prefixes regular?

- Answer: It can be recognized by a finite automaton over grammar symbols (the LR(0) item DFA).

Q6. What does an LR(0) item $A \rightarrow \text{beta} \cdot \text{gamma}$ mean?

- Answer: We've seen a viable prefix ending in beta and expect to see gamma next.

Q7. Where in the stack does a handle appear?

- Answer: At the top (its right end is the top of the stack).

Q8. How does the parser know a handle is present?

- Answer: The LR automaton reaches a state with a completed item (dot at end), possibly checking lookahead.

Q9. Relationship between handle and rightmost derivation?

- Answer: Reducing handles in sequence reconstructs the rightmost derivation in reverse.

Q10. Can a viable prefix contain a full handle plus more symbols to its right?

- Answer: No - by definition it stops at the right end of the handle.

7. Deep-Dive Questions

D1. Prove the set of viable prefixes of a grammar is a regular language.

- One can build an NFA whose states are LR(0) items with epsilon-closure transitions on $A \rightarrow \cdot \text{gamma}$; it accepts exactly the viable prefixes. Determinizing gives the canonical LR(0) DFA. A language recognized by a finite automaton is regular.

D2. Why must the handle be at the top of the stack in shift-reduce parsing?

- Because we reverse a *rightmost* derivation: at each backward step the replaced substring beta is immediately left of the already-generated terminal suffix w, which corresponds to the top of the stack (w has been consumed/reduced already).

D3. Two different handles in one sentential form - possible?

- In an unambiguous grammar there is a unique handle per right-sentential form. Multiple candidate handles at one step signal ambiguity / a conflict.

D4. How do LR(1) items refine handle detection over LR(0)?

- LR(1) items carry a lookahead terminal, so a completed item $A \rightarrow \text{beta} \cdot$, a triggers reduction only when the lookahead is a, avoiding spurious reductions that LR(0)/SLR would allow.

D5. Connection between viable prefixes and parser states.

- Each DFA state = the set of items valid for all viable prefixes reaching that state; the state summarizes "what handles could be forming," which is exactly the information needed to choose shift vs reduce.

8. Comparison Tables

Handle vs Viable Prefix

Aspect	Handle	Viable prefix
What it is	Substring to reduce next	Prefix that can be on the stack
Length	Exactly one production's RHS	Any prefix up to a handle's end
Role	Tells parser to reduce	Tells parser what's legal so far

Aspect	Handle	Viable prefix
Recognized by	Completed LR item (dot at end)	LR automaton reaching a state
Uniqueness	Unique (unambiguous grammar)	Many per sentential form

Handle vs arbitrary RHS match

	Handle	Arbitrary RHS match
On rightmost derivation	Yes	Not necessarily
Safe to reduce	Yes	May dead-end
Position	Correct spot at stack top	Anywhere

9. Common Mistakes

- Defining a handle as "any substring matching a production" (ignores position/derivation).
- Thinking a viable prefix can extend into the terminal suffix past the handle.
- Believing every right-sentential form has multiple handles (unambiguous => unique).
- Confusing handle (what to reduce) with viable prefix (what's on the stack).
- Forgetting handles are found at the top of the stack.

10. Edge Cases / Special Cases

- **epsilon-handles:** an $A \rightarrow \epsilon$ reduction has an empty handle (reduce with zero pops).
- **Ambiguous grammar:** multiple candidate handles => conflict.
- **Start/augmented production $S' \rightarrow S$:** its handle triggers accept.
- **Viable prefix = whole right-sentential form minus suffix:** boundary is the handle's right end.
- **Unit productions:** handles of length 1.

11. How to Explain in Interview

"A handle is the exact substring you should reduce next in a bottom-up parse - formally, if the rightmost derivation went $\alpha A w \Rightarrow \alpha \beta w$, then β is the handle for $A \rightarrow \beta$. Reducing handles in reverse order reconstructs the rightmost derivation. A viable prefix is any prefix of a right-sentential form that stops at or before the end of a handle - it's precisely what can legally sit on an LR parser's stack. The key insight is that the set of viable prefixes is regular, so a finite automaton built from LR items can recognize them and tell the parser exactly when the stack top has become a handle and it's time to reduce."

12. Quick Revision Notes

- **Handle:** substring matching $A \rightarrow \beta$ at the position that reverses a rightmost step.
- Not every RHS match is a handle; unambiguous grammar => **unique** handle.
- **Handle pruning** = repeatedly reduce handles = bottom-up parsing.
- **Viable prefix:** legal LR stack content; prefix up to (not past) a handle's right end.
- Set of viable prefixes is **regular** => recognized by the LR(0) item DFA.
- Handles appear at the **top of the stack**; completed item (dot at end) signals one.
- **Trap:** handle = "any matching RHS" is wrong.

13. Practice Tasks

1. For $E \rightarrow E + T \mid T$; $T \rightarrow T * F \mid F$; $F \rightarrow id$, list the handle at each step of parsing $id + id * id$.
2. For the right-sentential form $E + T * id$, identify the handle and the viable prefixes.
3. Show an epsilon-handle reduction in a grammar with $A \rightarrow \epsilon$.
4. Build the LR(0) item DFA for a tiny grammar and point out which states signal handles.
5. In the ambiguous grammar $E \rightarrow E+E \mid id$, show two candidate handles and the resulting conflict.
6. Explain why extending a viable prefix into the terminal suffix is illegal.

14. Final Cheat Sheet

- **Core definition:** Handle = substring to reduce next (reverses a rightmost step); viable prefix = legal LR stack content.
- **Why it matters:** Foundation of LR parsing correctness and automaton construction.
- **Most asked:** Define handle & viable prefix; is every RHS-match a handle (no).
- **Common comparison:** Handle vs viable prefix; handle vs arbitrary RHS match.
- **One-line answer:** "A handle is the exact substring whose reduction reverses a rightmost-derivation step, and a viable prefix is any stack-legal prefix ending at or before a handle - the LR item DFA recognizes viable prefixes to find handles."

15. LR Parsing Basics

1. Overview

Definition: LR parsing is a family of efficient, table-driven **bottom-up** parsing methods.

- **L** = scan input **L**eft to **r**ight,
- **R** = construct a **R**ightmost derivation in reverse,
- optional **(k)** = number of lookahead tokens (usually 1).

An LR parser uses a **stack of states**, an **ACTION table** (shift/reduce/accept/error keyed by state + terminal), and a **GOTO table** (state transitions on non-terminals). The four sub-types are **LR(0)**, **SLR(1)**, **LALR(1)**, **CLR/LR(1)**.

Why it matters: LR(1) recognizes **all deterministic context-free languages** - the most powerful practical deterministic parsing. It underlies Yacc/Bison and most serious parser generators.

Where it is used in real systems:

- Yacc/Bison (LALR), many compiler front-ends, SQL parsers.
- Language tooling that compiles a grammar into fast tables.

Why interviewers ask about it: LR parsing is the deep end: items, closure/goto, the automaton, ACTION/GOTO tables, and the SLR/CLR/LALR hierarchy. Strong signal for compiler roles.

2. Core Idea

Intuition: Build a DFA whose states are sets of **items** (productions with a dot marking progress). The stack records which state you're in. On each token, the ACTION table says shift (push a state), reduce (pop the RHS, then GOTO on the LHS), accept, or error. The DFA recognizes viable prefixes; completed items trigger reductions.

Real-world analogy: A subway map (the DFA). Your current station is the top-of-stack state. The next token is which train you board (ACTION). Some stations say "you've completed a route segment - collapse it" (reduce) and then reroute you via GOTO.

Items and construction:

- **LR(0) item:** a production with a dot, e.g. $E \rightarrow E \cdot + T$.
- **Augment** the grammar with $S' \rightarrow S$.
- **CLOSURE(I):** if $A \rightarrow \alpha \cdot B \beta$ is in I, add $B \rightarrow \cdot \gamma$ for all B-productions.
- **GOTO(I, X):** move the dot past X in all items, then closure.
- Collect all item sets = DFA states; that's the **canonical collection**.

Small example (fragment): Augmented $S' \rightarrow S, S \rightarrow C C, C \rightarrow c C \mid d$.

```
I0: S' -> .S, S -> .CC, C -> .cC, C -> .d
GOTO(I0, C) -> items with S -> C.C, C -> .cC, C -> .d
... (states I1..I6) form the DFA used to fill ACTION/GOTO.
```

3. Important Subtopics

3.1 ACTION and GOTO tables

- **What:** ACTION[state, terminal] in {shift s, reduce r, accept, error}; GOTO[state, non-terminal] = next state.
- **Why it matters:** These two tables ARE the parser.
- **Interview angle:** "What's the difference between ACTION and GOTO?"

3.2 The LR parsing algorithm (driver)

- **What:** Stack of states; on shift push state; on reduce $A \rightarrow \beta$, pop $2*|\beta|$ entries (state+symbol) then push GOTO[top, A].
- **Why it matters:** Same driver for SLR/LALR/CLR; only the table differs.
- **Interview angle:** "Describe the LR driver loop."

3.3 Items, closure, goto (automaton construction)

- **What:** Sets of items become states; closure adds predicted productions; goto defines transitions.
- **Why it matters:** How the DFA that recognizes viable prefixes is built.
- **Interview angle:** "Compute CLOSURE and GOTO for this item set."

3.4 The LR hierarchy: $LR(0) \subset SLR \subset LALR \subset CLR(=LR(1))$

- **What:** They share the same item automaton shape (except CLR uses more states) but differ in how reductions get lookahead.

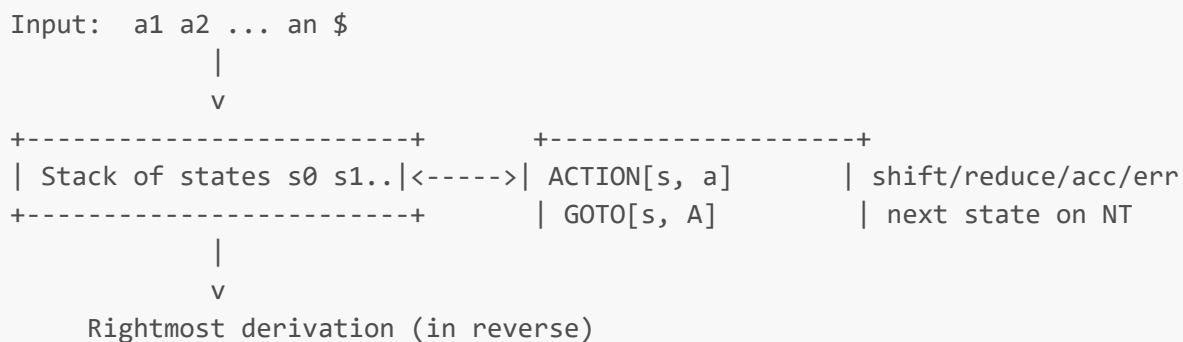
- **Why it matters:** Explains power vs table-size trade-offs.
- **Interview angle:** "Order these by power and table size."

4. Real-World Example

Bison building a C/SQL parser. You write grammar rules; Bison computes the LR item automaton, builds ACTION/GOTO tables (LALR by default), and emits a C driver that runs the stack-of-states loop at parse time. Reported "shift/reduce" and "reduce/reduce" conflicts are cells where the table would need two actions. The generated parser is $O(n)$ and handles left-recursive, complex grammars that no LL(1) parser could - which is why production compilers rely on it.

5. Diagrams / Mental Models

LR parser architecture:



Item dot meaning:

```

A -> alpha . beta      "seen alpha (on stack), expect beta"
A -> alpha .           "handle complete -> reduce by A->alpha"
  
```

Hierarchy:

```

LR(0) < SLR(1) < LALR(1) < CLR/LR(1)      (increasing power)
smallest tables -----> LALR≈SLR size, CLR largest
  
```

6. Common Interview Questions

Q1. What does LR stand for?

- Answer: Left-to-right scan, Rightmost derivation in reverse; (k) lookahead.
- Common mistake: "Right-to-left".

Q2. What are the ACTION and GOTO tables?

- Answer: ACTION maps (state, terminal) to shift/reduce/accept/error; GOTO maps (state, non-terminal) to a state.

Q3. What is an LR(0) item?

- Answer: A production with a dot marking how much of the RHS has been seen.

Q4. What is CLOSURE and GOTO?

- Answer: CLOSURE adds predicted productions for a non-terminal after the dot; GOTO moves the dot past a symbol and takes closure.

Q5. Describe the LR parsing algorithm.

- Answer: Use stack of states; shift pushes a state; reduce pops $2|RHS|$ and pushes GOTO[top, LHS]; accept on $S' \rightarrow S \dots$.

Q6. Why is LR more powerful than LL?

- Answer: It decides reductions after seeing the whole RHS with lookahead; LR(1) captures all deterministic CFLs.

Q7. Name the LR variants in order of power.

- Answer: $LR(0) < SLR(1) < LALR(1) < CLR/LR(1)$.

Q8. What triggers a reduce action?

- Answer: Being in a state with a completed item (dot at end), subject to lookahead rules of the variant.

Q9. Do LR parsers need left-recursion removal?

- Answer: No.

Q10. What is the augmented grammar and why add it?

- Answer: Add $S' \rightarrow S$ so acceptance is a single clean reduction ($S' \rightarrow S \dots$ on $\$$).

7. Deep-Dive Questions

D1. Why push states, not symbols, on the stack?

- The state encodes the entire viable prefix's relevant history; symbols are redundant given states and GOTO. States make table lookup $O(1)$.

D2. How many entries are popped on a reduce and why?

- $2|RHS|$ if you store (state, symbol) pairs (or $|RHS|$ states); because each RHS symbol added one state during shifts/gotos, so a reduction unwinds exactly that many.

D3. What makes a grammar LR(0)?

- Every state either has a single completed item (reduce) with no shift, or only shift items - no state mixes a shift and a reduce or two reduces. Very restrictive.

D4. Where does lookahead enter for SLR/LALR/CLR?

- SLR uses FOLLOW(A) for reduce columns; CLR computes exact per-item lookaheads (LR(1) items); LALR merges CLR states with identical cores, keeping merged lookaheads.

D5. Why is LR(1) sufficient for all deterministic CFLs?

- A theorem: any deterministic CFL has an LR(1) grammar. One lookahead plus the viable-prefix automaton captures exactly deterministic pushdown recognition.

8. Comparison Tables

LR variants

Variant	Lookahead source	States	Power	Notes
LR(0)	none	fewest	weakest	reduces regardless of lookahead
SLR(1)	FOLLOW(A)	= LR(0) count	moderate	simplest useful
LALR(1)	merged LR(1) lookaheads	= LR(0) count	strong	Yacc/Bison default
CLR/LR(1)	exact per-item	most	strongest	large tables

LR vs LL

Aspect	LR	LL
Direction	Bottom-up	Top-down
Derivation	Rightmost (reverse)	Leftmost
Power	Larger	Smaller
Left recursion	OK	Must remove
Tables	ACTION+GOTO	Single M[A,a]

9. Common Mistakes

- Thinking R means "right-to-left" scanning (it's rightmost derivation).
- Mixing up ACTION (terminals) and GOTO (non-terminals).
- Forgetting to augment the grammar with $S' \rightarrow S$.
- Popping the wrong count on reduce.
- Assuming all LR variants have the same power/size.

10. Edge Cases / Special Cases

- **Conflicts:** shift/reduce or reduce/reduce in a table cell.
- **epsilon-productions:** reduce with zero pops in the right state.
- **Augmented start:** acceptance via $S' \rightarrow S \dots$
- **LR(0) inadequacy:** most real grammars aren't LR(0); need lookahead.
- **State explosion in CLR:** motivates LALR merging.

11. How to Explain in Interview

"LR parsing is bottom-up, table-driven parsing that scans left to right and builds a rightmost derivation in reverse. The core is a DFA whose states are sets of items - productions with a dot showing progress - constructed via closure and goto over an augmented grammar. At parse time a stack of states plus an ACTION table (shift/reduce/accept/error on terminals) and a GOTO table (transitions on non-terminals) drive the parse in linear time. The variants LR(0), SLR, LALR, and CLR all share this machinery and differ only in how much lookahead the reduce actions use - trading table size for power, with LR(1)/CLR being strong enough for every deterministic context-free language."

12. Quick Revision Notes

- LR = Left-to-right, Rightmost derivation reversed, k lookahead.
- **ACTION** (state x terminal): shift/reduce/accept/error. **GOTO** (state x non-terminal): next state.
- States = sets of **items**; built by **closure** + **goto**; augment with $S' \rightarrow S$.
- Driver: stack of states; reduce pops 2[RHS], pushes GOTO[top, LHS].
- Hierarchy by power: **LR(0) < SLR < LALR < CLR/LR(1)**.
- LR(1) covers **all deterministic CFLs**; no left-recursion removal needed.
- **Trap**: R != right-to-left; ACTION vs GOTO mix-up.

13. Practice Tasks

1. Augment $S \rightarrow CC$; $C \rightarrow cC \mid d$ and build the LR(0) item sets (canonical collection).
2. Draw the GOTO DFA for that grammar.
3. Fill ACTION/GOTO for it as an SLR parser and parse cdd .
4. Show the reduce action popping the correct number of states.
5. Identify a grammar that is not LR(0) but is SLR(1).
6. Explain why augmenting the grammar simplifies the accept action.

14. Final Cheat Sheet

- **Core definition**: Bottom-up, table-driven parser; ACTION+GOTO over an item DFA; rightmost derivation reversed.
- **Why it matters**: Most powerful practical deterministic parsing (LR(1) = all det. CFLs).
- **Most asked**: ACTION vs GOTO; items/closure/goto; variant hierarchy.
- **Common comparison**: LR variants; LR vs LL.
- **One-line answer**: "LR parsing drives a stack of states with ACTION/GOTO tables built from an item DFA to reduce handles bottom-up - the strongest deterministic method, with SLR/LALR/CLR differing only in reduce-lookahead precision."

16. SLR Parser (Simple LR)

1. Overview

Definition: SLR(1) (Simple LR) is the simplest useful LR parser. It builds the **LR(0) item automaton** and resolves reduce actions using **FOLLOW sets**: place a reduce $A \rightarrow \alpha$ in ACTION[state, a] for every terminal a in **FOLLOW(A)** (for states containing the completed item $A \rightarrow \alpha$.).

Why it matters: SLR gives you a working bottom-up parser with the small LR(0) automaton, no per-item lookahead computation. It's the first LR variant taught and a common exam target. Its limitation (FOLLOW is too coarse) motivates LALR/CLR.

Where it is used in real systems:

- Teaching and simpler grammars; some lightweight generators.
- A stepping stone to LALR (which most real tools use).

Why interviewers ask about it: Constructing an SLR table (items + FOLLOW) and detecting SLR conflicts is a classic, self-contained problem that tests items, closure/goto, and FOLLOW together.

2. Core Idea

Intuition: Use the compact LR(0) states. LR(0) alone would reduce on *every* lookahead (causing conflicts), so SLR restricts each reduction $A \rightarrow \alpha$ to only the lookaheads in FOLLOW(A). If that's enough to make every cell single-valued, the grammar is SLR(1).

Real-world analogy: A bouncer (state) with a guest list. LR(0) lets anyone in for a reduction; SLR checks the guest's name against FOLLOW(A) - only names that could legitimately follow A get the reduce action.

Construction steps:

1. Augment grammar: $S' \rightarrow S$.
2. Build the canonical collection of **LR(0) items** (closure + goto) $\Rightarrow$ DFA states.
3. For transitions on terminals $\Rightarrow$ **shift**; on non-terminals $\Rightarrow$ **GOTO**.
4. For a state with completed item $A \rightarrow \alpha .$ ($A \neq S'$), set ACTION[state, a] = reduce $A \rightarrow \alpha$ for all a in FOLLOW(A).
5. For $S' \rightarrow S .$, set ACTION[state, \$] = **accept**.
6. If any cell gets two actions $\Rightarrow$ **SLR conflict** (grammar not SLR(1)).

Small example: $S \rightarrow C C, C \rightarrow c C \mid d$. Build LR(0) states; reductions $C \rightarrow d$ and $C \rightarrow c C$ use FOLLOW(C) = {c, d, }, and $S \rightarrow CC$ uses FOLLOW(S) = {}. This grammar is SLR(1) (no conflicts).

3. Important Subtopics

3.1 LR(0) automaton reused

- **What:** SLR uses the exact same states as LR(0)/LALR - no lookahead in the items themselves.
- **Why it matters:** Small table; lookahead only applied at reduce time via FOLLOW.
- **Interview angle:** "Do SLR items carry lookahead?" (No - only LR(1)/CLR items do.)

3.2 FOLLOW-based reduction rule

- **What:** Reduce $A \rightarrow \alpha$ only on terminals in FOLLOW(A).
- **Why it matters:** The defining feature of SLR; also its weakness.
- **Interview angle:** "How does SLR decide when to reduce?"

3.3 SLR conflicts (why FOLLOW is too coarse)

- **What:** FOLLOW(A) may include terminals that can't actually follow A *in this particular state*, causing spurious shift/reduce or reduce/reduce conflicts.
- **Why it matters:** Motivates LALR/CLR, which use context-specific lookaheads.
- **Example:** Classic grammar $S \rightarrow L = R \mid R; L \rightarrow * R \mid id; R \rightarrow L$ is not SLR (shift/reduce on $=$) but is LALR.
- **Interview angle:** "Give a grammar that's LALR but not SLR."

3.4 SLR vs LR(0)

- **What:** LR(0) reduces regardless of lookahead; SLR adds FOLLOW filtering.
- **Why it matters:** SLR fixes many (not all) LR(0) conflicts.
- **Interview angle:** "What does SLR add over LR(0)?"

4. Real-World Example

A teaching / small-DSL parser generator. For a modest, well-structured grammar (arithmetic expressions, simple statement lists), an SLR generator produces correct linear-time tables using just the LR(0) states plus FOLLOW sets - minimal machinery. Production tools graduate to LALR because real language grammars (like the $L = R$ pointer-assignment example) expose SLR's FOLLOW imprecision, but conceptually SLR is where LR table-building "clicks."

5. Diagrams / Mental Models

SLR reduce rule:

```
State contains  A -> alpha .      (completed item)
               for each terminal a in FOLLOW(A):  ACTION[state, a] = reduce A->alpha
```

Where SLR sits:

```
LR(0) items + FOLLOW(A) at reduce = SLR(1)
    |                               |
    |                               |
    | reduces on ANY token          | reduces only on FOLLOW(A)
```

SLR conflict shape:

```
State: { X -> u . a v (shift on a),  A -> w . (reduce, a in FOLLOW(A)) }
        -> ACTION[state, a] wants BOTH shift and reduce => conflict
```

6. Common Interview Questions

Q1. What is an SLR parser?

- Answer: An LR parser using LR(0) states with reduce actions restricted to FOLLOW(A).
- Common mistake: saying SLR items carry lookahead.

Q2. How is the SLR table's reduce action determined?

- Answer: Reduce $A \rightarrow \alpha$ on every terminal in FOLLOW(A) for states with the completed item.

Q3. Does SLR use LR(0) or LR(1) items?

- Answer: LR(0) items; lookahead comes only from FOLLOW at reduce time.

Q4. Why can SLR have conflicts that LALR resolves?

- Answer: FOLLOW(A) is grammar-global and too coarse; it may include terminals invalid in a specific state. LALR uses state-specific lookaheads.

Q5. Give a grammar that's LALR but not SLR.

- Answer: $S \rightarrow L = R \mid R; L \rightarrow * R \mid id; R \rightarrow L$ (shift/reduce conflict on =).

Q6. What is the accept condition in SLR?

- Answer: ACTION[state, \$] = accept when the state has $S' \rightarrow S \dots$

Q7. Steps to build an SLR table?

- Answer: Augment, build LR(0) canonical collection, fill shift/GOTO from transitions, fill reduces via FOLLOW, mark accept, check conflicts.

Q8. Is every LR(0) grammar SLR(1)?

- Answer: Yes - SLR is at least as powerful as LR(0).

Q9. Is every SLR(1) grammar LALR(1)?

- Answer: Yes - $SLR \subseteq LALR \subseteq CLR$ in power.

Q10. What size is the SLR automaton compared to CLR?

- Answer: Same as LR(0)/LALR (fewer states than CLR).

7. Deep-Dive Questions

D1. Precisely why does FOLLOW cause the $L = R$ conflict?

- After parsing $\dots R$ where a state has both $S \rightarrow L \cdot = R$ (shift =) and $R \rightarrow L \cdot$ (reduce, and = is in FOLLOW(R) because R can appear where = follows), SLR sees = as a valid reduce lookahead even though in this context reducing is wrong. LALR's lookahead excludes = here.

D2. Can SLR ever have reduce/reduce conflicts?

- Yes, if two completed items $A \rightarrow \alpha \cdot$ and $B \rightarrow \beta \cdot$ share a state and $FOLLOW(A) \cap FOLLOW(B)$ is non-empty.

D3. Is SLR strictly weaker than LALR in practice?

- Yes - there exist grammars SLR rejects but LALR accepts; the reverse never happens. Same number of states, but LALR's precise lookaheads resolve more.

D4. How would you convert an SLR construction into LALR?

- Compute LR(1) items, then merge states with identical LR(0) cores, unioning lookaheads - or propagate lookaheads over the LR(0) automaton. Reduces are then keyed on those merged lookaheads, not global FOLLOW.

D5. Why teach SLR if LALR is used in practice?

- SLR isolates the core idea (LR(0) states + a lookahead filter) with the least machinery, making the FOLLOW-imprecision problem - and thus the motivation for LALR/CLR - crystal clear.

8. Comparison Tables

SLR vs LR(0) vs LALR vs CLR

Feature	LR(0)	SLR(1)	LALR(1)	CLR/LR(1)
Items	LR(0)	LR(0)	LR(1) merged	LR(1)
Reduce lookahead	none	FOLLOW(A)	state-specific (merged)	state-specific (exact)
States	N	N	N	> N
Power	weakest	moderate	strong	strongest

SLR reduce vs CLR reduce

	SLR	CLR
Lookahead scope	Global FOLLOW(A)	Exact context per item
Spurious reduces	Possible (conflicts)	Avoided
Table size	Small	Large

9. Common Mistakes

- Thinking SLR items include lookahead symbols (they don't).
- Using FIRST instead of FOLLOW for reduce columns.
- Assuming SLR handles all LR(1) grammars (it doesn't).
- Forgetting the accept cell for $S' \rightarrow S \cdot$ on \$.
- Mislabeling an SLR conflict as "the grammar is ambiguous" (it may just be non-SLR but LALR).

10. Edge Cases / Special Cases

- **L = R grammar:** SLR shift/reduce conflict; LALR is fine.
- **Reduce/reduce via overlapping FOLLOW sets.**
- **epsilon-productions:** reduce $A \rightarrow \epsilon$ on FOLLOW(A) in the closure state.
- **Every LR(0) grammar is SLR;** not every SLR is LR(0).
- **State count identical to LALR** despite weaker power.

11. How to Explain in Interview

"SLR is the simplest practical LR parser. I build the LR(0) item automaton with closure and goto, then fill the table: shifts and gotos come from the DFA transitions, and for any state with a completed item $A \rightarrow \text{alpha} \cdot$, I add a reduce action under every terminal in FOLLOW(A). Accept is the augmented $S' \rightarrow S \cdot$ on \$. It's compact because it reuses the small LR(0) states, but FOLLOW is a global, context-insensitive lookahead, so it sometimes puts a reduce where it shouldn't - the classic $S \rightarrow L = R$ pointer grammar isn't SLR but is LALR. That imprecision is exactly why LALR and CLR use state-specific lookaheads."

12. Quick Revision Notes

- SLR = **LR(0) items** + reduce on **FOLLOW(A)**.
- Same states as LR(0)/LALR (small table).
- Accept: $S' \rightarrow S \cdot$ on \$.
- Weaker than LALR/CLR because **FOLLOW is too coarse**.
- Classic non-SLR-but-LALR grammar: $S \rightarrow L = R \mid R; L \rightarrow *R \mid \text{id}; R \rightarrow L$.
- Power: **LR(0) $\subseteq$ SLR $\subseteq$ LALR $\subseteq$ CLR**.
- **Trap:** SLR items don't carry lookahead; use FOLLOW not FIRST for reduces.

13. Practice Tasks

1. Build the SLR table for $S \rightarrow CC; C \rightarrow cC \mid d$ and parse cdd .
2. Build the SLR table for the E/T/F grammar and parse $\text{id} + \text{id} * \text{id}$.
3. Show the SLR shift/reduce conflict in $S \rightarrow L = R \mid R; L \rightarrow *R \mid \text{id}; R \rightarrow L$.
4. Find a reduce/reduce conflict caused by overlapping FOLLOW sets.
5. For one grammar, list which reduce cells come from which FOLLOW set.
6. Convert your SLR construction to LALR and note which conflict disappears.

14. Final Cheat Sheet

- **Core definition:** LR(0) automaton with reduces restricted to FOLLOW(A).
- **Why it matters:** Simplest working LR parser; shows the lookahead idea.
- **Most asked:** Build the SLR table; give an LALR-not-SLR grammar; why FOLLOW is coarse.
- **Common comparison:** SLR vs LALR vs CLR; SLR vs LR(0).
- **One-line answer:** "SLR uses the compact LR(0) states and reduces $A \rightarrow \text{alpha}$ only on FOLLOW(A); simple but sometimes conflict-prone because FOLLOW ignores state context."

17. CLR Parser (Canonical LR)

1. Overview

Definition: CLR(1) - also called **Canonical LR(1)** or just **LR(1)** - is the most powerful LR parser. It builds states from **LR(1) items**: LR(0) items augmented with a **lookahead terminal**, written $[A \rightarrow \text{alpha} \cdot \text{beta}, a]$. A reduction $A \rightarrow \text{alpha}$ is applied **only when the current lookahead equals the item's lookahead a** , giving the most precise reduce decisions.

Why it matters: CLR recognizes **every LR(1) grammar** - i.e., every deterministic context-free language. It never has the spurious conflicts SLR suffers from. Its downside is **many more states** (large tables), which is why LALR (a compression of CLR) is used in practice.

Where it is used in real systems:

- Reference/most-powerful table construction; some tools offer full LR(1) mode.
- Theoretical benchmark for "can this grammar be LR-parsed at all?"
- Modern IELR/canonical-LR options in advanced generators.

Why interviewers ask about it: It's the top of the LR hierarchy; understanding LR(1) items, per-item lookahead, and why CLR has more states than SLR/LALR demonstrates mastery.

2. Core Idea

Intuition: Attach to each item the exact set of terminals that may legitimately follow when that item completes - computed per state, not globally. This context-sensitivity removes the false reduces FOLLOW caused in SLR, at the cost of splitting states that differ only in lookahead.

Real-world analogy: A guest list per-room (per-state), not one master list for the whole building. SLR used one global list (FOLLOW); CLR keeps a tailored list for each room, so nobody gets into the wrong room.

LR(1) item and construction:

- **LR(1) item:** $[A \rightarrow \alpha \cdot \beta, a]$ where a is a lookahead terminal.
- **CLOSURE:** for $[A \rightarrow \alpha \cdot B \beta, a]$, add $[B \rightarrow \cdot \gamma, b]$ for every b in $\text{FIRST}(\beta a)$.
- **GOTO:** move the dot past a symbol (carry lookaheads), then closure.
- Build canonical collection of LR(1) item sets $\Rightarrow$ states.
- **Reduce:** in a state with $[A \rightarrow \alpha \cdot, a]$, set $\text{ACTION}[\text{state}, a] = \text{reduce } A \rightarrow \alpha$ (only for that a).

Small example: For $S \rightarrow C C, C \rightarrow c C \mid d$, the LR(1) construction distinguishes contexts where C is followed by c/d (inside the first C) vs $\$$ (the second C), producing separate states - which is exactly what LALR later re-merges.

3. Important Subtopics

3.1 LR(1) items with lookahead

- **What:** Each item carries a specific terminal lookahead computed via $\text{FIRST}(\beta a)$.
- **Why it matters:** Precise reductions $\Rightarrow$ no SLR-style false conflicts.
- **Interview angle:** "How is an LR(1) item different from an LR(0) item?"

3.2 CLOSURE with $\text{FIRST}(\beta a)$

- **What:** The lookahead of added items is FIRST of "rest of the production after B , followed by the parent's lookahead".
- **Why it matters:** The exact recipe for building CLR states; a common exam step.
- **Interview angle:** "Compute the closure of this LR(1) item set."

3.3 State explosion

- **What:** States that share a core (same LR(0) items) but differ in lookaheads stay separate in CLR, multiplying states.
- **Why it matters:** Motivates LALR (merge same-core states).

- **Interview angle:** "Why does CLR have more states than LALR?"

3.4 Power: exactly LR(1)

- **What:** CLR accepts all and only LR(1) grammars.
- **Why it matters:** The gold standard for deterministic parsing power.
- **Interview angle:** "Is CLR more powerful than LALR?" (Yes, strictly, on some grammars.)

4. Real-World Example

Advanced parser generators (e.g., Bison's `%define lr.type canonical-lr`, or IELR). When a grammar has subtle context-dependent reductions that LALR's state-merging mishandles (rare "mysterious conflicts" introduced by merging), engineers switch to canonical LR(1) to get correct, conflict-free tables - accepting larger tables in exchange. This is exactly the CLR-vs-LALR trade-off appearing in a real tool.

5. Diagrams / Mental Models

LR(1) item:

```
[ A -> alpha . beta , a ]
                        ^ lookahead: reduce only when next token = a (when dot at
end)
```

Closure lookahead computation:

```
[A -> alpha . B beta, a]
  add [B -> . gamma, b]  for each b in FIRST(beta a)
```

Why more states than LALR:

```
CLR:   state P = {core X, lookahead {a}}, state Q = {core X, lookahead {b}}
(kept separate)
LALR:  merge P,Q -> {core X, lookahead {a,b}}
(fewer states)
```

6. Common Interview Questions

Q1. What is a CLR (canonical LR) parser?

- Answer: The full LR(1) parser using LR(1) items (with per-item lookahead) for the most precise reduce decisions.
- Common mistake: confusing it with LALR.

Q2. What is an LR(1) item?

- Answer: `[A -> alpha . beta, a]` - an LR(0) item plus a lookahead terminal `a`.

Q3. How is the lookahead computed in closure?

- Answer: For $[A \rightarrow \alpha \cdot B \beta, a]$, added items $[B \rightarrow \cdot \gamma, b]$ take b in $\text{FIRST}(\beta a)$.

Q4. Why does CLR avoid SLR conflicts?

- Answer: It uses context-specific lookaheads instead of global FOLLOW, so reduces only fire on genuinely valid lookaheads.

Q5. What is the main drawback of CLR?

- Answer: Many more states $\Rightarrow$ large tables and memory.

Q6. CLR vs LALR - power and size?

- Answer: CLR is strictly more powerful (some grammars) but has many more states; LALR merges same-core states.

Q7. Is every LR(1) grammar handled by CLR?

- Answer: Yes, by definition.

Q8. When do you reduce in a CLR state?

- Answer: When the state has $[A \rightarrow \alpha \cdot, a]$ and the lookahead is exactly a .

Q9. Does CLR need left-recursion removal?

- Answer: No (it's bottom-up).

Q10. Why isn't CLR the default in Yacc/Bison?

- Answer: Table size; LALR gives nearly the same power with far fewer states.

7. Deep-Dive Questions

D1. Show how $\text{FIRST}(\beta a)$ with nullable β pulls in a .

- If β is nullable, $\text{FIRST}(\beta a)$ includes $\text{FIRST}(\beta)\{\epsilon\}$ plus a (the inherited lookahead), so the parent's lookahead propagates to the child item - crucial for correct reductions near ϵ .

D2. Give a grammar that is LR(1)/CLR but not LALR.

- Grammars where merging two same-core LR(1) states creates a reduce/reduce conflict absent in CLR. The standard textbook example (Aho-Sethi-Ullman) constructs such a grammar; the merged lookahead sets overlap only after union.

D3. Why can merging never create a shift/reduce conflict, only reduce/reduce?

- Merging unions lookaheads on completed items but doesn't change the core's shift transitions; shifts are determined by the core alone. Thus only reduce/reduce (two completed items) can newly clash.

D4. Roughly how many more states can CLR have than LALR?

- Up to a large multiplicative factor for real languages (e.g., LALR ~ hundreds of states vs CLR ~ thousands), because same-core states proliferate by lookahead differences.

D5. How would you derive LALR directly from the CLR construction?

- Build the full CLR collection, then merge all states with identical cores, unioning their item lookaheads. The resulting automaton is LALR(1).

8. Comparison Tables

CLR vs SLR vs LALR

Feature	SLR(1)	LALR(1)	CLR/LR(1)
Items	LR(0)	LR(1) merged	LR(1) full
Lookahead	FOLLOW(A)	merged exact	exact per state
States	N	N	many (> N)
Power	least of the three	middle	most
Table size	small	small	large
Conflict type risk	most	can gain reduce/reduce from merge	fewest

LR(0) item vs LR(1) item

	LR(0) item	LR(1) item
Form	$A \rightarrow \alpha \cdot \beta$	$[A \rightarrow \alpha \cdot \beta, a]$
Lookahead	none	one terminal
Used by	LR(0), SLR	CLR, LALR

9. Common Mistakes

- Confusing CLR (full LR(1)) with LALR (merged LR(1)).
- Computing item lookaheads with FOLLOW instead of FIRST(beta a).
- Forgetting to propagate the inherited lookahead when beta is nullable.
- Thinking CLR and LALR always have the same number of states.
- Believing CLR is used by default in practice (it's usually LALR).

10. Edge Cases / Special Cases

- **Nullable beta:** lookahead inheritance via FIRST(beta a).
- **Same-core states:** kept separate in CLR (the source of extra states).
- **LR(1)-only grammars:** parseable by CLR, may fail LALR after merging.
- **epsilon-productions with specific lookaheads:** precise reduces avoid conflicts.
- **Very large grammars:** CLR tables can be impractically big.

11. How to Explain in Interview

"Canonical LR - CLR - is the full LR(1) parser. Its states are sets of LR(1) items, each an LR(0) item plus a specific lookahead terminal, computed in closure via $\text{FIRST}(\beta a)$. Because the lookahead is context-specific rather than the global FOLLOW that SLR uses, CLR never fires a reduce on a token that can't actually follow in that context, so it handles every LR(1) grammar - the full class of deterministic CFLs. The catch is state explosion: states that share a core but differ in lookahead stay separate, giving huge tables. That's why practical tools use LALR, which merges those same-core states, keeping almost all of CLR's power at SLR's table size."

12. Quick Revision Notes

- CLR = full **LR(1)**: items are $[A \rightarrow \alpha \cdot \beta, a]$ with per-item lookahead.
- Closure lookahead = **FIRST(βa)**.
- Reduce only when lookahead = the item's terminal (precise).
- **Most powerful** LR variant; handles all deterministic CFLs.
- Drawback: **state explosion** / large tables.
- LALR = CLR with **same-core states merged**.
- **Trap**: CLR $\neq$ LALR; use $\text{FIRST}(\beta a)$, not FOLLOW.

13. Practice Tasks

1. Build the CLR(1) item sets for $S \rightarrow CC; C \rightarrow cC \mid d$ and note the state count.
2. Compare that state count to the SLR/LALR automaton for the same grammar.
3. Compute CLOSURE of $[S' \rightarrow \cdot S, \$]$ fully for a small grammar.
4. Show lookahead inheritance where β is nullable.
5. Merge same-core CLR states to obtain the LALR automaton.
6. Research/derive a grammar that is LR(1) but not LALR(1) and explain the merge conflict.

14. Final Cheat Sheet

- **Core definition**: Full LR(1) parser using items with per-item lookahead ($\text{FIRST}(\beta a)$).
- **Why it matters**: Most powerful deterministic parser; no SLR-style false conflicts.
- **Most asked**: LR(1) item; closure lookahead rule; CLR vs LALR trade-off.
- **Common comparison**: CLR vs SLR vs LALR; LR(0) vs LR(1) item.
- **One-line answer**: "CLR is canonical LR(1): states of LR(1) items with exact per-context lookaheads make it the most powerful LR parser, at the cost of state explosion that LALR later compresses."

18. LALR Parser (Look-Ahead LR)

1. Overview

Definition: LALR(1) (Look-Ahead LR) is the parser used by **Yacc/Bison** and most practical tools. It has the **power close to CLR(1)** but the **table size of SLR/LR(0)**. It is built by taking the **canonical LR(1) states and merging any two states with the same LR(0) core** (same items ignoring lookaheads), **unioning their lookaheads**.

Why it matters: LALR is the sweet spot: small tables (same state count as LR(0)/SLR) with almost all of LR(1)'s grammar coverage. It's the industry default for generated bottom-up parsers.

Where it is used in real systems:

- **Yacc, Bison, byacc, PLY (Python)** - all LALR by default.
- C, C++, SQL, and countless language/DSL parsers.

Why interviewers ask about it: It's the practically dominant LR variant; understanding the core-merge idea, why it can introduce reduce/reduce (but never shift/reduce) conflicts, and its place in the hierarchy is essential for compiler roles.

2. Core Idea

Intuition: CLR keeps many states that look identical except for lookahead labels - wasteful. LALR merges those, keeping one state per LR(0) core but retaining the union of lookaheads for precise-enough reductions. Usually the merge is lossless; occasionally it introduces a reduce/reduce conflict.

Real-world analogy: Consolidating duplicate spreadsheets that have the same rows but different note columns: you keep one sheet (the core) and merge the notes (lookaheads). Almost always fine; rarely two merged notes contradict (conflict).

Construction options:

1. **Merge method:** build full CLR(1), then merge states with identical cores, unioning lookaheads.
2. **Lookahead-propagation method:** build the LR(0) automaton, then compute lookaheads by propagation (spontaneous generation + propagation) - avoids ever materializing the huge CLR table. This is what real tools do.

Small example: For $S \rightarrow CC$; $C \rightarrow cC \mid d$, CLR produces separate states for "C followed by c/d" vs "C followed by ". **LALR merges them into one state per core with lookaheads {c, d}** appropriately - fewer states, still conflict-free.

3. Important Subtopics

3.1 Core merging

- **What:** Two LR(1) states with identical LR(0) cores become one; lookaheads are unioned.
- **Why it matters:** This is the defining operation that shrinks CLR to LALR.
- **Interview angle:** "How is LALR derived from CLR?"

3.2 Merge-induced reduce/reduce conflicts

- **What:** Merging can union lookaheads so that two completed items now clash on a common lookahead - a reduce/reduce conflict absent in CLR.
- **Why it matters:** The one way LALR is weaker than CLR.
- **Interview angle:** "What kind of conflict can merging introduce, and which can it never introduce?" (Reduce/reduce yes; shift/reduce never.)

3.3 Why merging never adds shift/reduce conflicts

- **What:** Shifts depend only on the core, unchanged by merging; only completed-item lookaheads change.
- **Why it matters:** Bounds the damage of merging.

- **Interview angle:** "Prove merging can't create a shift/reduce conflict."

3.4 Lookahead propagation (efficient construction)

- **What:** Compute which lookaheads are generated spontaneously and which propagate along the LR(0) automaton, filling LALR lookaheads without building CLR.
- **Why it matters:** How Bison actually builds tables efficiently.
- **Interview angle:** "How do real tools build LALR without the full LR(1) automaton?"

4. Real-World Example

Bison compiling a programming-language grammar. By default Bison constructs the LALR(1) automaton via lookahead propagation over the LR(0) states. If your grammar hits a merge-induced reduce/reduce conflict, Bison reports it, and you either refactor the grammar, add precedence declarations, or switch to `%define lr.type canonical-lr` (CLR) / IELR. This everyday workflow - "Bison says 2 reduce/reduce conflicts" - is the LALR core-merge trade-off in action.

5. Diagrams / Mental Models

CLR to LALR:

```
CLR states:  [I: core A, LA {a}]    [J: core A, LA {b}]    (same core A)
              \                      /
              merge (union LA)
LALR state:  [K: core A, LA {a, b}]
```

Conflict possibilities after merge:

```
Shift/Reduce : depends on core only -> NEVER newly created by merge
Reduce/Reduce : depends on merged LAs -> CAN be newly created by merge
```

Hierarchy recap:

```
LR(0) < SLR(1) < LALR(1) < CLR/LR(1)
      same #states -----^          ^ many more states
```

6. Common Interview Questions

Q1. What is an LALR parser?

- Answer: An LR(1) parser built by merging CLR states with identical LR(0) cores, unioning lookaheads - CLR-like power at SLR-like table size.
- Common mistake: equating it with SLR or with full CLR.

Q2. How is LALR related to CLR?

- Answer: LALR = CLR with same-core states merged.

Q3. What conflict can merging introduce? Which can it never introduce?

- Answer: Can introduce reduce/reduce; never shift/reduce.

Q4. Why is LALR preferred in practice?

- Answer: Nearly CLR power with far fewer states (same as LR(0)/SLR).

Q5. Which tools use LALR?

- Answer: Yacc, Bison, PLY, byacc, etc.

Q6. How many states does LALR have vs CLR?

- Answer: Same as LR(0)/SLR (fewer than CLR).

Q7. Is every LALR(1) grammar CLR(1)?

- Answer: Yes - $LALR \subseteq CLR$ in power.

Q8. Is every SLR(1) grammar LALR(1)?

- Answer: Yes - $SLR \subseteq LALR$.

Q9. How do real tools build LALR efficiently?

- Answer: Lookahead propagation over the LR(0) automaton (no full CLR).

Q10. What do you do about a merge-induced reduce/reduce conflict?

- Answer: Refactor the grammar, add precedence/**%prec**, or use canonical LR/IELR.

7. Deep-Dive Questions

D1. Prove merging cannot create a shift/reduce conflict.

- Shift actions come from items with the dot before a terminal, determined entirely by the LR(0) core; merging preserves the core. Only reduce actions (completed items) depend on lookaheads. So a merge can only alter reduce-related decisions => at worst reduce/reduce, never shift/reduce.

D2. Construct/describe an LALR reduce/reduce conflict that CLR avoids.

- Two CLR states with the same core each have completed items **A -> alpha .** and **B -> beta .** with disjoint lookaheads per state (no conflict). After merging, the unioned lookaheads overlap on some terminal, so both reduces apply => reduce/reduce conflict that didn't exist in CLR.

D3. Spontaneous vs propagated lookaheads - what's the difference?

- A lookahead is **generated spontaneously** in a state from the closure computation (independent of other states); it **propagates** if it flows from one item's lookahead to another via goto transitions. Bison seeds spontaneous ones then iterates propagation to a fixed point.

D4. Does LALR ever lose to SLR?

- No - LALR is always at least as powerful as SLR (its lookaheads are subsets of FOLLOW-derived ones, more precise). Every SLR grammar is LALR.

D5. Practical impact of LALR's weakness?

- Very small - most real language grammars are LALR(1); the rare merge conflicts are handled by minor grammar tweaks or precedence directives, so full CLR is seldom needed.

8. Comparison Tables

The full LR hierarchy

Variant	Items	Lookahead	# States	Power	Used by
LR(0)	LR(0)	none	N	weakest	teaching
SLR(1)	LR(0)	FOLLOW(A)	N	moderate	simple tools
LALR(1)	LR(1) merged	union of same-core LAs	N	strong	Yacc/Bison
CLR/LR(1)	LR(1)	exact per state	many (> N)	strongest	advanced/reference

LALR vs CLR

Aspect	LALR(1)	CLR/LR(1)
States	Fewer (merged)	Many
Table size	Small	Large
Reduce/reduce risk	Slightly higher (merge)	Lowest
Shift/reduce risk	Same as CLR	Baseline
Practical use	Default	Rare/special

9. Common Mistakes

- Calling LALR the same as SLR (LALR is more precise) or the same as CLR (LALR merges states).
- Claiming merging can cause shift/reduce conflicts (it can't - only reduce/reduce).
- Thinking LALR has more states than CLR (it has fewer).
- Assuming LALR handles all LR(1) grammars (it handles almost all, not all).
- Believing Bison builds the full CLR table (it uses propagation).

10. Edge Cases / Special Cases

- **Merge-induced reduce/reduce conflict:** the signature LALR weakness.
- **Grammars LR(1) but not LALR(1):** rare but exist.
- **Precedence declarations** resolve many practical conflicts without grammar changes.
- **epsilon-productions:** handled via propagated lookaheads.
- **Very large grammars:** LALR's compactness is a major practical advantage.

11. How to Explain in Interview

"LALR is the parser Yacc and Bison generate. It starts conceptually from canonical LR(1) but merges every pair of states that have the same LR(0) core, unioning their lookaheads - so you get the compact state count of SLR/LR(0) with nearly the full power of LR(1). The one price is that merging can occasionally union lookaheads into a new reduce/reduce conflict; it can never create a shift/reduce conflict, because shifts depend only on the core. In practice tools build it efficiently by propagating lookaheads over the LR(0) automaton rather than materializing the huge CLR table. Almost every real language grammar is LALR(1), which is why it's the default."

12. Quick Revision Notes

- LALR = **CLR with same-core states merged** (lookaheads unioned).
- **Same state count as LR(0)/SLR**, power close to CLR.
- Merge can create **reduce/reduce** conflicts, **never shift/reduce**.
- Built efficiently via **lookahead propagation** (spontaneous + propagated).
- Default in **Yacc/Bison/PLY**.
- Power order: $\text{SLR} \subseteq \text{LALR} \subseteq \text{CLR}$; $\text{LALR} \supseteq \text{SLR}$ always.
- **Trap:** $\text{LALR} \neq \text{SLR}$ and $\neq$ full CLR; more states in CLR, not LALR.

13. Practice Tasks

1. Build the LALR(1) table for $S \rightarrow CC$; $C \rightarrow cC \mid d$ by merging CLR states; parse `cdd`.
2. Show the state-count reduction vs the CLR automaton for that grammar.
3. Explain, on paper, why a shift/reduce conflict can't appear from merging.
4. Take the $L = R$ grammar (non-SLR) and show it's LALR(1).
5. Look up/construct an LR(1)-not-LALR grammar and identify the merge conflict.
6. Run a grammar through Bison and interpret any conflict report.

14. Final Cheat Sheet

- **Core definition:** LR(1) parser with same-core states merged (lookaheads unioned).
- **Why it matters:** Industry default - small tables, near-CLR power (Yacc/Bison).
- **Most asked:** LALR vs SLR vs CLR; merge $\Rightarrow$ reduce/reduce not shift/reduce; why preferred.
- **Common comparison:** LALR vs CLR (states/power); full LR hierarchy.
- **One-line answer:** "LALR merges canonical LR(1) states that share an LR(0) core, giving SLR-sized tables with almost LR(1) power - the Yacc/Bison default - at the cost of rare merge-induced reduce/reduce conflicts."

19. Parser Conflicts

1. Overview

Definition: A **parser conflict** occurs when a parsing table cell would need **more than one action**, so the parser can't decide deterministically. In LR parsing the two kinds are:

- **Shift/Reduce (S/R) conflict:** in some state, the parser could either shift the next token or reduce by a completed rule.
- **Reduce/Reduce (R/R) conflict:** two different rules could both reduce the same stack top.

(In LL parsing the analogous conflicts are **FIRST/FIRST** and **FIRST/FOLLOW**, i.e., a table cell with two productions.)

Why it matters: Conflicts are the practical face of ambiguity and grammar-class limits. Every time Bison prints "shift/reduce conflict", you're seeing this. Resolving them correctly is a core parser-engineering skill.

Where it is used in real systems:

- Bison/Yacc conflict reports; grammar debugging.
- Dangling-else resolution, operator precedence handling.
- Deciding whether a grammar fits SLR/LALR/CLR.

Why interviewers ask about it: Conflicts tie together ambiguity, FIRST/FOLLOW, LR items, and precedence - a compact way to test whether you can diagnose and fix real grammar problems.

2. Core Idea

Intuition: A conflict means "one lookahead token isn't enough to choose" in this parser construction. It may reflect true ambiguity (dangling-else), a too-weak parser class (SLR when you need LALR), or a genuine grammar bug. Resolution = give the parser a rule to break the tie (precedence, grammar rewrite, or a stronger construction).

Real-world analogy: A traffic intersection with two green lights (two valid actions) - someone must impose a rule (right-of-way / precedence) or redesign the junction (grammar) to avoid a crash.

Small examples:

- **S/R (dangling-else):** state has `stmt -> if e then stmt .` (reduce) and can also shift `else`. Resolve by shifting.
- **S/R (operators):** `E -> E + E .` with lookahead `*` - reduce (do `+` first) or shift `*`? Resolve by precedence (`*` $>$ `+` $\Rightarrow$ shift).
- **R/R:** stack top `id` where both `A -> id` and `B -> id` complete - two reduces.

3. Important Subtopics

3.1 Shift/Reduce conflicts

- **What:** Shift vs reduce both valid; default resolution in Yacc/Bison is **shift**.
- **Why it matters:** Common and often benign (dangling-else, operators).
- **Interview angle:** "Give an S/R conflict and how it's resolved."

3.2 Reduce/Reduce conflicts

- **What:** Two rules reduce the same handle; default resolution is the **rule listed first** in the grammar.
- **Why it matters:** Usually a real ambiguity/design flaw; more serious.
- **Interview angle:** "What typically causes an R/R conflict?"

3.3 Precedence and associativity resolution

- **What:** `%left`, `%right`, `%nonassoc` and `%prec` assign precedence to tokens/rules to auto-resolve S/R conflicts.

- **Why it matters:** The clean way to keep an ambiguous-but-convenient expression grammar.
- **Interview angle:** "How do precedence declarations resolve conflicts?"

3.4 LL conflicts (FIRST/FIRST, FIRST/FOLLOW)

- **What:** Top-down analogues: overlapping FIRST sets, or nullable production whose FOLLOW overlaps a sibling's FIRST.
- **Why it matters:** Same "two entries in a cell" problem, LL side.
- **Interview angle:** "What are the LL(1) conflict types?"

4. Real-World Example

Building a language grammar in Bison. You compile and Bison reports "1 shift/reduce conflict" from the if-else rule and "0 reduce/reduce". You inspect the `.output` file, confirm it's the benign dangling-else (resolved by shift = attach `else` to nearest `if`), and leave it or silence it with `%expect 1`. Later you add a new operator and get a genuine S/R conflict on precedence; you fix it with `%left '+' '-'` and `%left '*' '/'`. This is the day-to-day reality of conflict handling.

5. Diagrams / Mental Models

Conflict decision:

```
Table cell needs 2 actions?
  shift vs reduce -> Shift/Reduce conflict   (default: SHIFT)
  reduce vs reduce -> Reduce/Reduce conflict (default: earlier rule)
```

Precedence resolution of `E + E . *`:

```
compare prec(lookahead '*') vs prec(rule 'E -> E + E')
  '*' higher -> SHIFT (compute * first)    [correct]
  '+' higher -> REDUCE
  equal + left-assoc -> REDUCE ; right-assoc -> SHIFT
```

LL vs LR conflict names:

```
LR:  Shift/Reduce , Reduce/Reduce
LL:  FIRST/FIRST  , FIRST/FOLLOW
```

6. Common Interview Questions

Q1. What is a parser conflict?

- Answer: A table cell requiring more than one action; the parser can't choose deterministically.
- Common mistake: describing only ambiguity, not the table-cell view.

Q2. Name the two LR conflict types.

- Answer: Shift/reduce and reduce/reduce.

Q3. How does Yacc/Bison resolve each by default?

- Answer: S/R => shift; R/R => the earlier-listed rule. (With warnings.)

Q4. Give a classic shift/reduce conflict.

- Answer: Dangling-else, or $E \rightarrow E + E$ with lookahead $*$.

Q5. What usually causes a reduce/reduce conflict?

- Answer: Two productions with overlapping handles/lookaheads - often a real ambiguity or over-general rules.

Q6. How do precedence/associativity declarations resolve conflicts?

- Answer: They compare the precedence of the lookahead token vs the rule to pick shift or reduce; associativity breaks ties.

Q7. What are the LL(1) conflict types?

- Answer: FIRST/FIRST (overlapping FIRST) and FIRST/FOLLOW (nullable production vs FOLLOW).

Q8. Is a shift/reduce conflict always a bug?

- Answer: No - dangling-else is a benign, intentionally shift-resolved conflict.

Q9. Which conflict type is generally more serious?

- Answer: Reduce/reduce (usually a genuine grammar defect).

Q10. How can changing the parser class remove a conflict?

- Answer: SLR conflicts may vanish under LALR/CLR (more precise lookahead); some S/R vanish with more context.

7. Deep-Dive Questions

D1. Why does Bison default to shift for S/R conflicts?

- Because it matches the most common intended semantics (e.g., dangling-else attaching to the nearest `if`, and longest-match behavior). It's a pragmatic default, not a correctness guarantee.

D2. How exactly do `%left/%right/%nonassoc` change table construction?

- Each terminal gets a precedence level and associativity; each rule inherits the precedence of its last terminal (or `%prec`). At an S/R conflict, compare token vs rule precedence: higher token => shift, higher rule => reduce, equal => associativity decides (left=>reduce, right=>shift, nonassoc=>error).

D3. Can precedence declarations resolve reduce/reduce conflicts?

- Not directly/reliably - precedence is designed for S/R. R/R usually needs grammar restructuring; Bison's default (first rule) can silently pick the wrong one.

D4. How do you debug a conflict in Bison?

- Generate the `.output/-v` report, find the state and items involved, read which lookahead triggers the two actions, and decide: precedence directive, grammar rewrite, or accept the default (with `%expect`).

D5. Relationship between conflicts and the grammar hierarchy?

- A grammar with unavoidable conflicts under LALR but not under CLR is "LR(1) but not LALR". Conflicts present in CLR itself indicate the grammar isn't LR(1) (often truly ambiguous).

8. Comparison Tables

Shift/Reduce vs Reduce/Reduce

Aspect	Shift/Reduce	Reduce/Reduce
Choice	Shift or reduce	Two reduces
Common cause	Dangling-else, operator precedence	Overlapping rules, ambiguity
Default (Bison)	Shift	First-listed rule
Severity	Often benign	Usually serious
Fix	Precedence/associativity, rewrite	Grammar restructuring

LR conflicts vs LL conflicts

Parser	Conflict types	Typical fix
LR (bottom-up)	Shift/Reduce, Reduce/Reduce	Precedence, rewrite, stronger class
LL (top-down)	FIRST/FIRST, FIRST/FOLLOW	Left factoring, grammar redesign

9. Common Mistakes

- Treating Bison's default resolution as "no problem" without checking it's the intended one.
- Trying to fix reduce/reduce with precedence declarations (meant for shift/reduce).
- Confusing LL conflicts (FIRST/FIRST, FIRST/FOLLOW) with LR conflicts.
- Assuming any conflict means the grammar is ambiguous (may just be non-SLR/non-LALR).
- Silencing conflicts with `%expect` without understanding them.

10. Edge Cases / Special Cases

- **Dangling-else:** intentional S/R, resolved by shift.
- **%nonassoc:** turns an S/R into a syntax error for e.g. `a < b < c`.
- **Merge-induced R/R:** appears in LALR but not CLR.
- **epsilon-productions:** can create subtle R/R via nullable rules.
- **Precedence on unary minus:** handled with `%prec UMINUS`.

11. How to Explain in Interview

"A parser conflict is when the parse table would need two actions in one cell, so the parser can't decide with its available lookahead. In LR parsing there are two kinds: a shift/reduce conflict, where it could shift the next token or reduce a completed rule - like the dangling-else, which tools resolve by shifting - and a reduce/reduce conflict, where two rules could reduce the same handle, usually signaling a real ambiguity. We resolve shift/reduce conflicts cleanly with precedence and associativity declarations (`%left, %right, %prec`), reduce/reduce ones by restructuring the grammar. In LL parsing the analogues are FIRST/FIRST and FIRST/FOLLOW conflicts, fixed by left factoring or redesign."

12. Quick Revision Notes

- Conflict = table cell needs **>1 action**.
- LR: **Shift/Reduce** (default shift) and **Reduce/Reduce** (default first rule).
- LL: **FIRST/FIRST** and **FIRST/FOLLOW**.
- Resolve S/R with **precedence/associativity** (`%left,%right,%nonassoc,%prec`).
- R/R usually needs **grammar rewrite**; precedence won't reliably fix it.
- Dangling-else = benign S/R; merge => R/R in LALR.
- **Trap**: conflict != always ambiguous (could be class limitation).

13. Practice Tasks

1. Identify and resolve the dangling-else shift/reduce conflict.
2. Use `%left '+' '-' , %left '*' '/'` to resolve the ambiguous expression grammar in Bison.
3. Construct a grammar with a reduce/reduce conflict and explain it.
4. Show a FIRST/FIRST and a FIRST/FOLLOW conflict in an LL(1) grammar.
5. Add `%prec UMINUS` to handle unary minus correctly.
6. Read a Bison `-v` output and locate the conflicting state and items.

14. Final Cheat Sheet

- **Core definition:** A table cell needing more than one action (parser can't decide).
- **Why it matters:** Practical face of ambiguity/grammar-class limits; core debugging skill.
- **Most asked:** S/R vs R/R; default resolutions; precedence resolution; LL analogues.
- **Common comparison:** Shift/Reduce vs Reduce/Reduce; LR vs LL conflicts.
- **One-line answer:** "A parser conflict is a table cell needing two actions - shift/reduce (default shift) or reduce/reduce (default first rule) in LR, FIRST/FIRST or FIRST/FOLLOW in LL - fixed by precedence, grammar rewrite, or a stronger parser class."

20. Yacc / Bison Basics

1. Overview

Definition: **Yacc** ("Yet Another Compiler-Compiler") and its GNU successor **Bison** are **parser generators**: you write a **context-free grammar with embedded semantic actions**, and they generate a **C (LALR(1)) bottom-up parser** (function `yyparse()`). They are almost always paired with a **lexer generator (Lex/Flex)** that produces `yylex()` to supply tokens.

Why it matters: They let you build real, efficient parsers from a declarative grammar instead of hand-coding tables. Historically and today, huge amounts of compiler/interpreter/DSL front-end code is Yacc/Bison-generated.

Where it is used in real systems:

- Early C compilers, many SQL parsers (MySQL historically), PHP, Ruby (parse.y), Bash, PostgreSQL.
- DSLs, config languages, calculators, protocol parsers.

Why interviewers ask about it: It's the practical embodiment of everything above (CFG, LALR, conflicts, precedence). Knowing the file structure, the Lex/Yacc pipeline, and how conflicts/precedence work shows you can *ship* a parser.

2. Core Idea

Intuition: Declare tokens and grammar rules; attach C code (semantic actions) that runs on each **reduce**. Bison builds the LALR automaton and ACTION/GOTO tables; the generated `yyparse()` runs the shift-reduce loop, calling `yylex()` for tokens and executing your action code to build an AST or compute values.

Real-world analogy: A form-letter generator: you supply the template (grammar) with fill-in slots (actions); the tool produces the machine (parser) that fills them as input arrives.

Yacc/Bison file structure (three sections separated by `%%`):

```
%{ C declarations / includes %}
%token NUMBER
%left '+' '-'
%left '*' '/'
%%
expr : expr '+' expr    { $$ = $1 + $3; }
    | expr '*' expr     { $$ = $1 * $3; }
    | '(' expr ')'       { $$ = $2; }
    | NUMBER             { $$ = $1; }
    ;
%%
int main(){ return yyparse(); }
int yyerror(char*s){ fprintf(stderr,"%s\n",s); return 0; }
```

The pipeline:

```
grammar.y --bison--> parser (yyparse) \
lexer.l   --flex --> scanner (yylex)   >--- compiled together --> executable
```

3. Important Subtopics

3.1 File sections and `$$/$1` semantics

- **What:** Sections: declarations, rules, C code. In actions, `$$` = LHS value, `$1..$n` = RHS symbols' values.

- **Why it matters:** This is how you compute/build the AST during reduces.
- **Interview angle:** "What do `$$` and `$1` mean?"

3.2 Tokens and the Lex/Yacc interface

- **What:** `%token` declares terminals; the lexer returns them via `yylex()`, passing values in `yyval`.
- **Why it matters:** Parser and lexer must agree on token codes.
- **Interview angle:** "How do Flex and Bison communicate?"

3.3 Precedence and associativity directives

- **What:** `%left`, `%right`, `%nonassoc` set precedence (lower line = higher precedence); `%prec` overrides a rule's precedence.
- **Why it matters:** Resolves shift/reduce conflicts (operators, unary minus) without rewriting the grammar.
- **Interview angle:** "How do you make `*` bind tighter than `+` in Bison?"

3.4 Conflict reporting and resolution

- **What:** Bison reports S/R and R/R conflict counts; `-v` produces a `.output` file with states/items; defaults: shift, first rule.
- **Why it matters:** Everyday grammar debugging.
- **Interview angle:** "How do you diagnose a Bison conflict?"

3.5 Error handling (`error` token, `yyerror`)

- **What:** The special `error` token enables recovery; `yyerror()` reports messages.
- **Why it matters:** Real parsers need graceful error reporting/recovery.
- **Interview angle:** "How does Yacc do error recovery?"

4. Real-World Example

PostgreSQL's SQL parser (`gram.y`). PostgreSQL uses a Bison grammar (thousands of rules) plus a Flex lexer to parse SQL into parse-tree nodes; semantic actions in the `.y` file allocate C structs for each construct. Precedence declarations resolve operator conflicts; the LALR tables keep parsing linear. This is a production, battle-tested example of exactly the Yacc/Bison workflow described here.

5. Diagrams / Mental Models

Three-section layout:

```
+-----+
|  %{ C decls %}  %token... |  <- declarations (tokens, precedence)
+-----+
|  %%              |
|  rules with { actions }  |  <- grammar + semantic actions
+-----+
|  %%              |
```

```
| C support code (main,...) | <- user C code
+-----+
```

Value flow in a reduce `expr: expr '+' expr:`

```
    $1      $2      $3      ->  $$
(expr)    '+'    (expr)      $$ = $1 + $3
```

Precedence (top = lowest):

```
%left  '+' '-'      (lowest precedence)
%left  '*' '/'
%right '^'          (highest, right-assoc)
```

6. Common Interview Questions

Q1. What are Yacc and Bison?

- Answer: Parser generators that produce an LALR(1) bottom-up parser (`yyparse`) from a CFG with semantic actions.
- Common mistake: saying they generate top-down/LL parsers.

Q2. What parsing method does Yacc/Bison use?

- Answer: LALR(1), shift-reduce, bottom-up.

Q3. Describe the three sections of a `.y` file.

- Answer: Declarations (`%{...%}`, `%token`, precedence), rules with actions, and user C code, separated by `%%`.

Q4. What do `$$`, `$1`, `$3` mean?

- Answer: `$$` is the LHS's semantic value; `$1..$n` are the RHS symbols' values.

Q5. How do Flex and Bison work together?

- Answer: Bison calls `yyllex()` (from Flex) for tokens; values pass via `yylval`; token codes are shared via the generated header.

Q6. How do you resolve operator precedence conflicts?

- Answer: `%left/%right/%nonassoc` (order sets precedence) and `%prec` for rule overrides.

Q7. How does Bison resolve shift/reduce and reduce/reduce by default?

- Answer: Shift; and the earliest-listed rule, respectively (with warnings).

Q8. What is the `error` token?

- Answer: A special token enabling error recovery - the parser discards input until it can shift **error** and resync.

Q9. Why choose LALR (not LL) for Yacc?

- Answer: Broader grammar coverage, handles left recursion, precedence directives - practical for real languages.

Q10. When do semantic actions run?

- Answer: On reductions (mostly), following the bottom-up order.

7. Deep-Dive Questions

D1. Why can Bison handle left-recursive grammars, and why is left recursion actually preferred?

- Because it's bottom-up (reduces after seeing the RHS). Left recursion keeps the parser stack shallow (iterative reduction) versus right recursion which grows the stack with input size - Bison docs recommend left recursion.

D2. How does **%prec** fix unary minus?

- A rule like `expr: '-' expr %prec UMINUS` gives that specific rule the (high) precedence of the pseudo-token **UMINUS**, overriding the low precedence of binary `-`, so `-a*b` parses as `(-a)*b` correctly.

D3. What's in the `-v .output` file and how do you use it?

- The full state machine: each state's items, transitions, and the exact conflict (which lookahead, which two actions). You use it to decide precedence vs grammar change.

D4. Mid-rule actions - what are the pitfalls?

- Actions embedded before the end of a rule (`a { ... } b`) introduce an implicit empty non-terminal, which can create new conflicts; they run partway through and can surprise value numbering.

D5. GLR mode in Bison - when and why?

- **%glr-parser** switches to Generalized LR to handle nondeterministic/ambiguous grammars by forking parses; used when a natural grammar can't be made conflict-free LALR (e.g., C++ type-vs-expression ambiguities).

8. Comparison Tables

Yacc/Bison (LALR) vs ANTLR (LL(*))

Aspect	Yacc/Bison	ANTLR
Direction	Bottom-up LALR(1)	Top-down LL(*)
Left recursion	Native (preferred)	Handled (auto-rewrite)
Actions run	On reduce	During descent
Output language	C/C++	Java, C#, Python, ...

Aspect	Yacc/Bison	ANTLR
Error messages	Terser	Richer

Lex/Flex vs Yacc/Bison

Tool	Role	Generates	Based on
Lex/Flex	Lexer	<code>yyllex()</code>	Regular expressions / DFA
Yacc/Bison	Parser	<code>yyparse()</code>	CFG / LALR(1)

9. Common Mistakes

- Saying Yacc generates a top-down/LL parser (it's LALR bottom-up).
- Forgetting the `%%` separators or misordering the three sections.
- Mismatched token definitions between Flex and Bison.
- Ignoring conflict warnings / silencing them without understanding.
- Using right recursion for long lists (stack growth) instead of left recursion.
- Wrong `$n` indices after adding/removing RHS symbols or mid-rule actions.

10. Edge Cases / Special Cases

- **Dangling-else:** default shift; or `%nonassoc` / grammar rewrite.
- **Unary minus:** `%prec UMINUS`.
- `%expect N`: silence N known shift/reduce conflicts.
- **Reentrancy:** `%define api.pure` for thread-safe parsers.
- **GLR mode:** for genuinely ambiguous grammars.
- **Mid-rule actions:** hidden non-terminals and possible new conflicts.

11. How to Explain in Interview

"Yacc and its GNU version Bison are parser generators: you write a context-free grammar with C semantic actions, and they emit a bottom-up LALR(1) parser, `yyparse()`. The `.y` file has three `%%`-separated sections - declarations (tokens, precedence), grammar rules with actions using `$$` for the left side and `$1..$n` for the right side, and user C code. It pairs with Flex, which generates the `yyllex()` lexer that feeds tokens. Operator conflicts are resolved declaratively with `%left/%right/%nonassoc` and `%prec`, and Bison reports shift/reduce and reduce/reduce conflicts, defaulting to shift and the first rule. It's chosen over LL tools because it handles left-recursive, complex grammars like C and SQL directly."

12. Quick Revision Notes

- Yacc/Bison generate **LALR(1) bottom-up** parsers (`yyparse`).
- File: 3 sections split by `%%` - declarations, rules+actions, C code.
- `$$` = LHS value, `$1..$n` = RHS values (actions run on reduce).
- Pairs with **Lex/Flex** (`yyllex`, `yylval`) for tokens.
- Precedence: `%left/%right/%nonassoc` (later line = higher), `%prec` overrides.
- Conflicts default: **shift, first rule**; inspect with `-v .output`.
- **Prefer left recursion** (shallow stack); `error` token + `yyerror` for recovery.

- **Trap:** it's bottom-up LALR, not LL.

13. Practice Tasks

1. Write a Bison + Flex calculator for `+ - * / ( )` with correct precedence.
2. Add unary minus with `%prec UMINUS` and test `-3 * 2`.
3. Introduce the dangling-else and observe/resolve the shift/reduce conflict.
4. Rewrite a right-recursive list rule as left-recursive and note stack behavior.
5. Trigger a reduce/reduce conflict and read the `-v` output to diagnose it.
6. Add error recovery with the `error` token to skip to the next `;`.

14. Final Cheat Sheet

- **Core definition:** Parser generators producing LALR(1) bottom-up C parsers from a CFG + actions.
- **Why it matters:** Standard way to build real, efficient parsers (C, SQL, DSLs).
- **Most asked:** File structure; `$$/$1`; LALR; precedence directives; conflicts.
- **Common comparison:** Yacc/Bison vs ANTLR; Lex vs Yacc.
- **One-line answer:** "Yacc/Bison turn a CFG with semantic actions into a bottom-up LALR(1) parser (`yyparse`), pairing with Lex/Flex for tokens and resolving operator conflicts via precedence declarations."

21. Generalized LR (GLR) Parsing

1. Overview

Definition: Generalized LR (GLR) parsing extends standard LR parsing to handle **nondeterministic and ambiguous grammars**. When the LR table has a **conflict** (multiple actions), a GLR parser doesn't pick one - it **forks**, pursuing **all possibilities in parallel** using a **Graph-Structured Stack (GSS)**. Parses that hit dead ends are pruned; surviving parses represent all valid interpretations (a **parse forest**).

Why it matters: Some real languages (C++, natural language) have grammars that **cannot** be made conflict-free LALR/LR(1) without painful rewrites. GLR parses **any** context-free grammar, returning all parses, and runs efficiently (near-linear) on the mostly-deterministic parts.

Where it is used in real systems:

- **C++ front-ends** (Elsa, GCC historically explored it) for the type-vs-expression ambiguity.
- **Natural language processing** (ambiguous human-language grammars).
- Bison's `%glr-parser` mode; the **Tree-sitter** editor-parsing library uses a GLR-style algorithm.

Why interviewers ask about it: It's the advanced frontier - shows you understand LR's limits and how to go beyond deterministic parsing; strong differentiator for compiler/tooling roles.

2. Core Idea

Intuition: Run an LR parser, but whenever the table says "two actions" (a conflict) or "no single choice," **split** the parser into multiple logical stacks and continue each. Share common stack parts via a graph (GSS) so it

doesn't blow up exponentially. At the end, keep the stacks that consumed all input and reduced to the start symbol.

Real-world analogy: Exploring a maze with conflicting signs by **cloning yourself at each fork** and walking every path simultaneously, while sharing already-walked corridors to save effort. Dead ends disappear; successful walkers report their routes.

Key mechanisms:

- **Graph-Structured Stack (GSS):** merges shared stack prefixes/suffixes so parallel parses share memory.
- **Parse forest (SPPF):** a compact shared representation of *all* parse trees for ambiguous input.
- **Local nondeterminism:** forks only where conflicts occur; deterministic regions stay single-threaded (fast).

Small example (conceptual): For the ambiguous $E \rightarrow E + E \mid E * E$ on `id on id + id * id`, a plain LALR needs precedence to avoid conflicts. A GLR parser without precedence would fork at the S/R conflict and produce **both** parse trees $(id+id)*id$ and $id+(id*id)$ in the parse forest.

3. Important Subtopics

3.1 Graph-Structured Stack (GSS)

- **What:** A DAG replacing the single stack; nodes shared across parallel parses to avoid duplication.
- **Why it matters:** Keeps GLR efficient (polynomial, near-linear on deterministic input) instead of exponential.
- **Interview angle:** "How does GLR avoid exponential blowup?"

3.2 Parse forest (SPPF)

- **What:** A **Shared Packed Parse Forest** compactly encodes all parse trees of an ambiguous input.
- **Why it matters:** Lets you return every interpretation without exponential memory.
- **Interview angle:** "How does GLR represent multiple parses?"

3.3 Handling conflicts by forking

- **What:** At each conflicting table cell, spawn a parse per action; prune the ones that fail.
- **Why it matters:** This is the essence of "generalized" - conflicts become branches, not errors.
- **Interview angle:** "What does GLR do at a shift/reduce conflict?" (Do both, in parallel.)

3.4 Performance profile

- **What:** $O(n)$ on deterministic (LR) parts; up to $O(n^3)$ worst case for highly ambiguous grammars.
- **Why it matters:** Practical because real grammars are mostly deterministic.
- **Interview angle:** "What's GLR's time complexity?"

4. Real-World Example

Tree-sitter (used in editors like Neovim, GitHub code navigation). Tree-sitter parses source code incrementally with a GLR-style algorithm so it can handle the many locally-ambiguous, real-world grammars of dozens of languages and re-parse quickly on each keystroke. It stays near-linear because code is mostly

deterministic, forking only at genuine ambiguities. This gives editors fast, error-tolerant syntax trees for highlighting and navigation - a flagship modern use of generalized parsing.

5. Diagrams / Mental Models

Forking at a conflict:

```

... single stack ...
      |
conflict (shift AND reduce)
    /   \
[shift path] [reduce path]    <- both pursued
    \       /
merged again via GSS (shared nodes)

```

Graph-Structured Stack vs plain stack:

```

Plain LR stack:  s0 - s1 - s2          (one path)
GSS:             s0 - s1 < s3          (branches share s0,s1)
                  \ s2 /

```

Deterministic vs ambiguous cost:

```

deterministic input -> behaves like LR -> O(n)
highly ambiguous    -> many forks      -> up to O(n^3)

```

6. Common Interview Questions

Q1. What is GLR parsing?

- Answer: An extension of LR that handles nondeterministic/ambiguous grammars by forking at conflicts and exploring all parses in parallel via a graph-structured stack.
- Common mistake: describing it as "just LR with backtracking" (it's parallel, shared, not sequential undo).

Q2. How does GLR handle a table conflict?

- Answer: It splits into multiple parses (one per action) and prunes the ones that fail.

Q3. What is a Graph-Structured Stack?

- Answer: A DAG that lets parallel parses share common stack portions, preventing exponential blowup.

Q4. How are multiple parse results represented?

- Answer: As a shared packed parse forest (SPPF).

Q5. What is GLR's time complexity?

- Answer: Linear on deterministic input; up to cubic ($O(n^3)$) worst case for ambiguous grammars.

Q6. Why use GLR over LALR?

- Answer: To parse grammars that can't be made conflict-free (C++, natural language) and to get all interpretations.

Q7. Name real systems using GLR.

- Answer: Bison `%glr-parser`, Tree-sitter, C++ parsers, NLP toolkits.

Q8. Does GLR use backtracking?

- Answer: No - it explores branches simultaneously with sharing, not sequential undo.

Q9. Can GLR parse any CFG?

- Answer: Yes - any context-free grammar, ambiguous or not.

Q10. What's the downside of GLR?

- Answer: More complex, higher memory, potential ambiguity to disambiguate afterward, worst-case cubic time.

7. Deep-Dive Questions

D1. How does the GSS keep worst-case cost polynomial?

- By merging stack nodes that reach the same parser state at the same input position, the number of distinct nodes/edges is bounded polynomially in input length, so the parallel parses share work instead of multiplying it.

D2. GLR vs Earley parsing - both handle all CFGs; how differ?

- Both parse any CFG. GLR is table-driven (LR automaton) and near-linear on deterministic grammars; Earley uses dynamic-programming chart parsing, $O(n^3)$ general / $O(n^2)$ unambiguous / $O(n)$ for many grammars, and needs no LR tables. GLR reuses LR technology; Earley is more uniform.

D3. How do you disambiguate a GLR parse forest?

- Apply post-parse rules: priorities/precedence, semantic filters (e.g., "is this identifier a type?"), or user-supplied disambiguation callbacks (Bison lets you merge or choose among ambiguous reductions).

D4. Why is GLR a good fit for incremental editor parsing (Tree-sitter)?

- Real code is mostly deterministic (fast LR path), but editors need tolerance for locally ambiguous or incomplete input; GLR forks only where needed and the GSS supports efficient re-use, enabling fast incremental re-parsing.

D5. When would GLR degrade badly, and how do you mitigate?

- Pathologically ambiguous grammars (many forks everywhere) approach $O(n^3)$ and large forests. Mitigate by tightening the grammar, adding precedence/priorities, or applying semantic disambiguation to prune early.

8. Comparison Tables

GLR vs LALR/LR(1)

Aspect	LALR/LR(1)	GLR
Grammar class	Deterministic (LR)	Any CFG (incl. ambiguous)
On conflict	Error / forced choice	Fork and pursue all
Stack	Single	Graph-structured (shared)
Output	One tree	Parse forest (SPPF)
Time	$O(n)$	$O(n)$ det., up to $O(n^3)$
Complexity/memory	Lower	Higher

GLR vs Earley vs Backtracking

Aspect	GLR	Earley	Backtracking RD
Any CFG	Yes	Yes	PEG-limited
Deterministic speed	$O(n)$	often $O(n)$	varies
Worst case	$O(n^3)$	$O(n^3)$	exponential
Mechanism	LR + GSS	chart / DP	try+undo

9. Common Mistakes

- Describing GLR as LR "with backtracking" (it's parallel with sharing, not undo).
- Thinking GLR is always slow (it's linear on deterministic input).
- Forgetting it can produce multiple parses that still need disambiguation.
- Assuming GLR removes ambiguity (it surfaces all parses; you disambiguate after).
- Ignoring the memory/complexity cost versus LALR.

10. Edge Cases / Special Cases

- **Highly ambiguous grammars:** cubic time, large parse forests.
- **C++ "most vexing parse" / type-vs-expression:** classic GLR use case.
- **Incomplete/erroneous input:** GLR degrades gracefully (good for editors).
- **Disambiguation callbacks:** merging ambiguous reductions in Bison GLR.
- **Deterministic grammar:** GLR behaves exactly like LR (no overhead in practice).

11. How to Explain in Interview

"Generalized LR parsing extends standard LR to handle any context-free grammar, including ambiguous ones. Instead of failing on a table conflict, it forks and pursues every possible action in parallel, using a graph-structured stack so the parallel parses share common portions and don't blow up exponentially. Successful parses are collected into a shared packed parse forest representing all interpretations. It runs in linear time on the deterministic parts of the input and only pays extra where there's genuine ambiguity - worst case cubic.

That's why it's used for hard grammars like C++ and for incremental editor parsing in Tree-sitter, where you need speed on normal code but tolerance for local ambiguity."

12. Quick Revision Notes

- GLR = LR generalized to **any CFG** (ambiguous/nondeterministic).
- On conflict: **fork** and pursue all actions in **parallel** (not backtracking).
- Uses a **Graph-Structured Stack (GSS)** to share work => polynomial, not exponential.
- Produces a **parse forest (SPPF)** of all parses.
- Time: **O(n)** deterministic, up to **O(n³)** ambiguous.
- Used in **Bison %glr-parser**, **Tree-sitter**, **C++ parsers**, **NLP**.
- **Trap**: it surfaces ambiguity (all parses); you still disambiguate afterward.

13. Practice Tasks

1. Take the ambiguous $E \rightarrow E+E \mid E * E \mid id$ and describe how GLR yields both trees for $id+id*id$.
2. Draw a small graph-structured stack showing two parses sharing a prefix.
3. Explain why C++ `A * B;` is ambiguous and how GLR handles it.
4. Compare GLR and Earley on the same ambiguous grammar (complexity, mechanism).
5. In Bison, enable `%glr-parser` on a conflicting grammar and add a disambiguation merge.
6. Argue why Tree-sitter chose a GLR-style algorithm for editors.

14. Final Cheat Sheet

- **Core definition**: LR extended to all CFGs by forking at conflicts and exploring parses in parallel via a graph-structured stack.
- **Why it matters**: Parses ambiguous/hard grammars (C++, NLP) and powers incremental editor parsing.
- **Most asked**: How it handles conflicts; GSS; complexity; vs LALR/Earley.
- **Common comparison**: GLR vs LALR; GLR vs Earley.
- **One-line answer**: "GLR generalizes LR to any context-free grammar by forking at table conflicts and running all parses in parallel over a graph-structured stack - linear on deterministic input, up to cubic when ambiguous, producing a shared parse forest."

22. Master Cheat Sheet (All Topics)

A one-page revision layer over the whole guide. Skim this the night before an interview.

A. The Big Picture

```
tokens --> [ PARSER ] --> parse tree / AST
      |
      +-----+-----+
      |           |
TOP-DOWN (LL)   BOTTOM-UP (LR)
leftmost deriv. rightmost deriv. (reverse)
root -> leaves  leaves -> root
recursive descent shift-reduce
```

LL(1) table	LR(0)/SLR/LALR/CLR, GLR
needs: no left-	handles left recursion,
recursion, left	no factoring needed
factoring	

B. One-line definitions

Topic	One-liner
CFG	4-tuple (V,T,P,S); rules rewrite one non-terminal; Type-2, more powerful than regex.
Parse tree	Tree of a derivation; leaves' yield = input; two trees => ambiguous.
LMD / RMD	Expand leftmost / rightmost non-terminal; same string & tree (if unambiguous).
Ambiguous grammar	Some string has >1 parse tree; grammar property; fix with precedence/associativity.
Left recursion elim.	$A \rightarrow Aa \mid b$ becomes $A \rightarrow bA'$, $A' \rightarrow aA' \mid e$; needed for top-down only.
Left factoring	$A \rightarrow ab1 \mid ab2$ becomes $A \rightarrow aA'$, $A' \rightarrow b1 \mid b2$; for 1-token LL prediction.
Top-down parsing	Root-to-leaves, leftmost; predictive (linear) vs backtracking.
Recursive descent	One function per non-terminal; call stack = parser stack.
Predictive parsing	Non-backtracking top-down using FIRST/FOLLOW; table form = LL(1).
FIRST / FOLLOW	Terminals that can begin alpha (+e) / can follow A (+\$, never e).
LL(1)	Left-to-right, Leftmost, 1 lookahead; table with no conflicting cell.
Bottom-up parsing	Leaves-to-root by reducing handles; rightmost derivation reversed.
Shift-reduce	Stack machine: shift/reduce/accept/error.
Handle / viable prefix	Substring to reduce next / legal LR stack content.
LR parsing	Bottom-up, ACTION+GOTO over item DFA; strongest deterministic.
SLR	LR(0) items + reduce on FOLLOW(A); simplest, FOLLOW too coarse.
CLR/LR(1)	LR(1) items with exact per-item lookahead; most powerful; state explosion.
LALR	CLR states merged by LR(0) core; Yacc/Bison default; small tables.
Parser conflicts	Cell needs 2 actions: shift/reduce, reduce/reduce (LL: FIRST/FIRST, FIRST/FOLLOW).
Yacc/Bison	Generate LALR(1) bottom-up parser from CFG + actions; pair with Lex/Flex.
GLR	LR generalized to any CFG; fork at conflicts; graph-structured stack.

C. FIRST/FOLLOW rules (memorize)

- **Nullable A:** $A \Rightarrow^* e$. Compute first.

- **FIRST(X1..Xk)**: add FIRST(X1){e}; carry to X2 if X1 nullable; ...; add e only if ALL nullable.
- **FOLLOW**: \$ in FOLLOW(start); **A->aBb** adds FIRST(b){e} to FOLLOW(B); if b nullable/absent, add FOLLOW(A) to FOLLOW(B).
- **e never in FOLLOW**. \$ never in FIRST.

D. LL(1) table rule

For **A -> a**alpha: put it in **M[A, a]** for each **a** in FIRST(alpha); if e in FIRST(alpha), also for each **b** in FOLLOW(A).
Two entries in a cell => not LL(1).

E. The LR hierarchy (know cold)

	Items	Reduce lookahead	States	Power
LR(0)	LR(0)	none	N	weakest
SLR(1)	LR(0)	FOLLOW(A)	N	moderate
LALR(1)	merged LR(1)	union of same-core LAs	N	strong (Bison)
CLR/LR(1)	LR(1)	exact	many	strongest

- Power: **LR(0) $\subset$ SLR $\subset$ LALR $\subset$ CLR = LR(1)**.
- **LL(1) $\subset$ LR(1)**. Every LL(1) grammar is LR(1), not vice versa.
- LR(1) = all deterministic CFLs.
- Merging (CLR->LALR) can add **reduce/reduce**, never **shift/reduce**.

F. Which derivation / which method

Method	Derivation	Direction
LL / recursive descent / predictive	Leftmost	Top-down, forward
LR / SLR / LALR / CLR / shift-reduce	Rightmost	Bottom-up, reverse

G. Grammar transformations checklist (for LL)

1. Remove left recursion (immediate + indirect).
2. Left factor common prefixes.
3. Compute nullable, FIRST, FOLLOW.
4. Build LL(1) table; check for conflicts.
5. If conflicts remain => not LL(1); consider LR.

H. Conflict resolution quick guide

Conflict	Default (Bison)	Proper fix
Shift/Reduce	Shift	%left/%right/%nonassoc, %prec, or rewrite
Reduce/Reduce	First rule	Restructure grammar
FIRST/FIRST (LL)	-	Left factoring

Conflict	Default (Bison)	Proper fix
FIRST/FOLLOW (LL)	-	Rewrite / stronger parser

I. Classic example grammars

Ambiguous expression: $E \rightarrow E + E \mid E * E \mid (E) \mid id$ **Unambiguous (precedence + left-assoc):**

```
E -> E + T | T
T -> T * F | F
F -> ( E ) | id
```

LL(1) form (left recursion removed):

```
E -> T E'      E' -> + T E' | e
T -> F T'      T' -> * F T' | e
F -> ( E ) | id
```

$FIRST(E)=FIRST(T)=FIRST(F)=\{ (, id \}$; $FIRST(E')=\{ +, e \}$; $FIRST(T')=\{ *, e \}$ $FOLLOW(E)=FOLLOW(E')=\{), ; \}$
 $FOLLOW(T)=FOLLOW(T')=\{ +,), ; \}$; $FOLLOW(F)=\{ +,), \$ \}$

Dangling-else (ambiguous): $S \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } S \text{ else } S \mid \text{other}$ **Non-SLR but LALR:** $S \rightarrow L = R \mid R$; $L \rightarrow * R \mid id$; $R \rightarrow L$

J. Most-asked interview one-liners

- Why can't regex parse balanced parentheses? -> No stack/recursion; CFG needed.
- Why remove left recursion? -> Top-down parser loops forever otherwise (LR doesn't need it).
- LL(1) vs LR(1)? -> Top-down leftmost, weaker vs bottom-up rightmost, stronger.
- Difference SLR/LALR/CLR? -> Reduce-lookahead precision & state count; LALR = merged CLR.
- What does Yacc use? -> LALR(1) bottom-up.
- Two LR conflict types? -> Shift/reduce (default shift), reduce/reduce (default first rule).
- What is a handle? -> Substring whose reduction reverses a rightmost-derivation step.
- Ambiguity - grammar or language? -> Grammar (except inherently ambiguous languages).
- How does GLR handle conflicts? -> Forks and parses all options via a graph-structured stack.

K. Common traps (don't fall for these)

- "Context-free" != unambiguous.
- Second L in LL(1) = **Leftmost**, not left recursion.
- e can be in FIRST but **never** in FOLLOW.
- Different derivation *order* != ambiguity; different *trees* = ambiguity.
- Left recursion != ambiguity (independent).
- Bison default-resolving a conflict != grammar is correct.
- LALR has **fewer** states than CLR (not more).
- Bottom-up builds **rightmost** derivation (in reverse), not leftmost.

- Removing left recursion + factoring does NOT guarantee LL(1).
-

End of guide. You now have basics-to-interview-depth coverage of syntax analysis: grammars, derivations, ambiguity, the top-down (LL) stack, the bottom-up (LR) stack, conflicts, parser generators, and generalized parsing.